RATKO PILIPOVIĆ

RV32I · 5-STAGE PIPELINE



DESIGNING A RISC-V SoC

COURSE MATERIAL FOR THE SUBJECT DIGITAL DESIGN

Designing a RISC-V SoC

Course material for subject Digital Design

Ratko Pilipović

2026-11-07

Table of contents

Preface 9

1 Introduction to Digital Design 10

Introduction to Digital Design 11

1.1 Goal for this week . . . 11
1.2 The P-N junction . . . 11
1.2.1 The depletion region . . . 13
1.3 CMOS technology . . . 14
1.3.1 MOSFET transistors . . . 15
1.3.2 Complementary MOS . . . 18
1.4 CMOS characteristics . . . 19
1.4.1 Transfer function . . . 19
1.4.2 Noise margins . . . 20
1.4.3 Power . . . 20
1.4.4 Dynamic behavior . . . 21
1.5 Manufacturing CMOS integrated circuits . . . 21
1.5.1 Photolithography process . . . 22

2 Basics of Digital Integrated Circuits 23

Basics of Digital Integrated Circuits 24

2.1 Goal for this week . . . 24
2.2 Digital integrated circuits . . . 24
2.2.1 Scale of integration . . . 24
2.3 Design methodologies . . . 25
2.3.1 Standard integrated circuits . . . 25
2.3.2 Application-specific integrated circuits . . . 25
2.3.3 Building blocks for ASIC design . . . 27
2.4 Standard ASICs (SASIC) . . . 28
2.4.1 Configuration technologies . . . 28
2.4.2 Simple programmable logic devices . . . 29
2.4.3 Complex programmable logic devices . . . 30

3 FPGA Fundamentals 32

FPGA Fundamentals 33
3.1 Goal for this week . . . 33
3.2 Why reconfigurable hardware? . . . 33
3.3 The three building blocks . . . 35
3.3.1 Configurable Logic Block (CLB) . . . 36
3.3.2 Routing plane: programmable interconnect . . . 39
3.3.3 I/O blocks . . . 40
3.3.4 Reconfigurability, level by level . . . 41
3.4 Case study: Xilinx Series-7 . . . 41
3.4.1 CLB: SLICEL and SLICEM . . . 42
3.4.2 Block RAM and DSP slices . . . 44
3.4.3 Clock management and interconnect . . . 44
3.5 FPGA design flow . . . 45

4 SystemVerilog Basics 47

SystemVerilog basics 48
4.1 Goal for this week . . . 48
4.2 Why describe hardware in text . . . 48
4.3 What an HDL is used for . . . 48
4.4 From Verilog to SystemVerilog . . . 49
4.5 Anatomy of a module . . . 50
4.6 Values: 0, 1, X and Z . . . 51
4.7 Data types: nets and variables . . . 51
4.8 Vectors and integer types . . . 52
4.9 Writing constants . . . 53
4.10 Continuous assignment . . . 53
4.11 Not like assignment in C . . . 54
4.12 Two drivers on one net . . . 54
4.13 Operators at a glance . . . 55
4.14 Bitwise vs. logical . . . 55
4.15 Reduction, relational and shift operators . . . 56
4.16 Conditional operator, concatenation and replication . . . 56
4.17 Example: 4-bit 4:1 multiplexer . . . 57
4.18 Example: 4-bit adder . . . 57
4.19 From modules to behavior . . . 59

5 Designing with SystemVerilog 61

Designing with SystemVerilog 62
5.1 Goal for this week . . . 62
5.2 Two styles of design . . . 62

5.3 The `always` block . . . 63
5.4 Sequential logic: describe the hardware you want . . . 63
5.5 The D flip-flop . . . 64
5.6 The latch . . . 65
5.7 Adding a reset . . . 66
5.8 Registers and the enable signal . . . 67
5.9 Counters: two ways to write one . . . 68
5.10 Combinational logic . . . 72
5.11 The multiplexer . . . 72
5.12 The decoder . . . 74
5.13 Avoiding latches . . . 76
5.14 Blocking vs. non-blocking assignment . . . 77
5.14.1 In combinational logic . . . 77
5.14.2 In sequential logic . . . 78
5.14.3 The rule . . . 79
5.15 Structural design: module instances . . . 80
5.15.1 Parameters . . . 81
5.15.2 A real example: the program counter of HIPV . . . 81
5.16 Testbenches . . . 83
5.17 Testbench for a sequential design . . . 84
5.18 Running a simulation . . . 85
5.19 From design to the pipeline . . . 85

6 Finite State Machines 86

Finite State Machines 87
6.1 Goal for this week . . . 87
6.2 Theoretical foundations . . . 87
6.2.1 What is a finite state machine? . . . 87
6.2.2 Describing an FSM: the state diagram . . . 88
6.2.3 The structure of an FSM in hardware . . . 89
6.2.4 Moore and Mealy machines . . . 90
6.2.5 State encoding . . . 91
6.2.6 The ASM chart . . . 92
6.2.7 Describing an FSM in SystemVerilog . . . 93
6.2.8 FSMs with data registers . . . 95
6.2.9 Common mistakes . . . 96
6.3 Practical case: a traffic light . . . 96
6.3.1 Specification . . . 96
6.3.2 System architecture . . . 97
6.3.3 The 1 ms tick generator . . . 97
6.3.4 The ASM chart of the traffic light . . . 98

6.3.5 The FSM in SystemVerilog . . . 99
6.3.6 The traffic light in the waveform . . . 101
6.3.7 Putting it together . . . 102

7 Five-Stage Pipeline — Structure 104

Five-stage pipeline 105
7.1 Goal for this week . . . 105
7.2 Levels of abstraction . . . 105
7.3 System level . . . 106
7.4 Algorithmic level . . . 107
7.5 Register-transfer level . . . 107
7.6 Gate level . . . 108
7.7 Transistor level . . . 109
7.8 Material level . . . 109
7.9 Moving between the levels . . . 109

8 Instruction fetch unit 111

Instruction fetch unit 112
8.1 Goal for this week . . . 112
8.2 Introduction . . . 112
8.3 Interface of the IFU . . . 113
8.3.1 Handshake with Decode . . . 113
8.3.2 Redirect from Execute . . . 114
8.3.3 Stage outputs . . . 114
8.3.4 Port to instruction memory . . . 115
8.4 Structure of the IFU . . . 116
8.4.1 The building blocks . . . 118
8.4.2 The IFU in SystemVerilog . . . 119
8.5 The IFU controller . . . 120
8.5.1 The states . . . 120
8.5.2 The discard flag . . . 121
8.5.3 Transitions . . . 122
8.5.4 The controller in SystemVerilog . . . 123
8.5.5 The outputs . . . 124
8.5.6 The controller in the waveform . . . 125
8.6 Summary . . . 127

9 Instruction decode unit 128

Instruction decode unit 129
9.1 Goal for this week . . . 129

9.2 Introduction . . . 129
9.3 Interface of the IDU . . . 130
9.4 Structure of the IDU . . . 131
9.5 The valid/ready handshake . . . 132
9.5.1 Two signals . . . 132
9.5.2 Four cases . . . 133
9.5.3 Rules of the handshake . . . 134
9.6 Handshake logic of the IDU . . . 134
9.7 Pipeline registers . . . 136
9.8 Instruction fields . . . 137
9.9 The control decoder . . . 138
9.9.1 The HIPV package . . . 138
9.9.2 The control word . . . 140
9.9.3 The main decoder . . . 140
9.9.4 The ALU decoder . . . 141
9.10 Immediate extension . . . 142
9.11 Register file . . . 144
9.11.1 Storage: an array of registers . . . 144
9.11.2 Writing and reading . . . 145
9.12 Summary . . . 146

10 Integer execution unit 147

Integer execution unit 148
10.1 Goal for this week . . . 148
10.2 Introduction . . . 148
10.3 Interface of the IEU . . . 149
10.3.1 Handshake with Decode . . . 149
10.3.2 Signals from Decode . . . 150
10.3.3 Forwarding inputs . . . 150
10.3.4 Redirect to Fetch and Decode . . . 151
10.3.5 Handshake and outputs to Memory . . . 151
10.4 Structure of the IEU . . . 151
10.5 Handshake logic and pipeline registers . . . 151
10.5.1 The D/E pipeline registers . . . 154
10.6 Forwarding . . . 154
10.6.1 Refreshing the operands with forwarded values . . . 156
10.6.2 The refresh loop in SystemVerilog . . . 157
10.7 The ALU . . . 158
10.7.1 Operands of the ALU . . . 159
10.7.2 The ALU in SystemVerilog . . . 160

10.8 Jump mechanism . . . 162
10.8.1 Conditional branches: `branchcmp` . . . 162
10.8.2 Jump targets: `targetadd` and `targetmux` . . . 163
10.8.3 The redirect: `PCSrcE` . . . 163
10.9 Summary . . . 164

11 Load/store unit **165**

Load/store unit **166**
11.1 Goal for this week . . . 166
11.2 Introduction . . . 166
11.3 Interface of the LSU . . . 167
11.4 Structure of the LSU . . . 168
11.5 Handshake logic . . . 169
11.5.1 When is the output valid? . . . 169
11.5.2 When can the stage accept? . . . 170
11.6 E/M pipeline registers . . . 171
11.7 Access to memory . . . 171
11.7.1 Why an FSM? . . . 172
11.7.2 Interface of the controller . . . 173
11.7.3 States and transitions . . . 174
11.7.4 The outputs . . . 175
11.7.5 The controller in the waveform . . . 175
11.8 OBI signals: address, byte enables, write data . . . 176
11.9 Capturing and processing loaded data . . . 178
11.10 Forwarding and outputs to Writeback . . . 179
11.11 Summary . . . 180

12 Writeback unit **181**

Writeback unit **182**
12.1 Goal for this week . . . 182
12.2 Introduction . . . 182
12.3 Interface of the WBU . . . 183
12.4 Structure of the WBU . . . 183
12.5 Handshake control . . . 185
12.6 Pipeline registers . . . 185
12.7 Writeback logic . . . 186
12.7.1 The result multiplexer . . . 186
12.7.2 The register write enable . . . 187
12.7.3 Where the outputs go . . . 188
12.8 Summary . . . 188

13 Pipeline hazards and the HIPV core 189

Pipeline hazards and the HIPV core 190
13.1 Goal for this week . . . 190
13.2 Introduction . . . 190
13.3 Structural hazards . . . 191
13.4 Data hazards . . . 191
13.4.1 The distance decides the remedy . . . 192
13.4.2 The hazard unit . . . 192
13.4.3 The forwarding conditions . . . 194
13.4.4 Loads: no stall unit needed . . . 195
13.5 Control hazards . . . 196
13.6 Summary of hazards . . . 197
13.7 The HIPV core . . . 198
13.7.1 Interface of the CPU . . . 198
13.7.2 Inside the top level . . . 199
13.7.3 How fast is it? . . . 201
13.8 You have come a long way . . . 201
13.9 What comes next . . . 202
13.10 Directions for future work . . . 202

14 Memory — Implementation in SystemVerilog 204

Memory 205

15 Verification with CocoTB 206

Verification with CocoTB 207

References 208

Preface

This material was written alongside the lectures for the course DN, redesigned around the theme of RISC-V SoC design. Each chapter corresponds to one week of lectures and serves a dual purpose:

1. As **lecture notes** — rendered as a slide deck (reveal.js).
2. As a **book chapter** — rendered as part of the combined PDF/HTML course material.

Content intended for only one of the two formats (e.g. discussion questions used in class, or in-depth derivations meant for the book) is marked with `content-visible` blocks — see the example in the chapter on pipeline structure.

> ⚠ Work in progress
>
> This material is a work in progress. It is written and revised alongside the course as it is taught, so chapters vary in polish, and some sections still need further revision. Please read it with that in mind, and feel free to flag anything that is unclear, incomplete, or incorrect.

1 Introduction to Digital Design

Introduction to Digital Design

1.1 Goal for this week

- See why the P-N junction's one-way conduction is the basic building block behind every diode and every transistor
- Follow how a MOSFET's gate voltage creates, and destroys, a conducting channel between source and drain, turning it into a voltage-controlled switch
- Build the CMOS inverter from one NMOS and one PMOS, and see why that pairing is what gives CMOS its low static power
- Separate static and dynamic power, and see which design choices trade one for the other
- Connect the transistor-level picture to the chip-level one: how NMOS and PMOS actually end up side by side on the same piece of silicon, and roughly how a wafer becomes a chip

1.2 The P-N junction

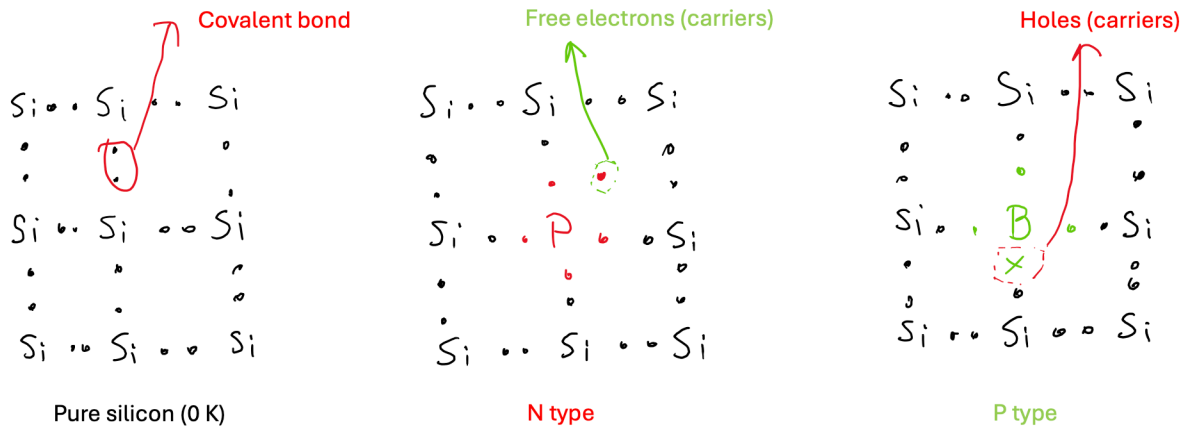


Figure 1.1: N-type and P-type silicon: doping replaces some silicon atoms with impurities that donate either a free electron or a hole

One of the fundamental facts of solid-state electronics is how a P-N junction behaves. On its own, pure silicon is a poor conductor. We fix that through doping, deliberately introducing a small number of impurity atoms into the silicon lattice to change how many free charge carriers it has. Dope silicon with an element that has more valence electrons than silicon, such as phosphorus or arsenic, and the extra electrons become free carriers. We call the result n-type silicon, “n” for the negative charge of its majority carrier. Dope it instead with an element that has fewer valence electrons, such as boron or gallium, and the missing electron leaves behind a “hole”, a position that behaves like a mobile positive charge. We call that p-type silicon.

Bring a block of n-type and a block of p-type silicon into contact, and the junction between them behaves in a way neither material does alone. Current flows easily in one direction, forward bias, and is almost completely blocked in the other, reverse bias. This one-way behavior is not a separate property we add. It falls directly out of how charge moves across the junction, and it is the basic mechanism behind every diode, and in turn, every transistor we build in this course.

1.2.1 The depletion region

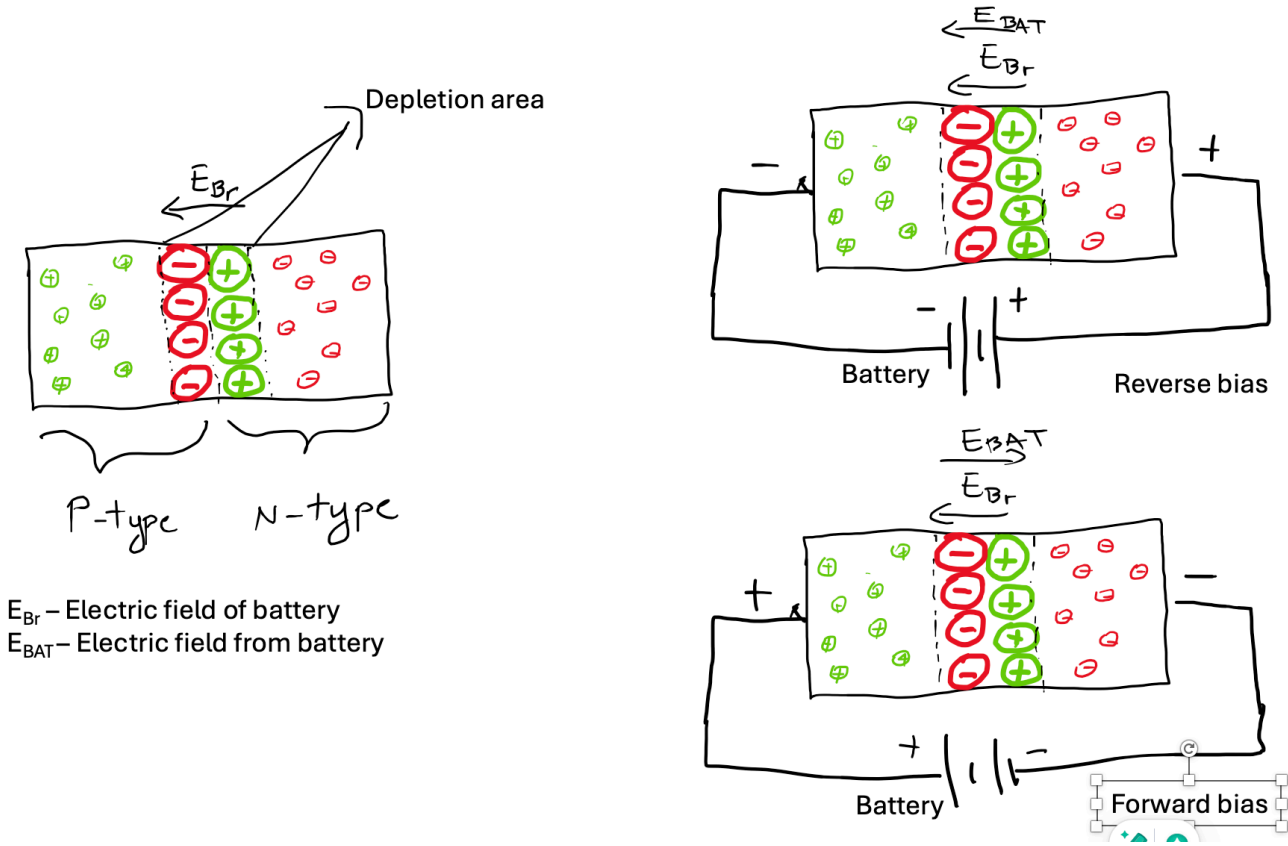


Figure 1.2: The depletion region at a P-N junction, and how reverse bias and forward bias each move the electric field relative to the battery's field

Right at the junction, free electrons from the n-side and free holes from the p-side meet and recombine, leaving behind a thin region with no free carriers at all. We call this the depletion region. What is left behind there is not neutral. It is the fixed, immobile dopant ions that donated those carriers in the first place, and those fixed charges set up an electric field across the depletion region, pointing from the n-side toward the p-side.

1.2.1.1 Equilibrium

With no external voltage applied, that built-in field reaches a balance point. It is strong enough to stop any further electrons or holes from diffusing across the junction, but no stronger. We call this the equilibrium state. Electrically, it behaves like a small built-in

barrier, often called the Coulomb barrier, that a carrier has to overcome before it can cross from one side to the other.

1.2.1.2 Reverse bias

Apply an external voltage with its positive terminal on the n-side, and that external field points the same way as the junction's built-in field. The two add together, the depletion region widens, and the barrier an electron would need to cross gets taller, not shorter. For current to flow at all, electrons from the n-side would have to reach the junction and recombine with holes on the p-side, and a reverse-biased junction actively drives electrons away from the junction instead. The result is that almost no current flows.

1.2.1.3 Forward bias

Flip the polarity, and the external field now opposes the built-in field instead of reinforcing it. The depletion region shrinks, the Coulomb barrier collapses, and carriers can cross the junction with very little resistance. This is forward bias, and it is the conducting state of a diode.

1.3 CMOS technology

Complementary Metal-Oxide-Semiconductor (CMOS) technology is, practically speaking, unavoidable in digital design. It builds every logic function from paired n-type and p-type MOSFETs, and that pairing is what lets it combine very low static power with high noise immunity and fast switching, all at once. That combination is exactly what large-scale integration needs, whether that means a battery-powered microcontroller or the millions of logic gates on a modern microprocessor. Every digital building block we design in this course, down to the CLB we met in the FPGA fabric, ultimately bottoms out in CMOS transistors like the ones in this chapter.

1.3.1 MOSFET transistors

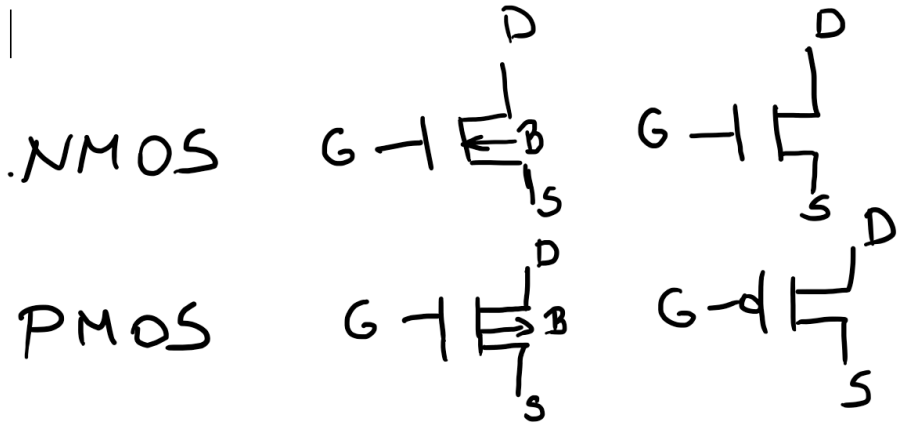


Figure 1.3: NMOS and PMOS schematic symbols: the four-terminal version shows the body connection explicitly, the three-terminal version assumes it

We will mostly use the MOSFET as a switch, though it can also amplify signals. At first glance it looks like a four-terminal device: gate, source, drain, and body. A voltage on the gate decides whether, and how strongly, current can flow between source and drain. The body terminal shifts the device's voltage-current characteristics and is normally just tied to a fixed supply rail, which is why many schematic symbols leave it out and show only three terminals.

The name spells out what the device is built from. "Metal-Oxide- Semiconductor" describes its physical structure. "Field-Effect Transistor" describes how it works: an electric field, not a direct physical connection, controls the current.

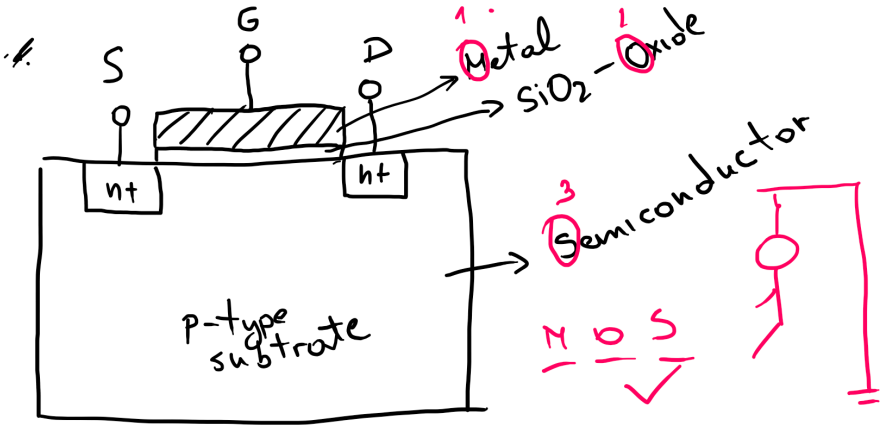


Figure 1.4: NMOS cross-section: a metal gate over an oxide insulator, sitting above a p-type substrate with two n+ regions for source and drain

We illustrate an NMOS. Its gate is a metal layer separated from the p-type substrate beneath it by a thin oxide layer, which insulates the gate and prevents any DC current from flowing into it. Source and drain are both n+ regions embedded directly in that p-type substrate. The names describe their role in conduction: the source supplies the charge carriers, electrons for an n-channel device and holes for a p-channel one, that flow through the channel, and the drain is where those carriers leave it.

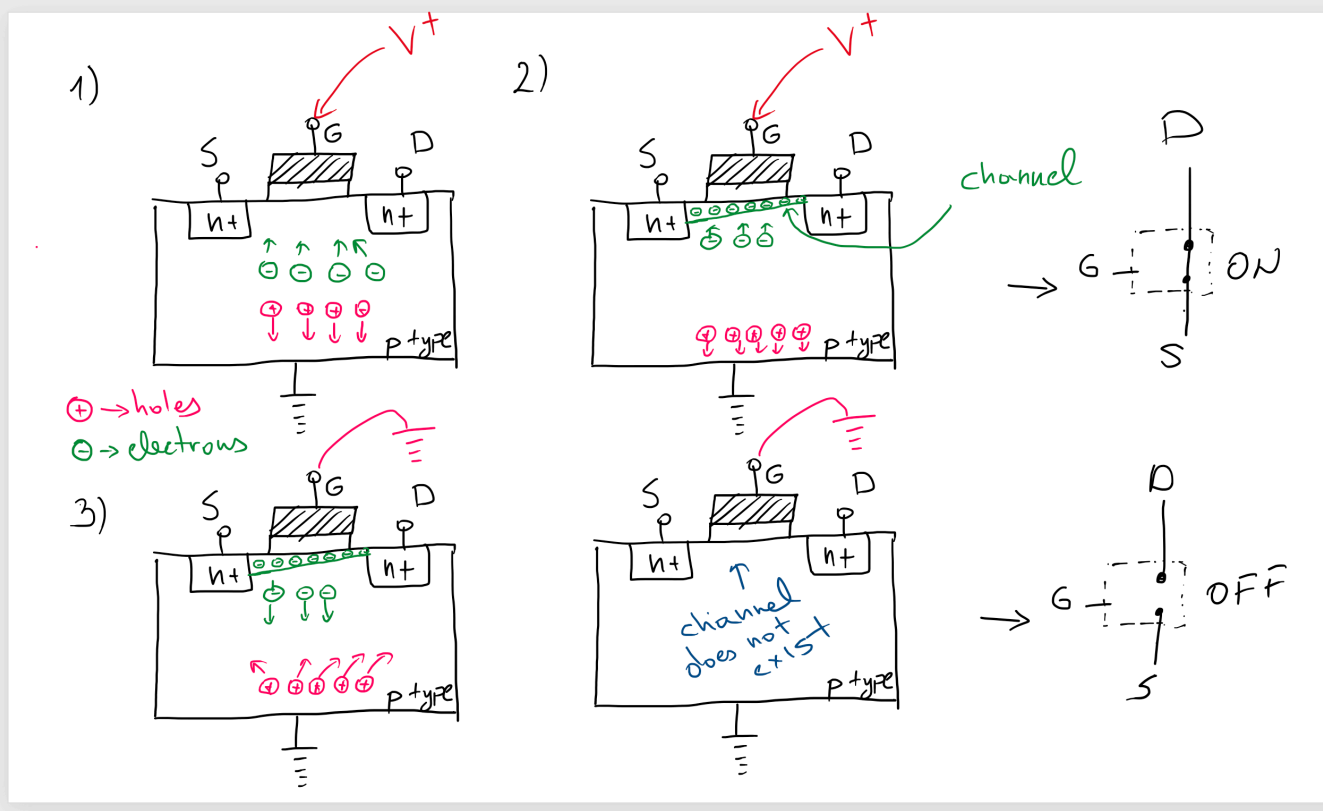


Figure 1.5: Turning an NMOS on: a positive gate voltage pulls electrons to the surface and forms a conducting channel between source and drain, which collapses again once the gate voltage is removed

To turn an NMOS on, we apply a positive voltage to the gate. That voltage attracts electrons from the p-type substrate toward the surface and pushes holes downward and away, purely through the electric field it creates, which is exactly what makes this a field-effect transistor. Once enough electrons have gathered at the surface, they form a bridge, the channel, connecting source and drain, and current can flow between them. How long that channel is, geometrically, is a property of the fabrication technology used.

In effect, the MOS transistor behaves as a controlled switch. Take the voltage off the gate, and the electrons no longer have anywhere to stay. The channel dissolves, and the connection between drain and source opens back up.

1.3.2 Complementary MOS

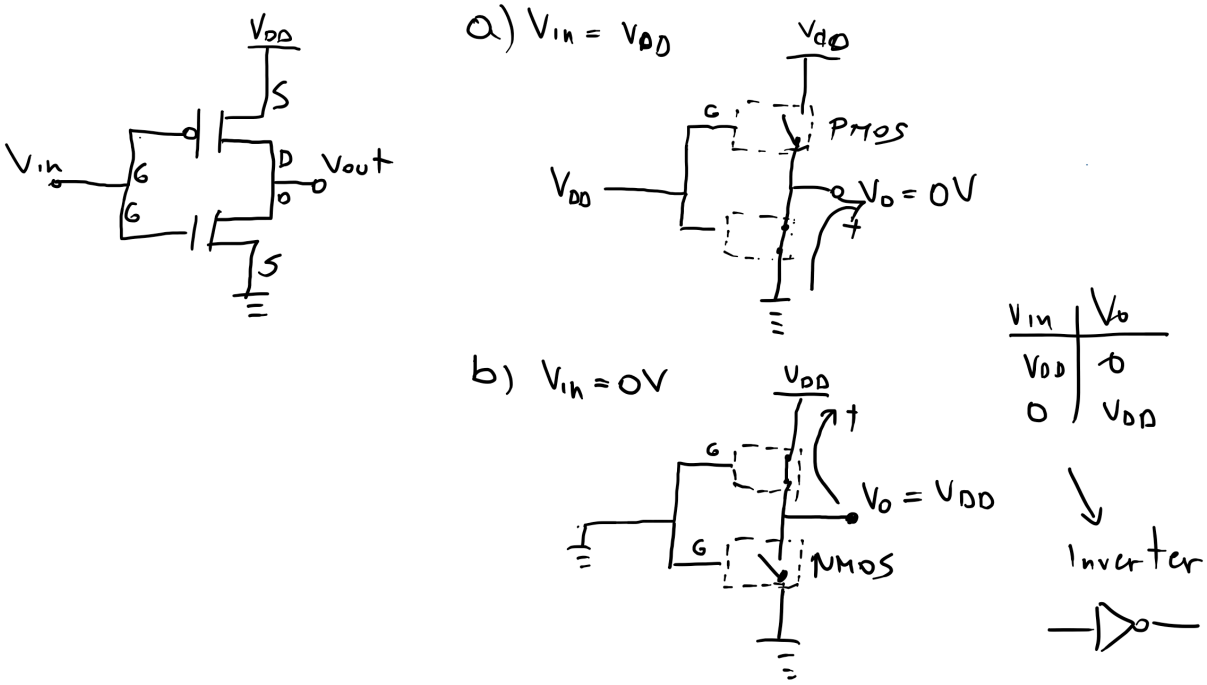


Figure 1.6: CMOS inverter: tying NMOS and PMOS gates together means exactly one of them conducts for any input, pulling the output to either ground or VDD

A CMOS gate pairs an NMOS and a PMOS transistor that share a common gate input. We already know how to turn an NMOS on, and a PMOS is its mirror image: it turns on when the gate is low rather than high. Tie a positive voltage to the shared gate, and the NMOS turns on while the PMOS turns off, pulling the output down to ground. Tie the gate to ground instead, and the PMOS turns on while the NMOS turns off, pulling the output up to VDD. In both cases, exactly one of the two transistors conducts, never both, and the output ends up at the opposite logic level from the input. That is an inverter, and it is the simplest possible CMOS gate.

1.4 CMOS characteristics

1.4.1 Transfer function

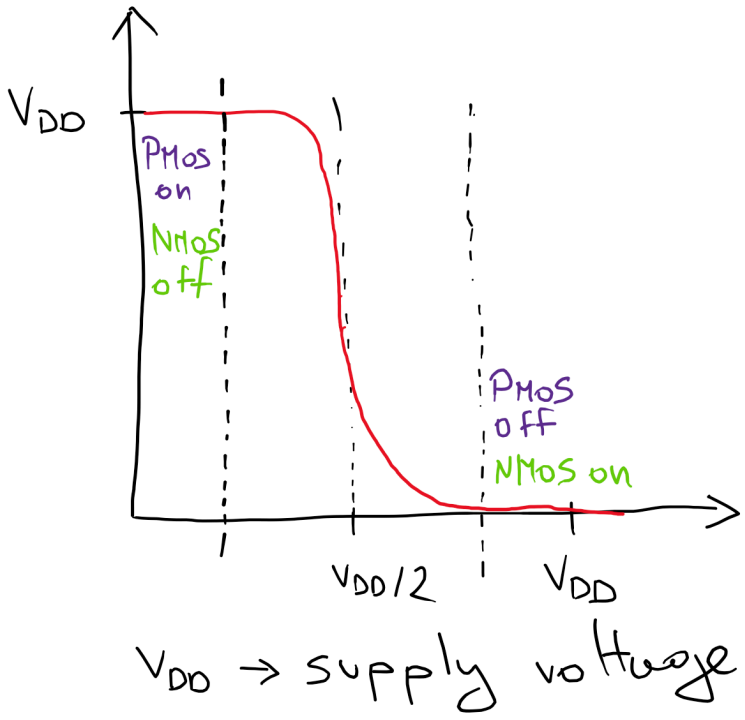


Figure 1.7: CMOS inverter transfer function: the output stays near VDD or near ground across most of the input range, and switches abruptly near VDD/2

The transfer function shows how the output voltage depends on the input voltage. For a CMOS inverter, it is almost a step function. For a wide range of low inputs, the PMOS is on and the NMOS is off, and the output sits close to VDD. For a wide range of high inputs, the roles reverse and the output sits close to ground. Only in a narrow band around VDD/2 do both transistors conduct at once, and the output swings sharply from one rail to the other. That sharp transition is exactly what we want from a digital gate. It is what keeps a “1” looking like a clean 1 and a “0” looking like a clean 0, right up until the input crosses the switching threshold.

1.4.2 Noise margins

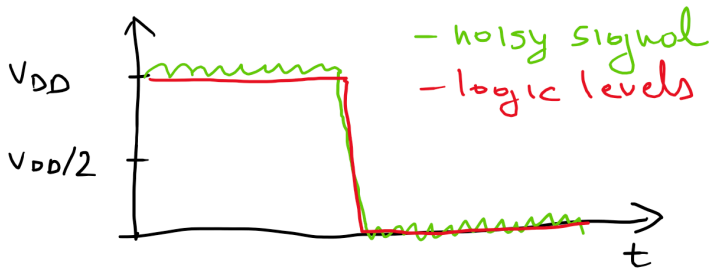


Figure 1.8: A noisy signal riding on a CMOS logic level. As long as the noise stays within the flat regions of the transfer function, it never gets interpreted as the wrong logic level

Because the transfer function stays flat near VDD and near ground across most of the input range, a CMOS gate tolerates a surprising amount of noise. A signal can pick up noise from neighboring wires or supply fluctuations and still be read correctly, as long as that noise does not push the voltage past the switching threshold in the middle of the transfer function. This margin between “close enough to a clean logic level” and “close enough to flip the gate’s decision” is the noise margin, and CMOS’s wide noise margins are one of the main reasons it dominates digital design.

1.4.3 Power

CMOS technology is known for its low power consumption, which is one of its biggest advantages over older logic families. We split that power consumption into two categories: static power and dynamic power.

Static power (also called leakage power) is what the circuit consumes while it is not switching at all. It comes mostly from small leakage currents through transistors that are nominally off, and it becomes a much bigger concern as transistor geometries shrink.

Dynamic power is what the circuit consumes while it is actively switching. Every time a gate’s output toggles, we have to charge or discharge its load capacitance, and that costs energy. We can express the dynamic power of a single node as

$$P_{dynamic} = \alpha C V^2 f$$

where α is the activity factor (the fraction of clock cycles in which the node actually switches), C is the load capacitance, V is the supply voltage, and f is the switching frequency.

1.4.4 Dynamic behavior

A CMOS gate does not switch instantaneously. To change its output, it first has to charge the load capacitance, when the output rises, or discharge it, when the output falls. That load capacitance is not one single thing. It is the sum of the input capacitance of every gate the output drives, the capacitance of the interconnect wiring between them, and the capacitance of the reverse-biased P-N junctions inside the transistors themselves. To make a gate switch faster, we have a few levers: raise the supply voltage, widen the transistor's channel width-to-length ratio, or simply drive fewer downstream gates from the same output.

1.5 Manufacturing CMOS integrated circuits

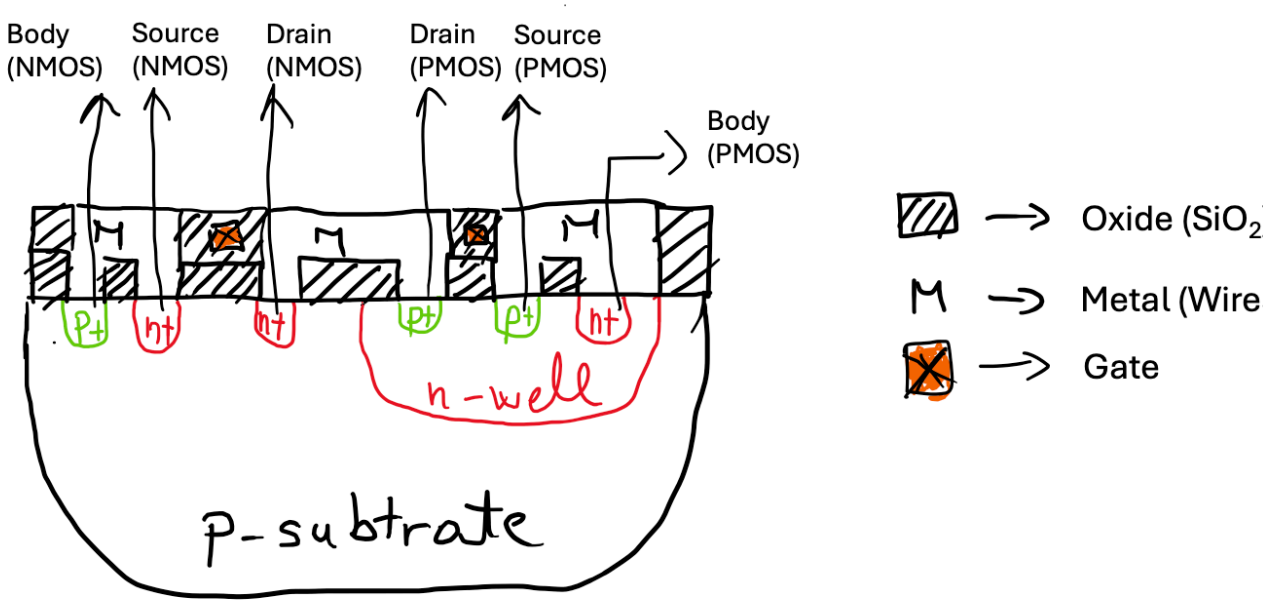


Figure 1.9: Cross-section of a CMOS chip: the PMOS sits inside an n-well so it can share the same p-type substrate as the NMOS

Building CMOS means putting NMOS and PMOS transistors on the very same piece of silicon, even though an NMOS wants a p-type substrate and a PMOS wants an n-type one. The usual solution is an n-well: a pocket of n-type silicon diffused into the p-type substrate, deep enough to host a complete PMOS transistor, source, drain, and body, while the NMOS transistors around it sit directly in the surrounding p-type substrate as before. Everything above that, the oxide layers and the metal wiring that connects the transistors into actual logic gates, is built in the same set of process steps for both transistor types.

1.5.1 Photolithography process

Turning a bare silicon wafer into a working chip is a repeated pattern of deposit, pattern, and etch, applied layer by layer. It starts with the wafer itself, onto which we grow a thin layer of silicon dioxide (SiO2) through oxidation. On top of that, we spin on a photoresist, a light-sensitive polymer. By default we use a negative resist, which becomes soluble wherever light hits it, though a positive resist, which behaves the opposite way, is also an option depending on the process.

Next comes the stepper exposure. A glass mask carrying the pattern for this layer is brought close to the wafer. Its opaque regions block UV light, and its transparent regions let it through, exposing the resist beneath in exactly the shape we want to process. After exposure, we develop and bake the wafer: an acid or base solution washes away the soluble part of the resist, and baking hardens what is left into a solid protective mask.

With that mask in place, acid etching removes material, typically the oxide layer, wherever the resist is not protecting it. A spin, rinse, and dry step cleans the wafer before whatever comes next, whether that is another masked layer or a different process step entirely. Once a layer is finished, we strip away the remaining photoresist, and the wafer is ready for the next pass through this same cycle, one layer at a time, until the full set of transistors and interconnect is complete.

2 Basics of Digital Integrated Circuits

Basics of Digital Integrated Circuits

2.1 Goal for this week

- Classify integrated circuits by fabrication technology (bipolar, MOS/CMOS, BiCMOS) and by scale of integration, and see why Moore's law is the reason those scales keep climbing
- Separate the IC business models: standard parts built for nobody in particular, full-custom ASICs built for one customer, and the semi-custom middle ground
- See how standard cells, macrocells, and gate arrays let an ASIC designer reuse pre-built blocks instead of hand-laying every transistor
- Meet the predecessors to the FPGA: SPLDs (PAL, PLA, PLE) and CPLDs, and the handful of underlying technologies a chip can use to store its configuration

2.2 Digital integrated circuits

A digital integrated circuit (IC) is, in simple terms, a miniature, low-cost circuit made of active and passive elements, all fabricated together on a single silicon die. We can classify ICs by the transistor technology they are built from: bipolar junction transistors (BJT), unipolar MOS or CMOS transistors like the ones we met in the previous chapter, or a mix of both, called BiCMOS.

Bipolar junction transistors have real advantages over CMOS. They switch at higher frequencies, offer better gain, and can drive higher output currents. CMOS wins anyway for almost all digital design, because a BJT's static power consumption is never quite zero, and BJTs are larger and slower to fabricate at a given density. That is exactly the low-static-power, high-density tradeoff that made CMOS the default technology in the previous chapter.

2.2.1 Scale of integration

We also classify ICs by their scale of integration, a rough measure of how many basic elements, transistors or gates, fit on one chip. Small scale integration (SSI) covers up to about 100 elements, medium scale (MSI) 100 to 1,000, large scale (LSI) 1,000 to 10,000, very

large scale (VLSI) 10,000 to 100,000, and ultra large scale (ULSI) 100,000 to 1,000,000. Beyond that, wafer-scale integration treats an entire silicon wafer as a single circuit rather than cutting it into individual dies.

This progression is not just a naming exercise. Gordon Moore observed in 1965 that the number of transistors we can fit on a chip roughly doubles every two years, a trend we now call Moore's law. It is the reason the scale of integration climbed from a handful of gates per chip in the 1960s to transistor counts in the billions today, and it is the main force behind nearly every capability we take for granted in modern digital design.

2.3 Design methodologies

We can also classify digital ICs by who they are designed for, which we call their design methodology: standard ICs (SIC), application-specific ICs (ASIC), and standard ASICs (SASIC).

2.3.1 Standard integrated circuits

A standard IC is designed for an unknown customer. Whoever eventually buys the chip has no influence on how it was designed, and that is exactly what lets it be mass-produced and sold cheaply.

The earliest standard ICs were simple, fixed-function parts, and building any larger system out of them meant wiring together many of them on a board to form whatever circuit was needed. Software-programmable standard parts changed that, above all the CPU: the same piece of hardware adapts to a new task just by changing the program it runs, instead of changing its wiring. That flexibility is not free. A general-purpose CPU is rarely the most efficient way to run any one specific task, a tradeoff we look at more closely when we get to FPGAs later in this book.

2.3.2 Application-specific integrated circuits

An application-specific integrated circuit (ASIC) is designed for one specific application or field, rather than for an unknown general-purpose market. We build ASICs in two ways: full-custom and semi-custom.

2.3.2.1 Full-custom design

A full-custom design is built specifically for one customer, with every mask layer needed to fabricate the chip designed to order for that customer's exact need, whether that is a systolic array, genome analysis hardware, or a dedicated accelerator. The design itself is typically strongly confidential. Since the design cost is comparable to a standard IC's but spread over a far smaller production run, full-custom chips carry a high price per unit.

2.3.2.2 Semi-custom design

Semi-custom design starts from a predesigned or preprocessed chip. Only certain mask layers get adapted to the customer's needs, while the rest stay identical across every customer who buys that base design. The manufacturer keeps a library of pre-tested modules on hand and connects them to fit each customer's requirements, rather than designing from scratch every time. Because the same base design is reused across many customers, semi-custom chips are much cheaper per unit than full-custom ones.

2.3.3 Building blocks for ASIC design

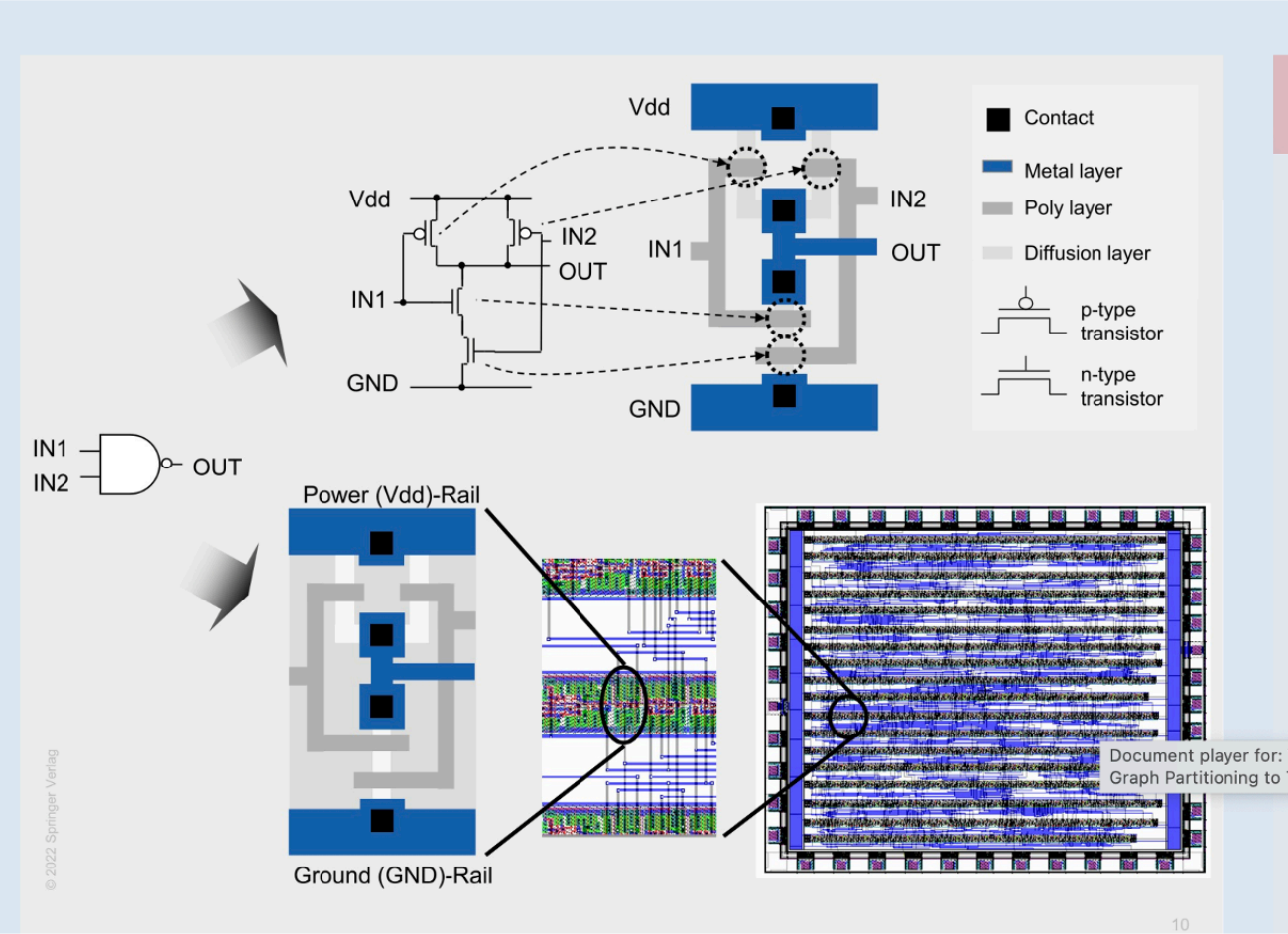


Figure 2.1: Standard-cell design flow: a NAND2 gate as a transistor schematic, its layout in metal, poly, and diffusion layers, and the resulting row-based placement on a die (source: Kahng et al., *VLSI Physical Design: From Graph Partitioning to Timing Closure*, © 2022 Springer Verlag)

Semi-custom and standard-cell ASIC design rests on a few kinds of reusable building block.

Standard cells are small, universal blocks, logic gates, flip-flops, multiplexers, and the like, each with a predefined layout and electrical behavior. The figure above shows a two-input NAND gate as a standard cell: its transistor schematic, its layout in metal, polysilicon, and

diffusion layers, and the fixed-height block that results, ready to sit in a row next to other cells. Every standard cell in a library shares that same row height, which is what lets thousands of them be tiled into rows and wired together with routing that runs above the cells, so-called over-the-cell routing. A chip's entire library of standard cells, together with the data needed to use them, is called a process design kit (PDK).

Macrocells and megacells are a step up from standard cells: larger, pre-laid-out blocks such as adders, multipliers, registers, or even whole microprocessors. The difference between the two is who they were designed for. A megacell is designed without knowledge of the specific application that will use it, which makes it broadly reusable, while a macrocell is designed to order for one customer's needs.

Gate arrays take a different approach. Their basic elements, individual gates or groups of unconnected transistors, are pre-fabricated across the whole die before any customer design is known. Implementing a specific design on a gate array is then purely a metal-wiring problem: connect the gates that are already there in whatever pattern the design needs. Early gate arrays separated these blocks with dedicated routing channels. Later "sea of gates" designs packed transistors edge to edge instead and used higher-level metal layers for routing, improving density, with connections made by shorting or bridging transistors that would otherwise stay unconnected.

2.4 Standard ASICs (SASIC)

A standard ASIC (SASIC), also called a configurable ASIC, is manufactured generically, the same way a standard IC is, and only afterward configured to behave like a custom one. What changes from one application to the next is not the silicon itself. It is a set of configuration bits, held in one of a handful of underlying technologies.

2.4.1 Configuration technologies

Static memory is the technology we will see the most of later in this book. Configuration bits live in SRAM cells and drive electronic switches (transmission gates, tri-state buffers, pass transistors) on or off. It is fast and fully reconfigurable, at the cost of being volatile. This is the same idea behind the configuration SRAM inside every FPGA we meet later in this book.

UV-erasable memory stores its configuration differently. Its key element is a MOSFET with a second gate, electrically isolated and sandwiched between the control gate and the substrate, called a floating gate. A strong lateral field between drain and source injects electrons onto that floating gate, and because it has no conductive path to leak through,

those electrons stay trapped once the field is removed, even with power off. Erasing the device means shining UV light on it, energetic enough to let the trapped electrons escape.

Electrically erasable memory keeps the same floating-gate mechanism, but the device is built so the trapped charge can be removed electrically instead, without needing a UV lamp.

Fuse and antifuse technologies work differently again, and only in one direction. A fuse is a narrow bridge of conducting material that blows open when a programming current is forced through it. An antifuse is the opposite: a thin dielectric separating two conductive layers, which ruptures under a programming voltage and creates a permanent short between them. Either way, programming a fuse-based device is a one-time, permanent decision, unlike the other three technologies, which can all be reconfigured.

2.4.2 Simple programmable logic devices

X	Y	Z	W
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

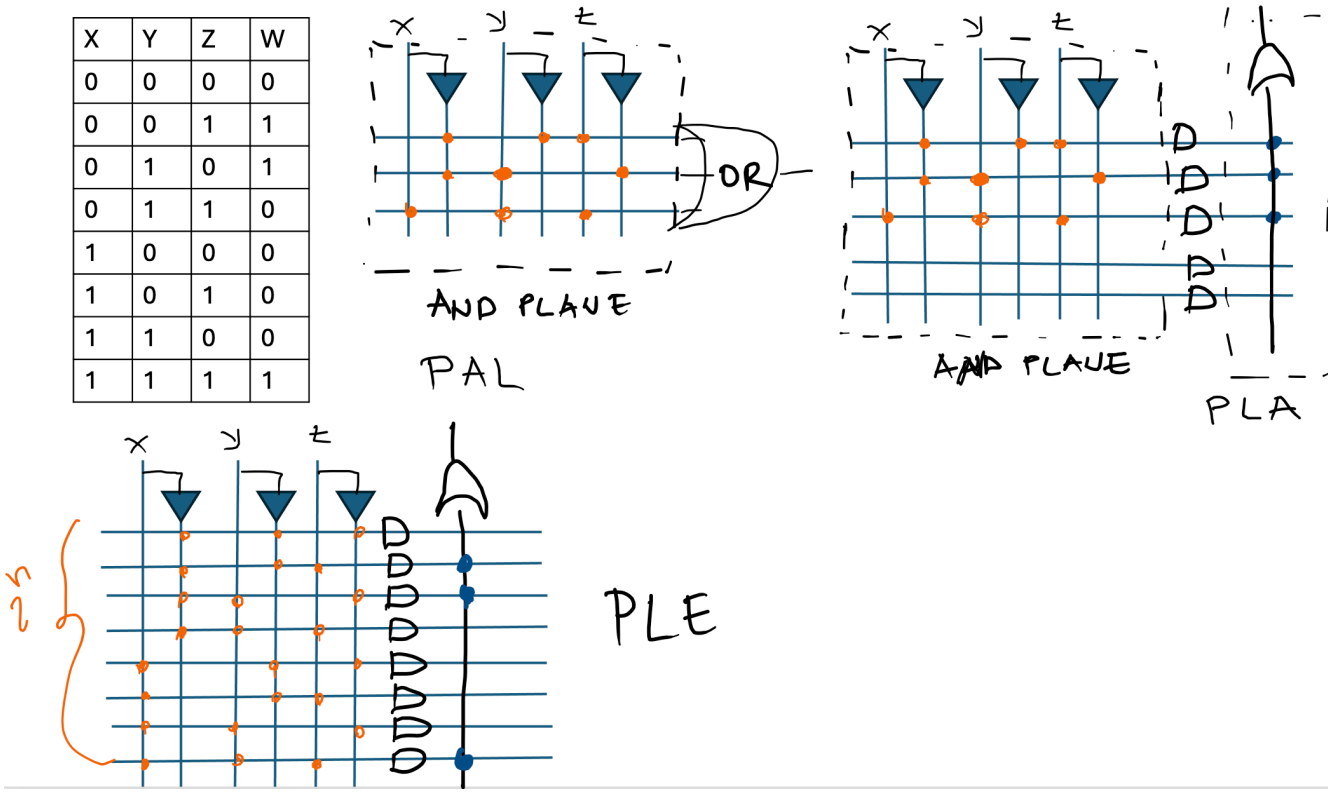


Figure 2.2: SPLD architectures: PAL (configurable AND plane, fixed OR plane), PLA (both planes configurable), and PLE (fixed AND plane covering every product term, configurable OR plane)

Simple programmable logic devices (SPLDs) are the direct ancestors of the FPGA, and the simplest configurable logic devices we will meet. Internally, an SPLD is built from one or two programmable logic planes, an AND plane and an OR plane, that implement a digital function exactly the way we would write it out by hand: as a sum of products. Every input, and its inverted value, runs as a vertical line through the AND plane, and every horizontal line represents one product term. Which vertical lines feed which horizontal line, and which product terms feed which output, is exactly what gets configured.

Which of the two planes is configurable, and which is fixed, gives us three SPLD variants. In a PAL (Programmable Array Logic) device, the AND plane is configurable but the OR plane is fixed. In a PLA (Programmable Logic Array), both planes are configurable, at the cost of more configuration bits and a larger device. In a PLE (Programmable Logic Element), the AND plane is fixed and already generates every possible product term, all 2^n of them for n inputs, and only the OR plane, choosing which of those terms feed each output, is configurable.

2.4.3 Complex programmable logic devices

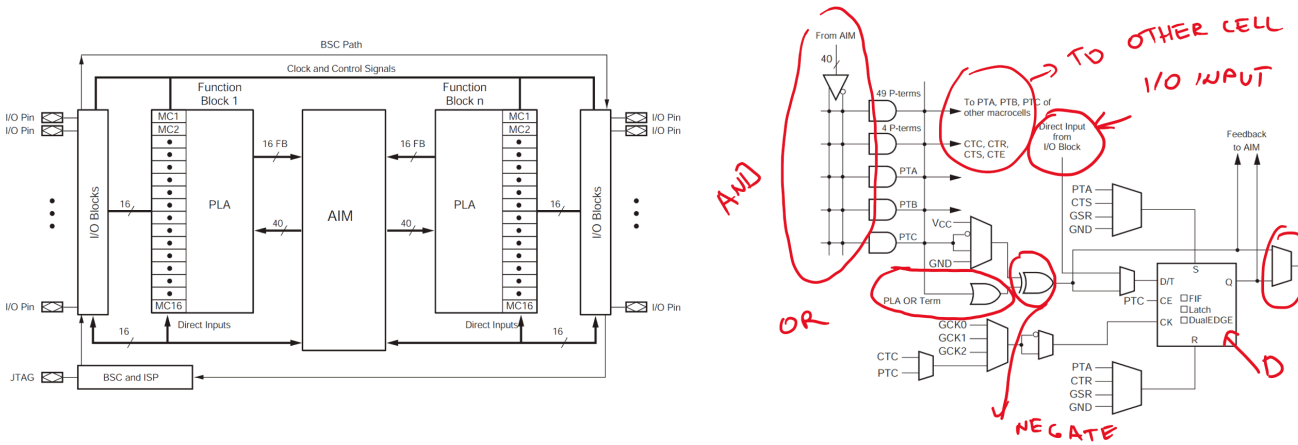


Figure 2.3: CoolRunner-II CPLD architecture: I/O blocks and function blocks tied together by the Advanced Interconnect Matrix (AIM), with one function block's macrocell detail (AND/OR terms, an XOR stage, and a configurable flip-flop) on the right (source: adapted from Xilinx CoolRunner-II documentation, DS090)

A complex programmable logic device (CPLD) is, at its core, many SPLDs packaged on one chip and tied together with a programmable interconnect, trading a single SPLD's limited

capacity for a much larger, still fairly predictable, device.

Take the Xilinx CoolRunner-II as a concrete example. Its I/O blocks connect the chip to the outside world, the same role they play in the FPGAs we meet later in this book. Thirty-two identical function blocks hold the actual logic, and an Advanced Interconnect Matrix (AIM) wires the function blocks to each other and to the I/O blocks, playing the same role the FPGA's routing plane plays later.

Each function block looks like a small PAL: 56 AND product terms feeding 16 OR sum outputs. Beyond that core, each output can pass through an XOR stage, useful for controlled inversion or parity-style logic, and into a flip-flop resource configurable as a D flip-flop, a level-sensitive latch, or a toggle (T) flip-flop. An output mux then picks whether the macrocell's output is the raw combinational result or the registered one, the same combinational-versus-registered choice the FPGA's CLB output mux makes.

Tiling many PAL-like blocks together through the AIM lets a CPLD implement far larger combinational and sequential designs than a single SPLD could, with timing that stays more predictable, and I/O that responds faster, than the larger and more routing-heavy FPGAs we look at later in this book.

3 FPGA Fundamentals

FPGA Fundamentals

3.1 Goal for this week

- Place the FPGA on the spectrum between a general-purpose CPU and a fixed ASIC, and see why that position makes it the right prototyping target for this course
- Break the FPGA fabric down into its three building blocks: CLB, routing plane, I/O blocks
- Open up a CLB far enough to see the LUT + flip-flop pair that will later become “the thing our synthesized SystemVerilog turns into”

3.2 Why reconfigurable hardware?

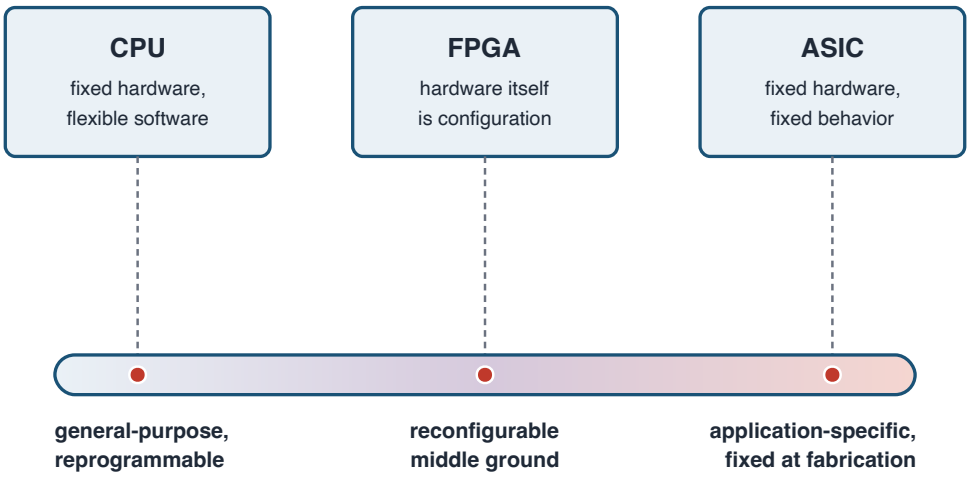


Figure 3.1: The CPU-FPGA-ASIC spectrum: flexibility and reprogrammability trade off against efficiency and fixed-function performance

As we saw in previous chapters, a CPU is a general-purpose integrated circuit: its function is defined not by its hardware but by the software program a developer writes for it. Development cost and time are low, since a single instruction set can be programmed in general-purpose or domain-specific languages to meet almost any demand — from database search to signal processing and beyond. But this same generality is also the CPU's biggest disadvantage. A CPU performs well on workloads like databases, yet it can be slow on compute-intensive problems such as neural network inference.

ASIC designs overcome this general-purpose bottleneck by mapping the application directly into hardware rather than software. A well-known example is Google's Tensor Processing Unit (TPU), built to accelerate the compute-intensive workloads of neural network inference. Its designers tailored the architecture to that one problem, wiring together a matrix of arithmetic logic units (ALUs) instead of following the standard CPU approach of a five-stage pipeline. Comparing a CPU and a TPU on neural network inference, the TPU wins decisively — but hand it a database search instead, and it is useless. This is the core limitation of ASIC accelerators: each one targets a single application, and once fabricated, it cannot be repurposed for anything else.

Comparing the CPU and the TPU, what we would really like is both properties at once: the flexibility to repurpose the device, and hardware tailored to the application at hand to squeeze out its full potential. Reconfigurable hardware, the field-programmable gate array (FPGA), gives us both, and its name spells out how. "Gate array" comes from the internal structure: logic blocks arranged on a matrix-like fabric, echoing the gate-array architecture used in some ASICs. "Programmable" does not mean the FPGA executes a software program in the traditional sense, though it can in a limited way. Instead, it refers to the reconfigurable property itself, achieved by loading a bitstream, a special binary configuration file, onto the device. "Field" refers to the field of application: an FPGA can adapt its internal structure to whatever application is at hand, out in the field, long after manufacturing.

This is exactly the trade-off an FPGA offers. It sits between the CPU and the ASIC by turning the hardware itself into a form of software: the logical function of every building block, and the wiring between them, is set by the bitstream loaded after manufacturing. We get most of an ASIC's advantage of "hardware built for the job" without paying its fixed non-recurring engineering (NRE) cost or its all-or-nothing commitment. That is exactly why this course targets FPGAs. A mistake in a lab assignment costs a re-synthesis and a few minutes, not a re-spun die. FPGAs are also the standard vehicle for rapid prototyping and hardware acceleration in industry for the same reason. Reconfigurability turns hardware design into something you can iterate on.

3.3 The three building blocks

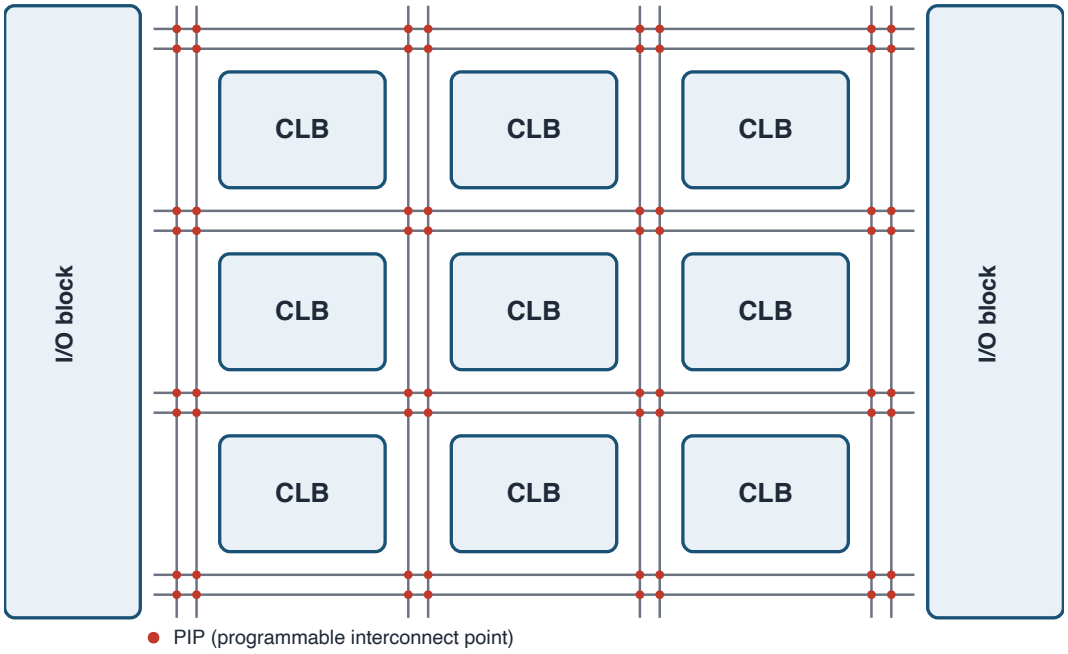


Figure 3.2: FPGA fabric: CLBs embedded in a routing plane, flanked by I/O blocks

Every FPGA, regardless of vendor or generation, is built from the same three kinds of block. CLBs (also called logic elements) sit in a regular grid and implement the actual logic functions and storage of the design. Between and around them runs the routing plane: a fabric of wires and programmable switches whose only job is to connect a CLB's outputs to the inputs of other CLBs, in whatever pattern the design needs. At the edges of the die, I/O blocks connect the fabric to physical pins, translating between the FPGA's internal signal levels and whatever the outside world expects. None of these three blocks is useful without the other two: a CLB with no routing to reach it is dead logic, and routing with nothing to connect is just wire. Placing a design onto an FPGA is precisely the problem of deciding which CLB implements which piece of logic (placement) and which routing resources carry the signals between them (routing). We will meet both steps again when we look at the FPGA design flow.

3.3.1 Configurable Logic Block (CLB)

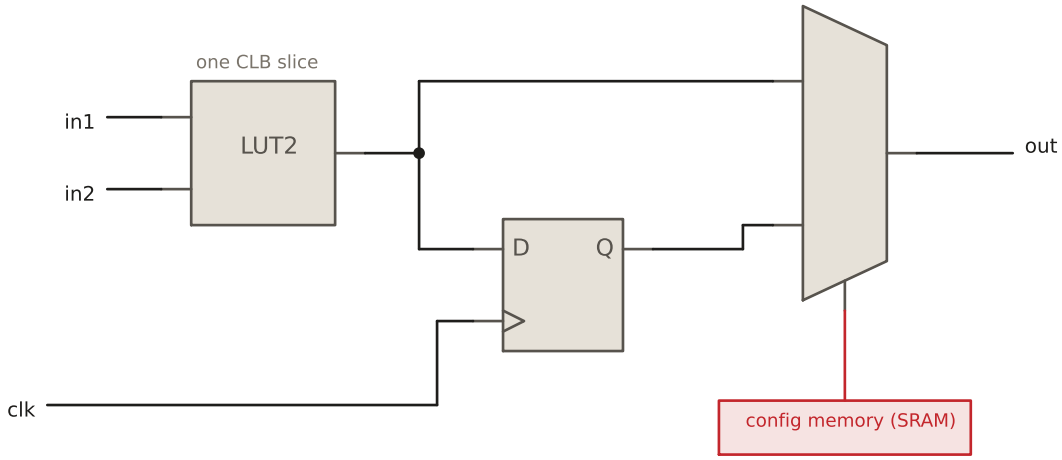


Figure 3.3: One CLB slice: a LUT2 and a flip-flop, with the output mux picking between the combinational and registered result

The CLB is the main carrier of reconfigurability in an FPGA and its elementary building block. You can think of the CLB as the FPGA equivalent of an assembler instruction, the most basic element everything else is built from. The term itself differs across vendors. Xilinx calls it a logic slice, while Altera calls it a logic element. Either way, they refer to the same thing. As the primary building block for digital circuits, a CLB must support combinational logic, sequential logic, and combinations of the two. It consists of a look-up table (LUT), a flip-flop, and an output mux.

A LUT implements combinational logic through memory lookup rather than by wiring gates together. Structurally, it consists of SRAM (static random-access memory) cells paired with a multiplexer that selects which cell's content is forwarded to the output, based on the signal inputs. To see this concretely, consider the smallest useful case, a two-input LUT, or LUT2. With two inputs there are four possible input combinations, so a LUT2 is built from four memory cells and a 4-to-1 multiplexer. The two inputs act as the multiplexer's select lines, in other words as an *address*, and the output is simply whatever bit is stored in the addressed cell.

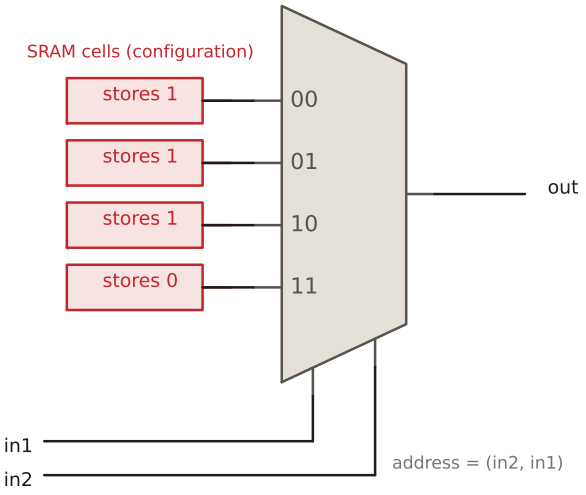


Figure 3.4: LUT2 internals: the two inputs address one of four SRAM cells, and the addressed cell's stored bit is forwarded to the output

To implement logical functions with two input variables, you need to map the truth table into SRAM cells. For example to implement a two-input NAND on a LUT2, we do not build a NAND gate at all: we store the NAND truth table's output column (1, 1, 1, 0) into the four memory cells, in the order addressed by the two inputs, and the LUT now behaves as a NAND for as long as that configuration is loaded. The reconfigurability of LUTs is what makes FPGAs so flexible: the same LUT can just as easily hold an AND, an XOR, or any other two-input function. Reconfiguring the FPGA never touches a wire between gates. It only rewrites these memory cells. Because those cells are ordinary SRAM, they are volatile: power off the board and the "circuit" is gone, which is why every FPGA design needs a configuration step after every power-up.

We do not build larger LUTs as one giant memory array wired to a huge multiplexer. Instead, real FPGA architectures build them recursively from smaller LUTs, adding one extra input and one extra multiplexer stage at a time. A LUT3 is nothing more than two LUT2s, both fed by the same two inputs, with their outputs chosen between by a 2:1 mux controlled by the third input. Extending the same idea, a LUT4 is built from two LUT3s, each in turn built from two LUT2s, combined by one more 2:1 mux controlled by the fourth input. Each additional input doubles the number of underlying LUT2 building blocks and adds one more layer of multiplexing on top. This recursive structure is exactly why vendors describe their larger LUTs as fracturable: the same physical hardware that implements one LUT4 can, with a small configuration change, split back into two independent LUT3s, or four independent LUT2s, letting the synthesis tool pack more, smaller functions into fewer CLBs when a design does not need the full input width.

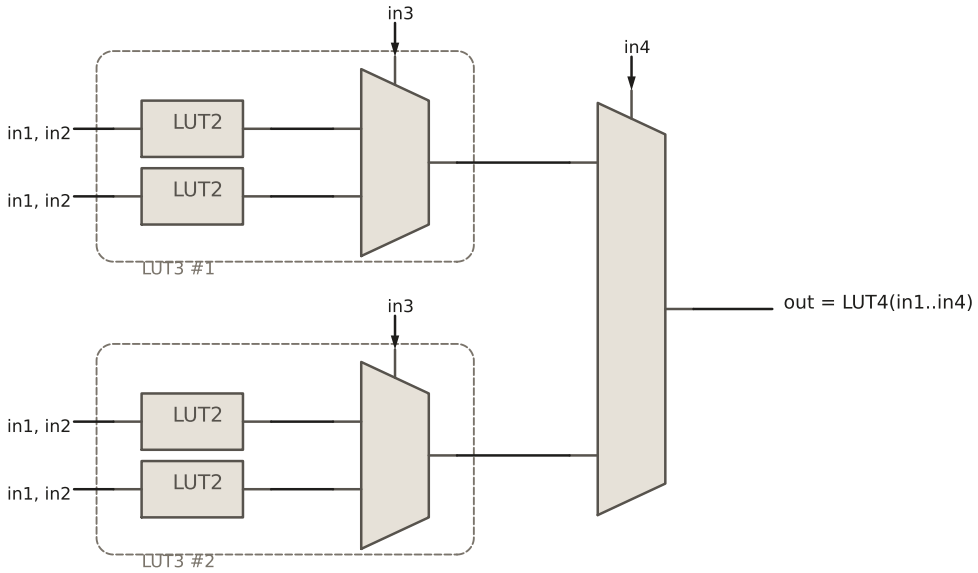


Figure 3.5: Building a LUT4 from four LUT2s: two levels of 2:1 muxing, selected by in3 and in4

Few real digital circuits are purely combinational. Almost every design also needs elements that are synchronous to a clock. To support that, every CLB slice also includes a D flip-flop (DFF) fed from the LUT's output. As we saw earlier, a DFF copies its input (D) to its output (Q) on the active edge of the clock signal, clk. Through configuration bits, the developer can adapt that DFF's behavior, choosing whether it triggers on the rising or falling edge, and setting its initial or reset value. This exposes yet another layer of reconfigurability inside the CLB.

The last element of the CLB is the output mux, which picks between the raw LUT output and the flip-flop's registered output. That mux's select line is, like the LUT's memory cells, just another bit in configuration SRAM. The same physical slice can act as combinational logic or as a single-bit register, purely depending on how it is configured. Real devices bundle several LUT + flip-flop slices, wider LUTs, and extra logic (carry chains, muxes for combining LUTs into wider functions) into one CLB. We look at a concrete example of that when we cover a real device architecture later.

3.3.2 Routing plane: programmable interconnect

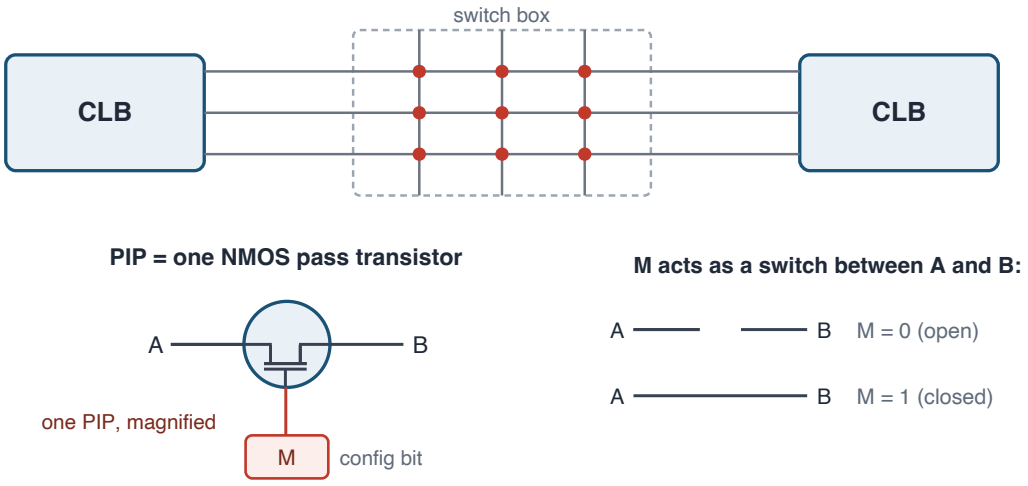


Figure 3.6: Two CLBs connected through a switch box. A PIP is one config-bit-controlled NMOS pass transistor

Routing plays two closely related roles. First, it lets many individual CLBs act as one large digital design: a real circuit needs far more logic and storage than a single CLB slice can hold, so the design is spread across hundreds or thousands of CLBs, and routing is what stitches their individual outputs together into one coherent datapath. Second, routing connects that internal logic to the outside world, joining CLBs to the I/O blocks that carry signals in and out of the fabric. Without it, CLBs and I/O blocks would be correct but disconnected islands of logic.

Between the rows and columns of CLBs run wiring tracks, and at every point where a horizontal and a vertical track cross, a switch box can optionally join them. The atomic component of a switch box is the PIP: a single NMOS transistor used purely as a switch, with its gate driven by one configuration SRAM bit. When that bit is 0, the transistor is off and the two tracks it sits between are electrically separate. When it is 1, the transistor conducts and the tracks become one net. A design's entire wiring, meaning every connection from one CLB's output to another CLB's input, reduces to which of these thousands of PIPs are turned on. This is also why the router is typically the slowest step in the FPGA implementation tools: placement decides which CLB does what, but routing has to find, among a huge and only partially connected mesh of tracks and switch boxes, one specific set of PIPs that realizes every net in the design without any two nets shorting together.

3.3.3 I/O blocks

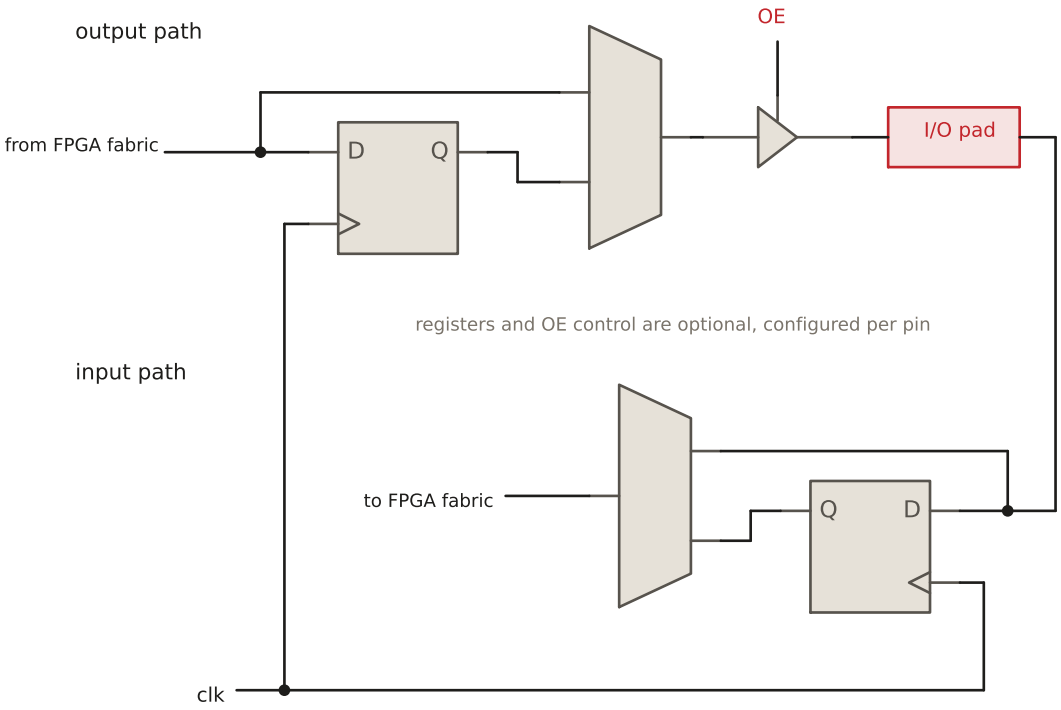


Figure 3.7: I/O block output and input paths: an optional register and a tri-state buffer on output, an optional register on input

An I/O block is the boundary between the reconfigurable fabric and a fixed physical pin, and it has to support more than "just wire a signal through." On the output side, a signal from the fabric can optionally be registered right at the pin before it reaches a tri-state buffer. That buffer's output-enable (OE) input decides whether the pad is actively driven at all, which is what makes a genuinely bidirectional pin possible. The same pad can be released to high-impedance and read as an input whenever the design is not driving it. The input side mirrors this with its own optional register. Registering directly in the I/O block, rather than in the nearest CLB, shortens the path the signal has to travel before it is captured, which matters because I/O timing margins are usually tighter than internal fabric timing. As with everything else on the chip, whether each of these registers is used, and whether a pin behaves as input, output, or bidirectional, is a configuration decision, not a fact fixed at fabrication.

3.3.4 Reconfigurability, level by level

We have now seen five separate places where a single configuration SRAM bit changes what the FPGA does. Inside the CLB, one set of bits picks the function stored in each LUT, another set picks whether the D flip-flop triggers on the rising or falling edge and what value it resets to, and one more bit decides whether the slice's output is the LUT's raw combinational result or the flip-flop's registered one. Out in the routing plane, a bit per PIP decides which pairs of tracks are joined into the same net. At the I/O block, bits decide whether a register is used on the input or output path, and whether a pin drives, receives, or does both. None of these is a special case. Every one of them is the same primitive: an SRAM cell whose value the configuration step writes before the device does anything useful. What looks like five different kinds of reconfigurability is really one mechanism, repeated at every point in the fabric where a design decision has to be made. This is also, in one sentence, what a bitstream is: the full list of values for every one of these bits, for every CLB, PIP, and I/O block on the device.

3.4 Case study: Xilinx Series-7

We have described the CLB, the routing plane, and the I/O block as three abstract building blocks, present in every FPGA regardless of vendor or generation. To see how a real device turns those abstractions into a concrete architecture, we now look at a specific family: Xilinx Series-7. Series-7 comprises four sub-families, Spartan-7, Artix-7, Kintex-7, and Virtex-7, all built from the same underlying fabric while trading off cost, logic capacity, and signal-processing performance. Spartan-7 targets the low end of that range: small, cost-sensitive, high-volume applications. Virtex-7 sits at the opposite end, aimed at the highest connectivity bandwidth, logic capacity, and DSP throughput. Artix-7 and Kintex-7 fill the middle ground. Every member of the family, from the smallest Spartan-7 to the largest Virtex-7, is assembled from the same handful of building blocks described below; only their quantity and mix change from device to device.

3.4.1 CLB: SLICEL and SLICEM

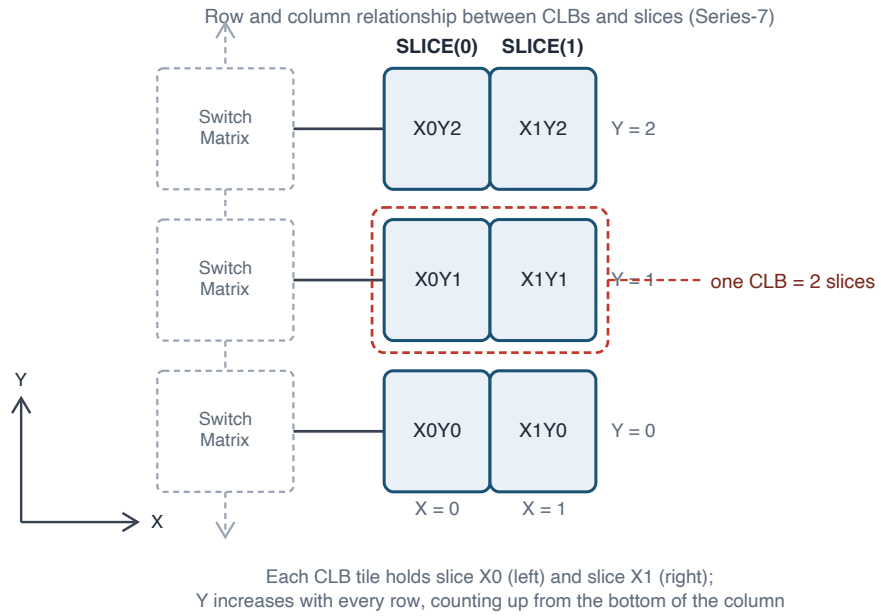


Figure 3.8: Row and column relationship between CLBs and slices: each CLB tile holds slice X0 and slice X1, wired to its switch matrix, with Y counting up from the bottom of the column (redrawn from AMD/Xilinx UG474, Fig. 1)

Where the generic CLB we described earlier held one LUT and one flip-flop, a Series-7 CLB is considerably richer, and Xilinx builds it from two smaller units called slices. Two slice types exist. Every CLB is either a CLB0, containing two SLICEL (“slice logical”) slices, or a CLB1, containing one SLICEM (“slice memory”) and one SLICEL slice. Each slice, of either type, contains four six-input LUTs (LUT6) rather than the single LUT2 we used to introduce the idea — the same recursive fracturable-LUT construction we saw earlier, just taken further, and each LUT6 can equally well operate as two independent LUT5s sharing the same five inputs. To build functions wider than six inputs, the slice adds dedicated combining multiplexers on top of the LUTs themselves: F7AMUX and F7BMUX each merge a pair of LUT6 outputs into one seven-input function, and a further F8MUX merges the two resulting LUT7 outputs into a single eight-input function. This is the same output-mux idea from the generic CLB, just repurposed to widen LUTs instead of choosing between a combinational and a registered output.

Because addition is one of the most common operations in digital design, each slice also includes a dedicated carry chain: fast, purpose-built hardware that propagates carry signals

between adjacent slices without routing them back through the general interconnect, considerably speeding up adders and comparators compared to building the same carry logic out of ordinary LUTs. On the sequential side, each slice packs eight flip-flops rather than the one flip-flop per LUT we introduced earlier. Four of these are configurable to behave as either an edge-triggered flip-flop or a level-sensitive latch, while the remaining four are flip-flop only; all eight share one clock and one clock-enable signal, though each still has its own configuration bits for init value and set/reset behavior, echoing the per-flip-flop reconfigurability we discussed for the generic CLB.

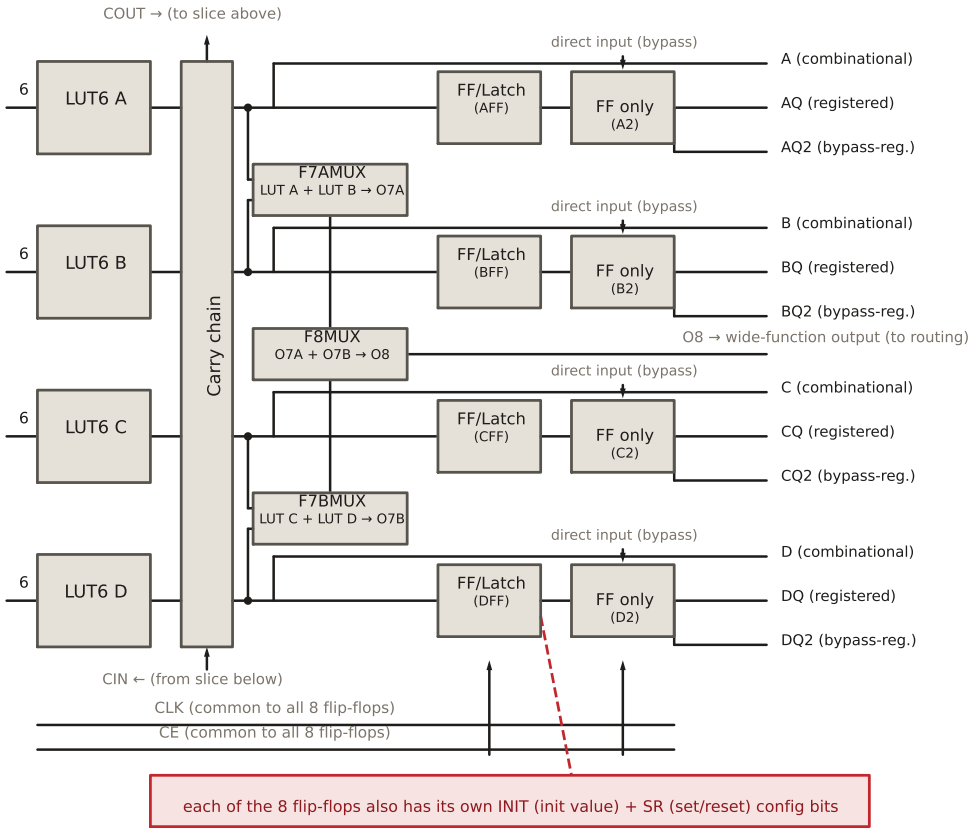


Figure 3.9: SLICEL, simplified block diagram: four LUT6s feed the F7A/F7B/F8 combining muxes and the carry chain, each lane ending in an FF/Latch and an FF-only register sharing one clock and clock-enable (redrawn, not to scale, after AMD/Xilinx UG474, Fig. 2)

SLICEM slices add one more capability that SLICEL slices lack. A LUT's whole purpose, as we saw earlier, is to hold a fixed truth table in SRAM cells that the design's inputs address; that content does not change while the design runs. SLICEM's LUTs can instead be

repurposed as small, fast, genuinely writable memory: either distributed RAM, where the LUT's SRAM cells are written and read directly by the design as if they were a tiny local memory, or a 32-bit shift register, where the same cells behave as a chain of delay elements. Both uses trade away the LUT's normal role as a combinational lookup table in exchange for a small amount of local storage exactly where it is needed, without consuming a dedicated block RAM.

3.4.2 Block RAM and DSP slices

SLICEM's distributed RAM is useful for small amounts of storage, but real designs routinely need much larger memories: register files, FIFOs, line buffers, caches. Building those out of LUTs alone would be both wasteful of logic resources and slow, so Series-7 devices additionally provide dedicated block RAM (BRAM): hard-wired memory macros scattered through the fabric, entirely separate from the CLBs. Each BRAM comes in one of two hard-wired sizes, 36 Kb or 18 Kb, and each size can be configured, at implementation time, into a range of width and depth combinations — a 36 Kb block, for instance, can be organized as 32K words of 1 bit, 16K words of 2 bits, 8K words of 4 bits, and so on, trading width for depth as the design requires. Configuration also selects the memory's port structure: single-port, true dual-port (two fully independent address, read, and write ports), or simple dual-port (one port dedicated to writing, the other to reading). A FIFO is not a separate hardware primitive at all; it is simply a block RAM combined with dedicated pointer and flag logic that turns its linear address space into a first-in-first-out queue.

Arithmetic is common enough, and multiplication expensive enough to build efficiently out of general LUT fabric, that Series-7 devices dedicate a third kind of hard-wired block to it: the DSP slice. A DSP slice implements a fixed pipeline of multiply, add, and accumulate hardware directly in silicon, entirely bypassing the LUT and carry-chain logic used for general-purpose addition. Just as with the CLB and BRAM, a DSP slice is configured rather than wired by hand — configuration bits select which of its internal operations are active and how they are chained together — but the arithmetic itself runs on dedicated hardware rather than being synthesized from LUTs, which is what makes DSP slices dramatically faster and more area-efficient than an equivalent LUT-based multiplier.

3.4.3 Clock management and interconnect

A clock signal is unlike any other net on the chip: every register that uses it must see the same edge at very nearly the same instant, or the design's timing margins erode. General-purpose routing, built from PIPs and switch boxes with no particular guarantee on delay matching, is not good enough for this job, so Series-7 devices route clocks through a dedicated network instead. Global clock buffers distribute a small number of clock signals across the device

with deliberately low skew, and the device's area is partitioned into clock regions, each spanning 50 CLB rows, with its own local clock distribution so that skew stays low even far from the clock's entry point. Before a clock reaches those buffers, it can optionally pass through a clock management tile (CMT), dedicated hardware that synthesizes new frequencies, shifts phase, or removes accumulated skew — capabilities well beyond anything the general fabric could provide.

Everything that is not a dedicated clock net still relies on the same general-purpose interconnect introduced earlier: horizontal and vertical routing tracks running between rows and columns of CLBs, joined at switch boxes wherever they cross. A switch box is simply a cluster of PIPs, and, exactly as in the generic architecture, each PIP reduces to one NMOS pass transistor gated by a single configuration SRAM bit. Series-7 does not introduce a new mechanism here at all — it is the same one-bit-per-connection primitive met earlier, just deployed at the scale a real device needs: thousands of switch boxes, each with many PIPs, giving the router enough freedom to connect any CLB, BRAM, or DSP slice's output to any other block's input.

3.5 FPGA design flow

Earlier, when introducing routing, we noted that placing a design onto an FPGA comes down to two questions: which CLB implements which piece of logic, and which routing resources carry the signals between them. The FPGA design flow is the sequence of tool steps that answers those questions automatically, turning an RTL description into the bitstream that configures every SRAM bit we have discussed in this chapter. That flow closely resembles the one used for ASICs, since both ultimately map a hardware description down to physical resources on silicon, but it differs in one important respect: the target primitive throughout is a LUT and its associated flip-flop, not a standard cell from an ASIC library, which shapes what several of the intermediate steps actually do.

The flow begins with design entry, typically RTL written in a hardware description language, and proceeds to synthesis. Synthesis interprets that RTL and produces a synthesized circuit netlist, generally stored in the standard EDIF format, along with preliminary performance estimates that give an early, optimistic sense of whether the design will meet its timing goals before the slower steps that follow. A translate step then merges this netlist with any others belonging to the same design (for instance, netlists from separately synthesized IP blocks) into one single netlist: a text description of every component in the design and how its terminals connect to every other component's terminals.

The next two steps are where the LUT-based nature of FPGAs really shows. Mapping takes the logical symbols in that merged netlist, ordinary gates such as ANDs, ORs, and multiplexers, and groups them into the physical components the target device actually provides: LUTs of a specific width, flip-flops, carry-chain segments, and so on. This is conceptually

similar to technology mapping in an ASIC flow, except the "cell library" being mapped to is not a fixed set of gates but the fracturable-LUT building blocks we described earlier in this chapter. Placing then takes every one of those mapped components and assigns it to one specific site on the chip, a particular slice within a particular CLB, deciding, in other words, which CLB implements which piece of logic. Routing follows, connecting every placed component to every other one it needs to reach by selecting a path through the switch boxes and PIPs of the routing plane, deciding which routing resources carry each signal.

Routing also produces the reports that tell us whether the design actually works at the clock frequency we want. A static timing analyzer performs a static analysis of the fully placed and routed circuit, accounting for every source of delay along every path, logic delay through LUTs, routing delay through PIPs, and everything in between, to identify the circuit's critical path: the longest path from the output of one register to the input of another. Because a synchronous design can run no faster than its slowest register-to-register path allows, the critical path directly determines the design's maximum clock frequency, which is exactly the number this whole flow exists to maximize.

Once placement and routing succeed and timing closes, the last step is bitstream generation: writing out the configuration file for the FPGA, the complete list of values for every LUT content bit, every flip-flop's configuration, every PIP, and every I/O block setting that we introduced across this chapter. That bit file can be downloaded directly onto the FPGA, configuring its (volatile) SRAM cells as we described earlier, or converted into a PROM file: a non-volatile store that reprograms the FPGA automatically, from power-up, every time the board is powered on.

4 SystemVerilog Basics

SystemVerilog basics

4.1 Goal for this week

- Explain what a hardware description language is, and why we describe digital systems in text instead of drawing every gate
- Place SystemVerilog in the Verilog family, and know which subset of it this course uses
- Read and write a module: ports, directions, internal signals
- Tell nets from variables, and know the four values a signal can hold
- Write numeric constants with an explicit size and base
- Describe combinational logic with `assign` and the SystemVerilog operators, and avoid the classic mistakes: `&` vs. `&&`, and two drivers on one net

4.2 Why describe hardware in text

The previous chapters described digital circuits bottom-up: transistors, gates, and the standard and programmable devices built from them, up to the FPGA. Nobody designs a processor by placing those pieces one at a time. A modern SoC has millions of gates, and a schematic of that size is neither writable nor reviewable. Instead we describe the system in text, in a **hardware description language** (HDL), and let tools do the placing.

The key idea is *abstraction*: an HDL lets us specify how a system operates without getting consumed prematurely by how it is implemented. The same description can be used in two very different ways. A **simulator** runs it as a model, so we can check behavior long before any chip exists. A **synthesizer** turns it into a netlist of real gates, or into the LUTs and flip-flops of the FPGA from the previous chapter. The rest of this course is about writing descriptions that are good for both.

4.3 What an HDL is used for

A single HDL covers a wide range of jobs. At the top, we can write a high-level behavioral model that says *what* a block computes, with no hardware structure at all. One level down

is **register-transfer level** (RTL): the design is a set of registers, and the combinational logic that moves data between them on each clock edge. This is the level at which this course, and almost all digital design, is done. Below it, the output of synthesis is a gate-level netlist, which can itself be written in the same language, and which can be annotated with timing information for timing simulation. Finally, the same language is used to write testbenches that verify the design.

Not everything an HDL can express can be turned into hardware. A simulator will happily run a testbench that waits 10 ns and prints a message, but a synthesizer has no gate for "wait 10 ns". Only a *subset* of the language is synthesizable, and writing in that subset is the first skill of this course.

4.4 From Verilog to SystemVerilog

Verilog was introduced in 1984 with a dual purpose: to model digital systems, and to describe the testbenches that check those models. As we saw, synthesis supports only a subset of what the language can express. Verilog was standardized several times, and the last Verilog standard is the one that SystemVerilog builds on:

Standard	Year	Nickname
IEEE 1364-1995	1995	Verilog-1995
IEEE 1364-2001	2001	Verilog-2001
IEEE 1364.1-2002	2002	Verilog RTL Synthesis (patch for the synthesizable subset)
IEEE 1364-2005	2005	Verilog-2005, the last Verilog standard
IEEE 1800-2005	2005	SystemVerilog-2005, extensions of Verilog-2005
IEEE 1800-2009, -2012, -2017, -2023		SystemVerilog, each extending the previous one

Too many new features had accumulated to keep calling the result Verilog, so a new language was defined. SystemVerilog extends Verilog in two directions: it improves hardware modeling (for example `logic`, `always_comb`, `always_ff`, enumerations and packed structures, all of which we use), and it adds verification features such as assertions, object-oriented programming, and constrained random stimulus.

Each SystemVerilog standard is an extension of the previous one, so backward compatibility is preserved. A synthesizer for SystemVerilog-2005 understands plain Verilog, but a tool

that only supports Verilog-2001 will report an error on the first SystemVerilog construct it meets.

We will cover only a small, synthesizable part of the language. There are two reasons. The language is far too large for one course. And once you can prototype with the basic constructs, new ones are easy to pick up when you need them.

4.5 Anatomy of a module

Every piece of hardware in SystemVerilog lives inside a **module**. The keywords `module` and `endmodule` mark the beginning and the end of the description, and a module can instantiate other modules inside it, which is how hierarchical designs are built.

```
module and_or (
    input  logic a, b, c,
    output logic y
);
  logic t;                 // internal signal
  assign t = a & b;
  assign y = t | c;
endmodule
```

The module name is followed by a **port list** in parentheses. Each port has a *direction*, a *type*, and a name. Several ports of the same direction and type can share one declaration, as `a, b, c` do above. After the semicolon comes the module body: declarations of internal signals such as `t`, and the statements that describe behavior, here two continuous assignments. Figure 4.1 shows what a synthesis tool builds from this module.

Direction	Meaning
`input`	Data flows into the module through this port
`output`	Data flows out of the module through this port
`inout`	Bidirectional: data can flow both ways (used for tri-state pins)

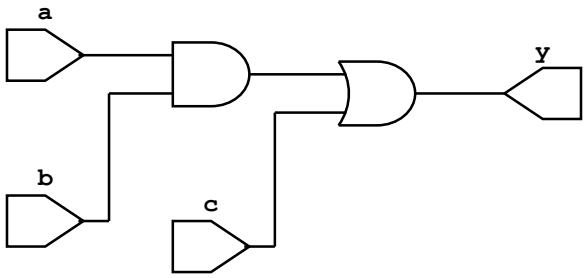


Figure 4.1: The `and_or` module, drawn by Yosys directly from the source above.

4.6 Values: 0, 1, X and Z

A wire in a real circuit is at a high or a low voltage, so you might expect two values, 0 and 1. SystemVerilog uses **four**, because the simulator also has to represent situations that a real circuit can get into:

Value	Meaning
`0`	logic low, false
`1`	logic high, true
`X`	unknown: an uninitialized register, or a net that two sources drive to different values
`Z`	high impedance: nothing is driving the signal (an unconnected input, or a tri-state buffer that is turned off)

You will meet `X` in simulation when something is wrong, and it is one of the most useful debugging signals you have: it spreads through logic that depends on it, so an `X` on an output points back to an uninitialized or double-driven source. `Z` appears with tri-state buffers, like the I/O block of the FPGA chapter. A synthesizer never creates an `X` or a `Z`; they exist only in the simulation model.

4.7 Data types: nets and variables

SystemVerilog has two kinds of signals, and the difference between them is what the whole language is built around.

A **net**, declared with `wire` or `logic`, models a physical connection between things. It stores nothing: its value at any moment is whatever its drivers are producing right now. If several things drive it, the net resolves their values, which is how a tri-state bus works. A **variable** is a signal that *remembers*. It holds the last value assigned to it, and keeps it until the next assignment.

	Net (`wire`,`logic`)	Variable (`logic`, `int`, …)
Models	a connection	a stored value
Holds a value	no, resolved from its drivers	yes, the last one assigned
Driven by	`assign`, module outputs	`assign` (once), or procedural blocks
Multiple drivers	allowed, resolved	an error

In plain Verilog you had to choose between `wire` and `reg` depending on *how you assigned* the signal, and `reg` was widely mistaken for "a register" when it is nothing of the kind. SystemVerilog fixes this with **`logic`**: a four-state data type that can be driven by a continuous assignment, by a procedural block, or by a module output. Its one rule is that it may have only one driver, and the tools check it. For a signal that legitimately has several drivers, such as a tri-state bus, use `wire`.

`bit` is the two-state counterpart of `logic`: it can only hold 0 and 1, so it can hide an unknown value in simulation by silently turning it into a 0. For that reason this course uses `logic` for signals.

4.8 Vectors and integer types

A signal declared with a range in square brackets is a **vector**: `logic [7:0] data` is an 8-bit signal whose most significant bit is `data[7]`. You can select a single bit, `data[3]`, or a contiguous slice, `data[7:4]`. Vectors are unsigned unless declared otherwise.

The types we use most are summarized below. The integer types come from C, and are mainly useful for loop counters and constants. Signals that become wires and registers in hardware should be `logic` vectors, which make the width explicit.

Type	Bits	Signed	States
`logic [n-1:0]`	n	no	4 (0, 1, X, Z)
`bit [n-1:0]`	n	no	2 (0, 1)
`byte`	8	yes	2
`shortint`	16	yes	2

Type	Bits	Signed	States
`int`	32	yes	2
`longint`	64	yes	2
`integer`	32	yes	4 (legacy from Verilog)
`real`	64, floating point	n/a	simulation only: not synthesizable

4.9 Writing constants

A SystemVerilog constant carries its own width, written as `<size>'<base><value>`, where `<size>` is the number of bits and `<base>` is `b`, `o`, `d` or `h` for binary, octal, decimal and hexadecimal. The four constants below all have the value ten, but different widths:

```
logic [3:0] binary_num  = 4'b1010;  // 4-bit binary 1010
logic [5:0] octal_num   = 6'o12;    // 6-bit octal 12
logic [7:0] decimal_num = 8'd10;    // 8-bit decimal 10
logic [7:0] hex_num     = 8'hA;     // 8-bit hexadecimal A
```

Underscores are ignored and are there for readability, so `8'b1010_1010` is easier to read than `8'b10101010`. The characters `x` and `z` are allowed in a constant: `4'b10xz`.

Always write the size. A bare number such as `10` is a **32-bit signed decimal**, and silently mixing it with narrower signals is a common source of width mismatches. If the value is bigger than the stated size, the extra upper bits are discarded: `4'd20` is `4'b0100`. To fill every bit of a signal of any width with the same value, use the unsized forms `'0` and `'1`.

4.10 Continuous assignment

The `assign` keyword connects a value to a signal **continuously**:

```
assign out = a & b;   // out always equals a AND b
```

Whenever `a` or `b` changes, `out` changes with it. A continuous assignment is not a step in a program. It is a gate that exists as long as the circuit is powered, and it models combinational logic.

4.11 Not like assignment in C

A programming language would read the next two lines top to bottom:

```
assign b = a;
assign c = b;
```

First copy `a` into `b`, then copy `b` into `c`. In hardware both lines exist at the same time as two connections, and `b` is just the wire between them. A synthesis tool removes it and infers `assign c = a`. For the same reason, swapping the two lines changes nothing.

4.12 Two drivers on one net

The second example is the one that breaks C intuition completely:

```
logic c;
assign c = a;
assign c = b;
```

In a programming language the second line would overwrite the first, and `c` would end up with the value of `b`. In hardware, both lines are always active, so the net `c` is **driven by two sources at once**. When `a` and `b` agree, nothing is wrong. When they differ, one source pulls the wire high while the other pulls it low, and the simulator reports an unknown value `X` (Figure 4.2). In a real circuit, this is a short circuit between two gate outputs.

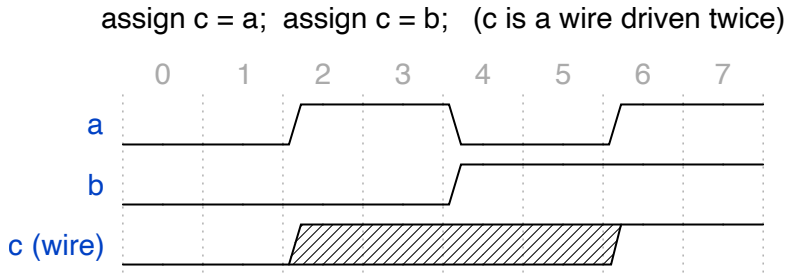


Figure 4.2: A net driven by `a` and by `b`: whenever they disagree, `c` is `X`.

If `c` were declared as `logic`, the tool would reject the second assignment at compile time, because a `logic` signal may have only one driver. That is the reason to prefer `logic`: it turns a silent `X` in simulation into an error you see immediately. This situation must be avoided in every design that is not a deliberate tri-state bus.

4.13 Operators at a glance

SystemVerilog has a rich set of operators, most of them familiar from C. The best way to keep them straight is to group them by *what they return*, because the most common mistakes come from expecting one width and getting another.

Family	Operators	Result
Arithmetic	`+ - * / %`	a number, sized by the surrounding expression
Bitwise	`& | ^ ~`	same width as the operands, computed bit by bit
Reduction	`& | ^ ~& ~| ~^`	1 bit
Logical	`&& || !`	1 bit (true or false)
Relational	`== != < <= > >=`	1 bit
Shift	`<< >> <<< >>>`	width of the left operand
Conditional	`? :`	width of the chosen branch
Concatenation, replication	`{a, b}`, `{n{a}}`	sum of the parts

The sections below take them in turn. Hardware cost matters here: each operator becomes real logic, so `a + b` is an adder and `a * b` is a multiplier, and a division by a non-constant is expensive enough that you should hesitate before writing it.

4.14 Bitwise vs. logical

The bitwise operators `&`, `|`, `^` and `~` work on each bit position separately, and the result is as wide as the operands. The logical operators `&&`, `||` and `!` treat each whole operand as a single truth value (zero is false, anything else is true) and return one bit.

```
logic [3:0] a = 4'b1010;
logic [3:0] b = 4'b0101;

assign x = a & b;     // 4'b0000: no bit position has a 1 in both
assign y = a && b;    // 1'b1: a is non-zero, and so is b
```

Both lines look like “a AND b”, and they give opposite answers. For single-bit signals the two are interchangeable, which is why mixing them up usually goes unnoticed until the signals become vectors. Use `&` and `|` for data, and `&&` and `||` only in conditions.

4.15 Reduction, relational and shift operators

A **reduction** operator is a bitwise operator written with a single operand. It combines all the bits of a vector into one:

```
logic [5:0] a = 6'b101010;
logic c, d, e;

assign c = &a;   // AND of all bits: 0
assign d = |a;   // OR of all bits: 1
assign e = ^a;   // XOR of all bits: 1, the parity of a
```

Reduction operators are a compact way to write a wide gate: `&a` is a six-input AND, and `^a` is a parity tree.

The **relational** operators `== != < <= > >=` compare two values and return one bit. `==` gives `X` if either operand has an `X` or `Z` bit. There is also a case-equality operator, `===`, that compares `X` and `Z` literally, but it exists only for simulation and is not synthesizable. The **shift** operators move the bits of a vector: `<<` and `>>` shift in zeros, and `<<<` and `>>>` are the arithmetic versions, which keep the sign bit of a signed operand when shifting right.

Operator precedence mostly follows C, but you should not rely on remembering it. Add parentheses.

4.16 Conditional operator, concatenation and replication

The **conditional** operator `cond ? x : y` selects between two values. In hardware it is a multiplexer: both `x` and `y` exist, and `cond` chooses which one reaches the output. It can be nested, which is how we build larger multiplexers below.

The **concatenation** operator `{}` joins bits, vectors or expressions into one longer vector. The **replication** operator `{n{...}}` repeats a value `n` times:

```
logic [3:0] a = 4'b1010;
logic [3:0] b = 4'b0101;
logic [7:0] c;
logic [1:0] d = 2'b11;
logic [5:0] e;

assign c = {a, b};     // 8'b10100101
assign e = {3{d}};     // 6'b111111
```

Concatenation is also valid on the left of an assignment, which we use below to capture the carry of an adder.

4.17 Example: 4-bit 4:1 multiplexer

A 4-bit, 4-input multiplexer passes one of four 4-bit inputs to the output, chosen by a 2-bit select signal. Built from nested conditional operators:

```
module mux4to1 (
    input  logic [3:0] in0, in1, in2, in3,   // four 4-bit data inputs
    input  logic [1:0] sel,                  // 2-bit select
    output logic [3:0] out                   // 4-bit output
);
  assign out = (sel == 2'b00) ? in0 :
               (sel == 2'b01) ? in1 :
               (sel == 2'b10) ? in2 :
                                in3;
endmodule
```

Each `?:` tests one value of `sel` and, if it does not match, hands the decision to the next. The last branch, `in3`, is the default that is taken when no earlier condition is true, so there is always a value for `out`. The synthesized result is in Figure 4.3.

4.18 Example: 4-bit adder

The adder adds two 4-bit numbers and produces a 4-bit sum and a carry-out bit. We build it exactly as on paper: a half adder for the least significant bit, then a full adder for each following bit, with an internal carry wire `c1`, `c2`, `c3` passed from one stage to the next.

```
module adder4bit (
    input  logic [3:0] a, b,
    output logic [3:0] sum,
    output logic       carry_out
);
  logic c1, c2, c3;                      // carries between stages

  // Half adder: least significant bit
  assign sum[0] = a[0] ^ b[0];
```

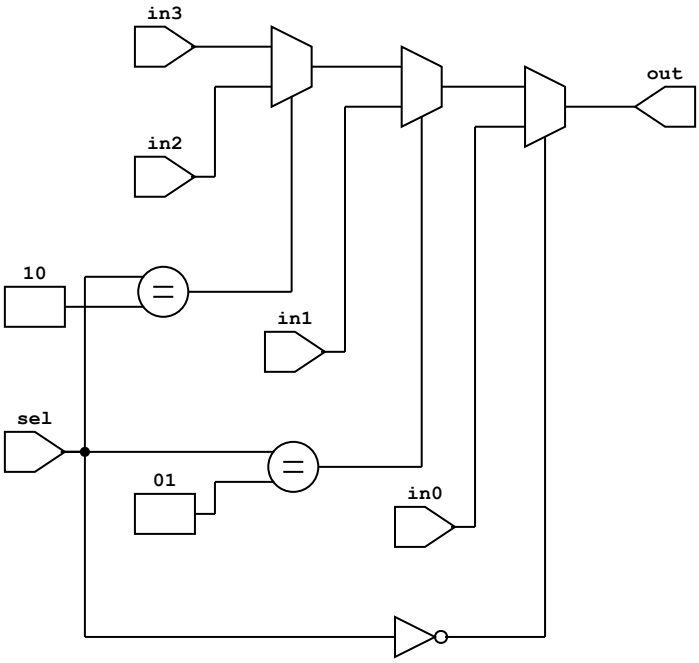


Figure 4.3: The `mux4to1` module as drawn by Yosys.

```
  assign c1     = a[0] & b[0];

  // Full adders
  assign sum[1] = a[1] ^ b[1] ^ c1;
  assign c2     = (a[1] & b[1]) | (c1 & (a[1] ^ b[1]));

  assign sum[2] = a[2] ^ b[2] ^ c2;
  assign c3     = (a[2] & b[2]) | (c2 & (a[2] ^ b[2]));

  assign sum[3]    = a[3] ^ b[3] ^ c3;
  assign carry_out = (a[3] & b[3]) | (c3 & (a[3] ^ b[3]));
endmodule
```

Writing out the gate equations is useful once, to see what an adder is. In practice we let the tool do it. The + operator, with a concatenation on the left to catch the carry, describes the same adder in one line:

```
assign {carry_out, sum} = a + b;
```

The 5-bit result of `a + b` is split across the 1-bit `carry_out` and the 4-bit `sum`. Figure 4.4 shows the hand-built carry chain, and Figure 4.5 the single adder block that a synthesis tool is free to map onto the dedicated carry logic of an FPGA.

4.19 From modules to behavior

Everything we wrote in this chapter is combinational: outputs depend only on the current inputs, and nothing remembers anything. That was enough to describe multiplexers and adders, but a processor also needs state. That calls for procedural blocks, which the next chapter introduces: `always_comb` for larger combinational descriptions, and `always_ff` for registers that update on a clock edge.

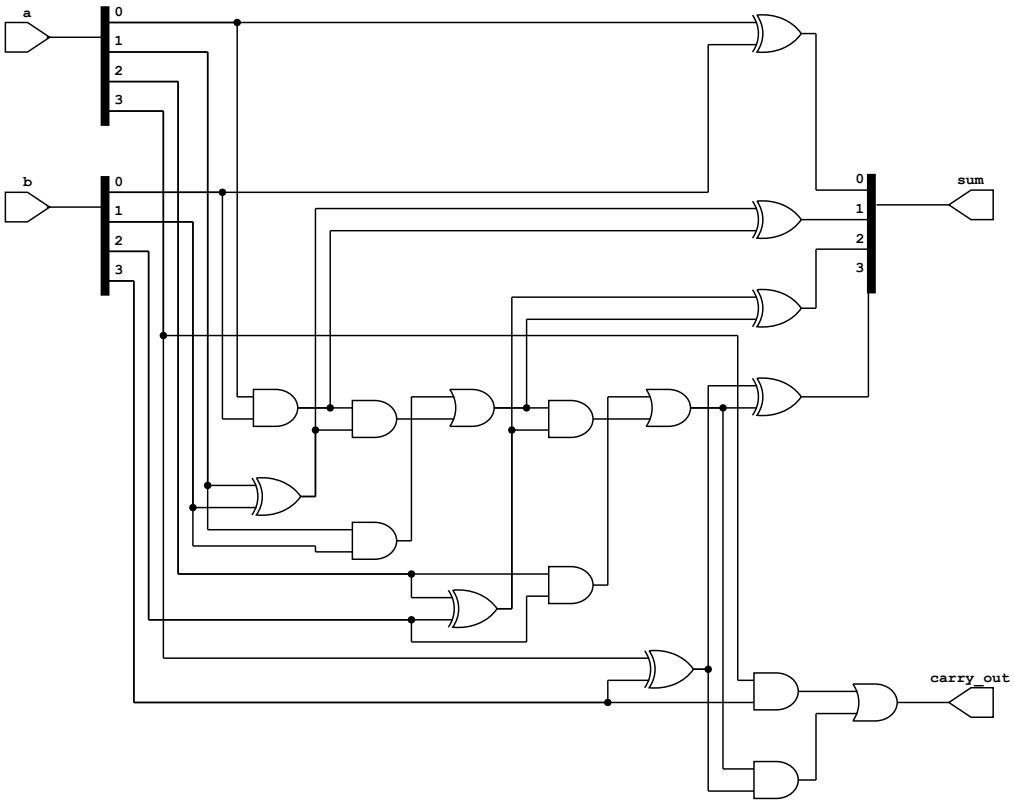


Figure 4.4: The gate-level `adder4bit`: a half adder followed by three full adders, with the carry rippling from bit to bit.

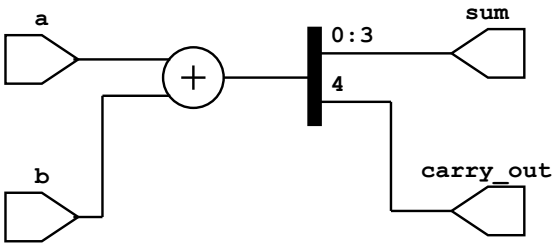


Figure 4.5: The same adder written with the + operator.

5 Designing with SystemVerilog

Designing with SystemVerilog

5.1 Goal for this week

- Tell the two styles of hardware description apart: behavioral and structural
- Describe sequential logic with `always_ff`: flip-flops, resets, registers with an enable, and counters
- Know what a latch is, and write code that cannot produce one by accident
- Describe combinational logic with `always_comb`: multiplexers and decoders
- Explain the difference between `=` and `<=`, and why each block uses one of them
- Assemble a design from smaller modules, and test it with a testbench

5.2 Two styles of design

There are two ways to describe a piece of hardware in SystemVerilog, and a real design uses both.

In **behavioral** design we describe *what* a block does: its outputs as a function of its inputs and, for sequential logic, of its stored state. We write that with `assign` and with `always` blocks, and leave it to the synthesis tool to pick the gates and flip-flops. In **structural** design we describe *how* a block is composed of other blocks: we create instances of smaller modules and connect their ports with wires, much as one would wire chips on a board.

	Behavioral	Structural
Describes	**what** a block does	**how** it is built from other blocks
Written with	`assign`, `always_*`	module **instances**
Example	a register, a multiplexer	a pipeline stage

The two styles meet at the leaf cells. In the HIPV SoC that we use as a running example, the folder `rtl/generic` holds small, reusable cells (`flopenr`, `mux2`, `mux3`, `adder`) written in behavioral style, and the larger modules of the processor instantiate them structurally. This

chapter begins with the behavioral style, because those leaf cells have to be written before they can be assembled, and ends with the structural style.

5.3 The `always` block

The previous chapter described logic one line at a time with `assign`. That is enough for an adder, but awkward the moment a function needs an `if` or a `case`. For those we use an **`always` block**: a block of statements that describes how a set of outputs depends on a set of inputs.

SystemVerilog has three kinds of `always` block for hardware, and which one you use is dictated by what you are building:

Block	Describes	Runs when
`always_comb`	combinational logic	any signal it **reads** changes
`always_ff @(posedge clk)`	flip-flops	the clock edge
`always_latch`	latches	any of its inputs changes

The names are a promise to the tool: if the code inside does not describe the kind of hardware the block name claims, the tool reports it. Older Verilog has a single `always @(...)` for all three purposes, and a whole class of bugs came from getting it wrong without any warning.

Two facts about `always` blocks are worth keeping in mind from the start. First, every `always` block in a module is a separate piece of hardware that exists all the time, like an `assign`, so they all run *concurrently* and the order in which you write them does not matter. Second, the statements *inside* one block run in the order written, like a small program. A block is a description of hardware, not a function that gets called.

5.4 Sequential logic: describe the hardware you want

Hardware description is not programming, and the habit that separates the two is this: **decide on the hardware first, then write the code that describes it.** If you cannot sketch the flip-flops and gates you expect, you are not ready to write the `always` block, and the synthesis tool will build something, but not necessarily what you meant.

So for every circuit in this chapter we follow the same four steps. First, the hardware we want. Second, its **waveform**, which says what the circuit must do over time and is the most precise specification there is. Third, the code that describes it. Fourth, a walk through the

waveform to check that the code matches it. Learning to read waveforms is half of the work: it is also how you will debug in simulation.

A convention for all the diagrams in this chapter: the clock rises at the start of each numbered column, and input signals change in the *middle* of a column, after the clock has fallen. That way an input never changes at the same instant as a rising edge, and it is always clear which value the circuit sampled.

5.5 The D flip-flop

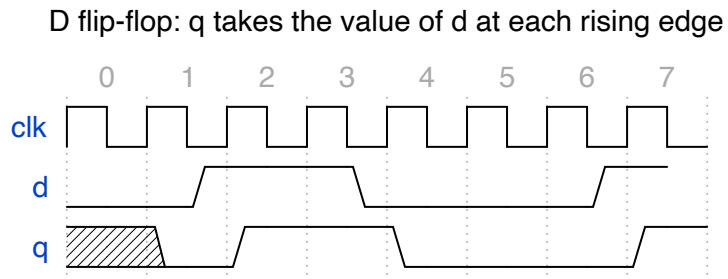


Figure 5.1: A D flip-flop: `q` takes the value of `d` at each rising clock edge.

The flip-flop is the memory element of synchronous design. It has a data input `d`, a clock `clk` and an output `q`. At every rising edge of the clock it copies `d` to `q`, and between edges it holds that value no matter what `d` does (Figure 5.1).

Follow the waveform from left to right. In column 0, `q` is unknown (the hatched area): nothing has been stored yet. At the rising edge that starts column 1, `d` is 0, so `q` becomes 0. In the middle of column 1, `d` rises, and `q` does **not** react: the flip-flop is not looking. At the next rising edge, starting column 2, it samples `d = 1` and `q` rises. The same happens in reverse when `d` falls in the middle of column 3: `q` falls only at the edge that starts column 4. A flip-flop is a circuit that looks at its input at one instant per cycle.

The code is as short as the circuit:

```
module dff (
    input  logic clk, d,
    output logic q
);
  always_ff @(posedge clk) begin
    q <= d;
```

```
  end
endmodule
```

The sensitivity list `@(posedge clk)` names the event that makes the block run, and the block runs *only* then. Between clock edges nothing happens and `q` keeps its value: that holding is the memory. The operator is `<=`, the non-blocking assignment, which we explain at the end of the chapter; for now, remember that it goes with `always_ff`.

5.6 The latch

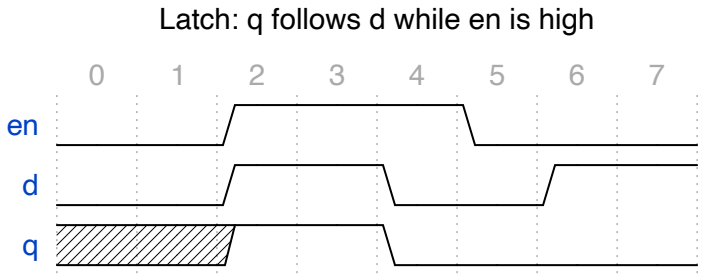


Figure 5.2: A latch: `q` follows `d` while `en` is high, and holds its value while `en` is low.

A **latch** is a memory element too, but it looks at its input for a whole *level*, not at one instant. While the enable `en` is high, the latch is *transparent*: `q` follows `d` continuously. When `en` falls, the latch closes and `q` holds the last value (Figure 5.2).

In the waveform, `en` is high during columns 2 to 4. As soon as it rises, `q` takes the value of `d`, which is 1. When `d` falls at the start of column 4, `q` falls with it, because the latch is still open. `en` falls at the start of column 5, and from then on `q` stays 0: when `d` rises again in column 6, the latch is closed and does not notice. Compare this with the flip-flop of the previous waveform, where `q` could only change at a clock edge.

```
always_latch begin
  if (en) q <= d;
end
```

A latch has no clock edge, so a signal passing through it can ripple along for as long as `en` is high, which makes timing analysis difficult. In a synchronous design we want flip-flops everywhere, and we use a latch only on purpose. The reason to know it exists is that it is easy to build one *by accident*, and the section on combinational logic below shows how.

5.7 Adding a reset

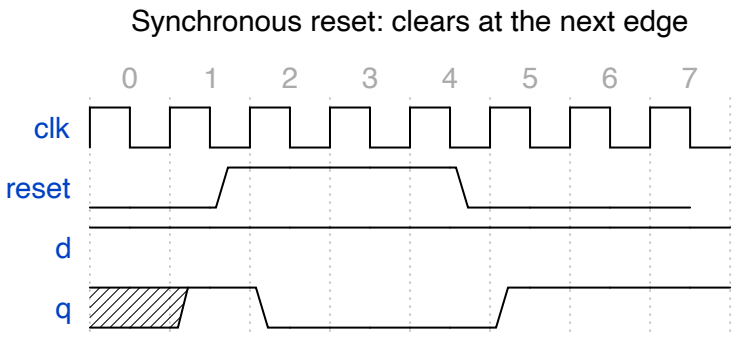


Figure 5.3: Synchronous reset: `q` clears at the next rising edge after `reset` rises.

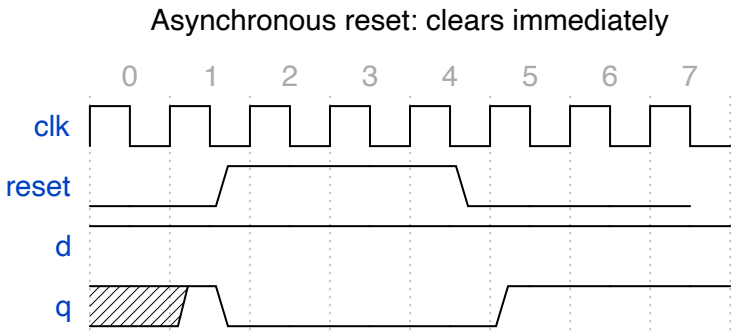


Figure 5.4: Asynchronous reset: `q` clears the moment `reset` rises, without waiting for a clock edge.

A flip-flop that powers up with an unknown value is a bug waiting to happen, so registers get a **reset** that forces a known value. There are two ways to write it, and both clear `q` when `reset` is high. They differ in *when* the clear takes effect.

With a **synchronous** reset (Figure 5.3), `reset` is just another input that the flip-flop looks at on the clock edge. In the waveform `d` stays 1. `q` becomes 1 at the edge that starts column 1. `reset` rises in the middle of column 1 and `q` does not move; only at the edge starting column 2 does the flip-flop see `reset` high and clear. It stays cleared through the edges of columns 3 and 4 because `reset` is still high. `reset` falls in the middle of column 4, and at the edge starting column 5 the flip-flop loads `d` again.

```
always_ff @(posedge clk) begin
  if (reset) q <= 1'b0;
  else       q <= d;
end
```

With an **asynchronous** reset (Figure 5.4), the flip-flop reacts to `reset` itself, whatever the clock is doing. The same stimulus now clears `q` in the *middle* of column 1, the instant `reset` rises, not at the next edge. It stays 0 while `reset` is high, and after `reset` falls it waits for the next edge, at the start of column 5, to load `d`.

```
always_ff @(posedge clk or posedge reset) begin
  if (reset) q <= 1'b0;
  else       q <= d;
end
```

This is why `reset` appears in the sensitivity list next to `posedge clk`: the block must also run when `reset` rises. An asynchronous reset needs a flip-flop with a dedicated reset pin, and its release matters: if `reset` falls too close to a clock edge the flip-flop can misbehave, a problem we return to when we discuss synchronizers. A synchronous reset needs a running clock, but is just logic in front of the D input (or a dedicated set/reset input on an FPGA flip-flop, like the one in the slice of the previous chapter).

The HIPV SoC chooses one style for the whole design: its coding standard requires a **synchronous** reset throughout. That is the style we use from now on. Pick one style, use it everywhere, and know which one you picked.

5.8 Registers and the enable signal

A single flip-flop stores one bit. A **register** is a group of flip-flops that load together, and in a processor most registers also need an **enable**: a signal that says "load a new value at this edge", and otherwise "keep the old one". A pipeline register that must hold its contents while the stage ahead is busy is exactly that. This is the most-instantiated cell in HIPV:

```
module flopenr #(parameter WIDTH = 8) (
  input  logic             clk, reset, en,
  input  logic [WIDTH-1:0] D,
  output logic [WIDTH-1:0] Q);

  always_ff @(posedge clk)
    if (reset) Q <= '0;
```

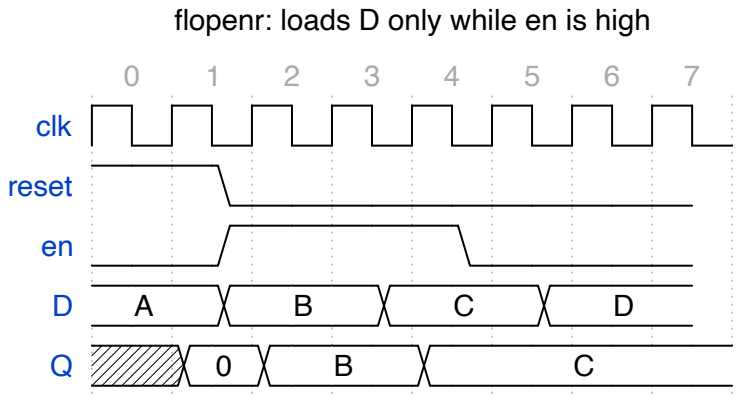


Figure 5.5: `flopenr`: after the reset, `Q` loads `D` only at the edges where `en` is high.

```
    else if (en) Q <= D;

endmodule
```

The module is the D flip-flop with a synchronous reset, plus one more condition. `WIDTH` is a parameter, so the same description serves for a 1-bit flag and for the 32-bit program counter. The priority is spelled out by the `if`/`else if` chain: reset wins, then enable, and if neither is active there is no assignment at all, so the register keeps its value. That "no assignment means hold" is legal inside `always_ff`, since a flip-flop naturally remembers; it is the opposite of the rule for `always_comb`, where it would build a latch.

Figure 5.5 shows the behavior. `reset` is high around the first edge, so at the start of column 1 `Q` loads 0. `D` carries the values A, B, C, D, changing mid-column. `en` is high for the edges starting columns 2, 3 and 4. At column 2 the register loads B. At column 3 `D` is still B, so `Q` stays B. At column 4 `D` is C, so `Q` loads C. Then `en` falls in the middle of column 4, and from the edge at column 5 on the register ignores `D`, even though `D` keeps changing: `Q` holds C. Figure 5.6 is what the synthesis tool builds: one flip-flop cell with clock, data, enable and synchronous reset inputs, which is what an FPGA slice flip-flop provides directly (the clock-enable and set/reset inputs of the previous chapter).

5.9 Counters: two ways to write one

A counter is a register whose next value is its own current value plus one. Its waveform is the simplest of any sequential circuit: after the reset, the output goes up by one at every

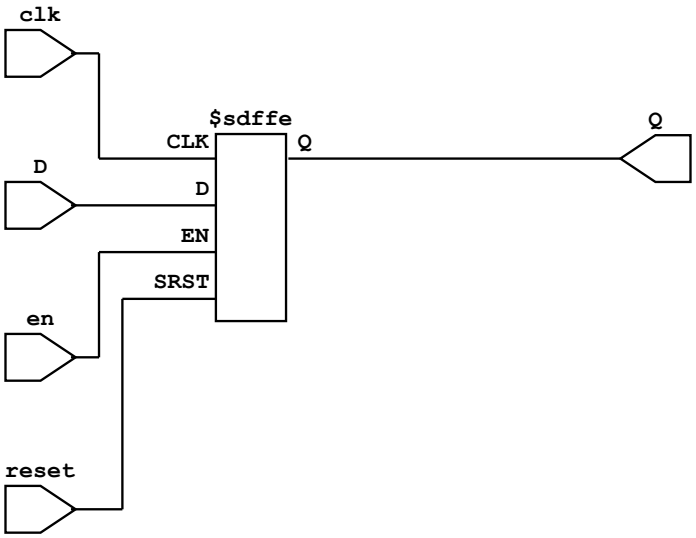


Figure 5.6: The `flopenr` module (8-bit by default), drawn by Yosys from the real HIPV file. `$sdffe` is a flip-flop with **s**ynchronous reset and **e**nable.

clock edge (Figure 5.7). There are two common ways to write it, and the best habit is to know both.

Way 1: everything in one `always_ff`. The straightforward form says, at each edge, "if reset, clear; otherwise `count <= count + 1`":

```
module counter4 (
    input  logic       clk,
    input  logic       reset,
    output logic [3:0] count
);
  always_ff @(posedge clk) begin
    if (reset) count <= 4'b0000;
    else       count <= count + 1;
  end
endmodule
```

At the edge starting column 1, `reset` is still high and `count` loads 0. From then on `reset` is low, and at each edge `count` takes its own value plus one: 1, 2, 3 and so on. When it reaches 15, the next `+ 1` wraps around to 0, because the result is truncated to 4 bits.

Way 2: next-state logic and a register. The second form splits the description in two.

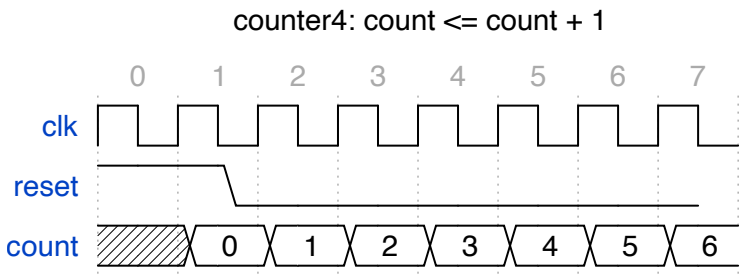


Figure 5.7: Timing of `counter4`: after the reset, `count` increases by one at every rising edge.

A combinational block computes what the next value *would be*, `count_next`, and a plain register stores it:

```
module counter4_next (
    input  logic       clk,
    input  logic       reset,
    output logic [3:0] count
);
  logic [3:0] count_next;

  always_comb begin
    count_next = count + 1;
  end

  always_ff @(posedge clk) begin
    if (reset) count <= 4'b0000;
    else       count <= count_next;
  end
endmodule
```

The waveform now has one more signal (Figure 5.8). `count_next` is combinational, so it changes as soon as `count` does, and is always one higher. In column 1, `count` is 0 and `count_next` is already 1, but the register ignores it at the edge starting column 1 because `reset` is high. At the edge starting column 2 the register copies the 1, and so on.

Both modules synthesize to exactly the same hardware: an adder, a register with a synchronous reset, and the feedback wire (Figure 5.9).

So why bother with the second form? Because it separates two questions: *what is the next value?* (combinational logic) and *when do we store it?* (the register). For a counter the

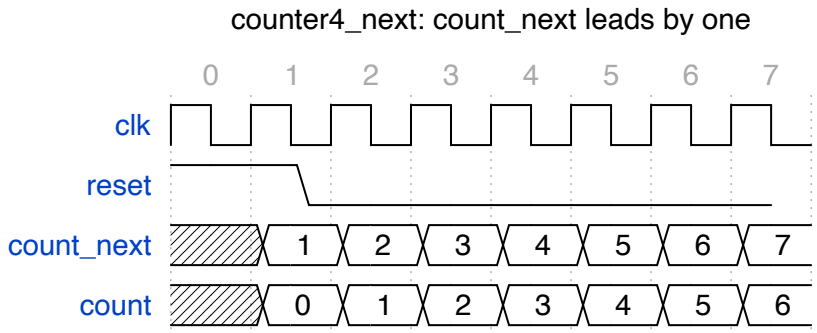


Figure 5.8: Timing of `counter4_next`: `count_next` is always one ahead of `count`, and the register copies it at each edge.

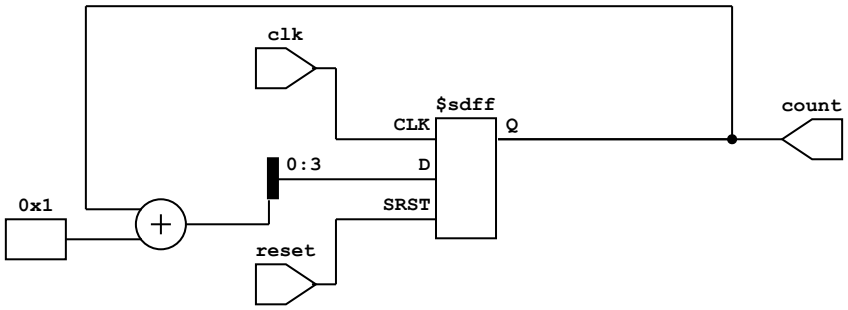


Figure 5.9: Both counter descriptions give this circuit, drawn by Yosys: the register output feeds the output port and the adder, and `reset` drives the synchronous reset input.

answer to the first is trivial, and way 1 is perfectly fine. For anything bigger, such as a state machine (see the next chapter) or a pipeline stage, the next-state logic can be dozens of lines of `if` and `case`, and keeping it out of the clocked block makes it far easier to read and to check. The HIPV SoC uses this idea in its 64-bit timer counter, `cntr`, which computes `plus_one = count + 1` with an `assign` and feeds it to a register cell with an enable and a clear, instead of writing the increment inside the clocked block.

5.10 Combinational logic

We now do the same for combinational circuits. The method is identical: hardware, waveform, code, walk through. The difference is in the waveform: there is no clock, because a combinational output depends only on the current inputs, so it changes as soon as an input does.

The block for it is `always_comb`. It re-evaluates whenever any signal it reads changes. It is closer to a spreadsheet formula that recalculates on every input edit than to a function you call. You write no sensitivity list, since the tool works it out from the signals the block reads. The assignment inside is the blocking `=`, which reads as "this is what the output equals right now". There is no state to remember, which gives one rule that matters more than any other: **every output must be assigned on every path through the block**.

5.11 The multiplexer

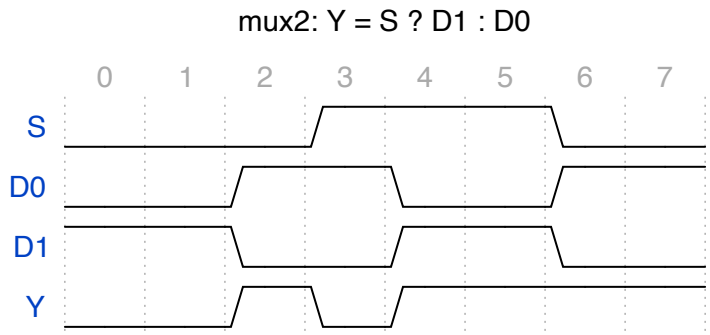


Figure 5.10: A 2-to-1 multiplexer: `Y` follows `D0` while `S` is 0 and `D1` while `S` is 1, with no clock.

A multiplexer selects one of several inputs. The 2-to-1 version is the first cell in `rtl/generic`, and HIPV writes it as a continuous assignment:

```
module mux2 #(parameter WIDTH = 8) (
  input  logic [WIDTH-1:0] D0, D1,
  input  logic             S,
  output logic [WIDTH-1:0] Y);

  assign Y = S ? D1 : D0;

endmodule
```

Figure 5.10 shows what it does. There is no clock: `Y` equals `D0` while `S` is 0 and `D1` while `S` is 1, and it changes the moment the selected input or the select changes. At the start of column 2 `D0` rises, and since `S` is 0, so does `Y`. At the start of column 3 `S` rises, `Y` switches over to `D1`, which is 0, and falls. At column 4 `D1` rises, `Y` follows. At column 6 `S` falls and `Y` returns to `D0`, which is 1.

The same multiplexer in an `always_comb` block reads like this:

```
always_comb begin
  if (S) Y = D1;
  else   Y = D0;
end
```

Both give the same circuit. The `assign` form is shorter for one line of logic; the `always_comb` form is better when there are several outputs or several conditions. For a multiplexer with more than two inputs, the `case` statement is clearer than a chain of `?:` operators. HIPV's 3-to-1 multiplexer, written with a chain of conditional operators in the repository, looks like this as an `always_comb` with `case`:

```
module mux3 #(parameter WIDTH = 8) (
  input  logic [WIDTH-1:0] D0, D1, D2,
  input  logic [1:0]       S,
  output logic [WIDTH-1:0] Y);

  always_comb begin
    case (S)
      2'b01:   Y = D1;
      2'b10:   Y = D2;
      default: Y = D0;    // 2'b00, and 2'b11 falls back to D0
    endcase
  end
endmodule
```

The `case` compares `S` against each listed value and runs the matching branch. The **`default`** branch is not optional: it makes sure `Y` is assigned for *every* value of `S`, including the one the designer never intended to use (`2'b11`), and values containing X or Z in simulation. The synthesized circuit (Figure 5.11) is two comparators and two 2-to-1 multiplexers in series.

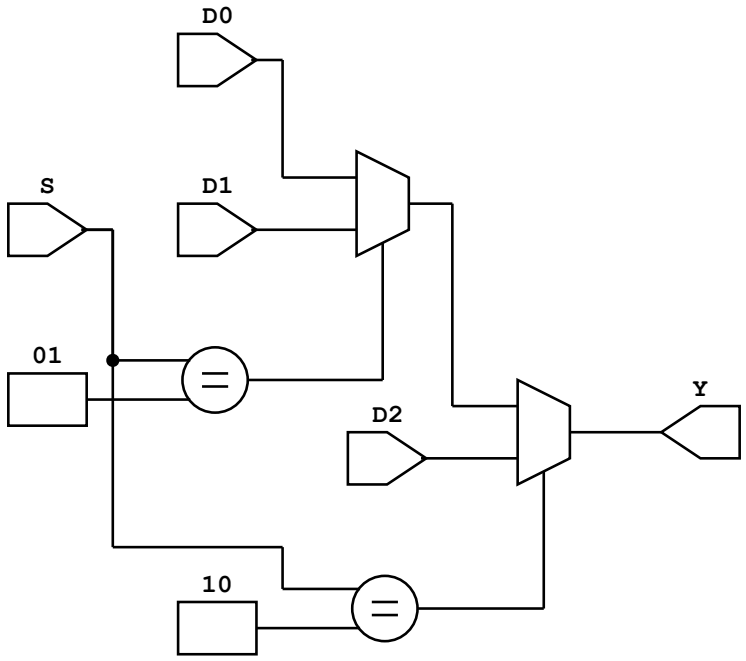


Figure 5.11: The `mux3` cell as drawn by Yosys from the HIPV file: `S == 2'b10` selects `D2`, otherwise `S == 2'b01` selects `D1`, otherwise `D0`.

5.12 The decoder

A **decoder** turns an encoded value into a set of separate control signals, and it is everywhere in a processor: the instruction decoder turns an opcode into control signals, and the bus decoder turns an address into "which device is being accessed". The second one is a clean example. The HIPV crossbar has five slaves (instruction memory, data memory, UART, timer and GPIO), each mapped to a region of the address space, and the decoder produces a **one-hot** select, one bit per slave, plus an error flag for an address that belongs to no slave. Here it is with the address map written out:

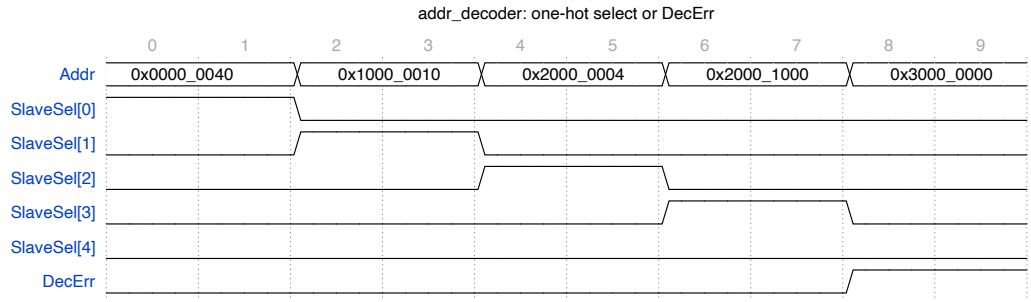


Figure 5.12: An address decoder: one `SlaveSel` bit goes high for each mapped region, and `DecErr` goes high for an unmapped address.

```
module addr_decoder (
    input  logic [31:0] Addr,
    output logic [4:0]  SlaveSel,
    output logic        DecErr
);
  always_comb begin
    SlaveSel = '0;                                      // default: nothing selected
    if (Addr[31:16] == 16'h0000)  SlaveSel[0] = 1'b1;   // instruction memory, 64 KiB
    if (Addr[31:16] == 16'h1000)  SlaveSel[1] = 1'b1;   // data memory, 64 KiB
    if (Addr[31:12] == 20'h20000) SlaveSel[2] = 1'b1;   // UART, 4 KiB
    if (Addr[31:12] == 20'h20001) SlaveSel[3] = 1'b1;   // timer, 4 KiB
    if (Addr[31:12] == 20'h20002) SlaveSel[4] = 1'b1;   // GPIO, 4 KiB
  end

  assign DecErr = ~|SlaveSel;
endmodule
```

Read the waveform first (Figure 5.12). Five addresses arrive one after another, two columns each. The first, `0x0000_0040`, lies in the instruction-memory region, so `SlaveSel[0]` goes high. The next addresses hit data memory, the UART and the timer in turn, and each raises exactly one select. The last, `0x3000_0000`, matches no region: every select stays low, so `DecErr` rises. Only one select is ever high, because the regions do not overlap.

Now the code. The first statement in the block is the important one: `SlaveSel = '0` assigns the default, *every bit low*, before anything else. Each `if` then overrides one bit. Because every bit of `SlaveSel` has a value on every path, whatever the address, the block cannot build a latch, and none of the `if`s needs an `else`. This **default first, then override** pattern is the safest way to write `always_comb`, and you will see it throughout HIPV. The last line

uses the reduction operator from the previous chapter: `~|SlaveSel` is true when no select bit is set.

(The real `obi_decoder` in the HIPV repository is the same block, with the base addresses and sizes taken from a shared package, so that adding a slave changes one file.)

5.13 Avoiding latches

The decoder showed the right way. Here is the wrong way:

```
always_comb begin
  if (sel) y = in1;
end
```

When `sel` is 0, no branch assigns `y`. In a spreadsheet an unhandled case would show a blank cell, but in hardware an output cannot go blank. If no branch assigns it, the only way to keep it at *some* value is to remember the previous one, so the tool builds a **latch**, the element from earlier in this chapter (Figure 5.13). It adds a storage element you did not intend, it complicates timing analysis, and it is the most common bug in `always_comb`. Tools warn about it, which is the reason to use `always_comb` instead of the old `always @(*)`.



Figure 5.13: What synthesis builds for an incomplete `if`: a latch whose enable is `sel`, so `y` holds its old value whenever `sel` is 0.

There are three ways to follow the rule. Give every output a **default** at the top of the block, as the decoder does. Or write an `else` on every `if`. Or give every `case` a `default` branch, as in `mux3`. Sequential blocks are different: inside `always_ff`, a missing assignment simply means “keep the value”, which is what a flip-flop does anyway.

5.14 Blocking vs. non-blocking assignment

We have used two assignment operators in an `always` block, and the pairing has been the same every time: `=` in `always_comb`, and `<=` in `always_ff`. Now we can see why.

A **blocking** assignment, `=`, takes effect immediately: the next statement in the block sees the new value. A **non-blocking** assignment, `<=`, only *schedules* the update, which lands at the end of the current time step: the next statement still sees the old value. The difference only shows when there is *interdependency*, when a later statement reads something an earlier one wrote.

5.14.1 In combinational logic

Take three inputs and the function `s = c ^ (a ^ b)`, written with an intermediate signal, once with each operator:

```
module xor_gate (
  input  logic a, b, c,
  output logic s1, s2
);
  logic n1, n2;

  // case 1: blocking
  always_comb begin
    n1 = a ^ b;
    s1 = c ^ n1;
  end

  // case 2: non-blocking
  always_comb begin
    n2 <= a ^ b;
    s2 <= c ^ n2;
  end
endmodule
```

(Some tools refuse non-blocking assignments inside `always_comb` altogether; the example is written this way only to show what would happen.)

Figure 5.14 shows the result for the same inputs. In case 1, `n1` updates immediately, and the next line uses the new `n1`, so `s1` is always `c ^ n1`. In case 2, `n2` is only *scheduled* to change, so `s2 <= c ^ n2` reads the **previous** `n2`. Compare the last two rows: `n` changes at the start of

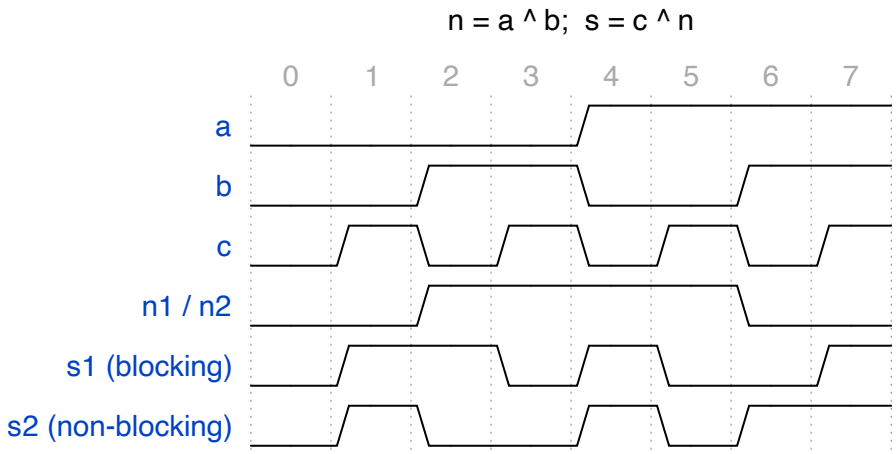


Figure 5.14: The two XOR descriptions: `s1` always matches `c ^ n1`, while `s2` is computed from the previous value of `n2`.

columns 2 and 6, and at exactly those two points `s2` differs from `s1`, because it still uses the old `n`. Case 1 models a combinational circuit; case 2 does not, because its output depends on a stale value. Synthesis tools often ignore the distinction in combinational blocks and build the right gates anyway, which makes it worse: the simulation shows one behavior and the hardware has another, a *simulation/synthesis mismatch*, the kind of bug that is hardest to find.

5.14.2 In sequential logic

The same experiment on two registers in a chain:

```
module chain (
  input  logic clk, a,
  output logic f1, f2
);
  logic b1, b2;

  // case 1: blocking
  always_ff @(posedge clk) begin
    b1 = a;
    f1 = b1;
  end
```

```
  // case 2: non-blocking
  always_ff @(posedge clk) begin
    b2 <= a;
    f2 <= b2;
  end
endmodule
```

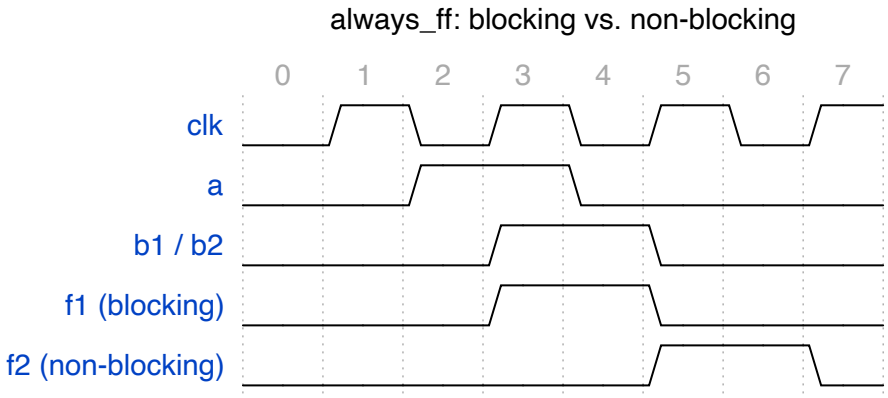


Figure 5.15: Two registers in a chain: with blocking assignment `f1` follows `b1` in the same cycle, with non-blocking assignment `f2` lags by one more clock.

In Figure 5.15 the clock is drawn with two columns per period, so it rises at the start of columns 1, 3, 5 and 7. The input `a` is high during columns 2 and 3, so the edge at the start of column 3 is the one that sees it. In case 1, `b1 = a` happens first, and `f1 = b1` then reads the *new* `b1`, so `f1` gets the value of `a` in the same cycle: both change at the edge starting column 3, and the second register adds nothing. In case 2, `b2 <= a` and `f2 <= b2` are both evaluated before either update lands, so `f2` takes the value `b2` had *before* this edge. `b2` rises at column 3, but `f2` only at the next edge, at column 5, and it falls one edge after `b2` does, at column 7. That is a two-stage chain, which is what two registers in a row should be. This is the behavior every pipeline relies on, and it is why the swap `a <= b; b <= a;` works with `<=` and silently fails with `=`.

5.14.3 The rule

	Blocking =	Non-blocking <=
Update takes effect	immediately	at the end of the time step
The next statement reads	the new value	the old value

	Blocking =	Non-blocking <=
Use in	`always_comb`	`always_ff`

= for combinational logic, <= for clocked logic, every time. Do not mix the two in one block.

5.15 Structural design: module instances

Behavioral modeling describes what a block *does*. **Structural** modeling describes a block by how it is *composed* of simpler modules. Each copy of a smaller module is an **instance**, and it has a unique name. Here is the smallest useful example, a 4:1 multiplexer assembled from three copies of a 2:1 multiplexer:

```
module mux2to1 (
    input  logic a, b, sel,
    output logic y
);
  assign y = sel ? b : a;
endmodule

module mux4to1_struct (
    input  logic [3:0] d,
    input  logic [1:0] sel,
    output logic       y
);
  logic y0, y1;                 // wires between the instances

  mux2to1 mux_low   (.a(d[0]), .b(d[1]), .sel(sel[0]), .y(y0));
  mux2to1 mux_high  (.a(d[2]), .b(d[3]), .sel(sel[0]), .y(y1));
  mux2to1 mux_final (.a(y0),   .b(y1),   .sel(sel[1]), .y(y));
endmodule
```

An instantiation has three parts. First comes the **module name** (`mux2to1`), then the **instance name** you choose (`mux_final`), then in parentheses the **port connections**. The notation `.a(y0)` reads “the module’s port `a` is connected to my signal `y0`”. These connections by name make the code easy to read and safe to reorder. There is also a positional form, `mux2to1 mux_final (y0, y1, sel[1], y);`, which is shorter and fails silently if someone later reorders the ports, so we avoid it. Figure 5.16 shows the result: the tool leaves the three instances as boxes, exactly as the hierarchy appears in the code.

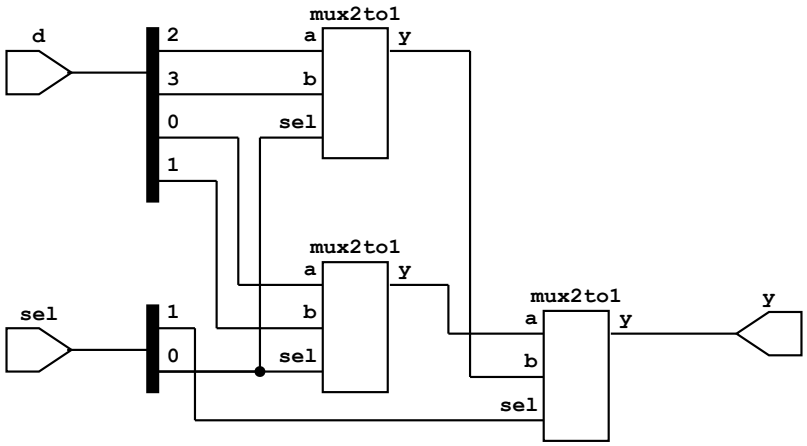


Figure 5.16: The structurally built `mux4to1_struct`: two instances select within a pair, and a third selects between the two results.

5.15.1 Parameters

A **parameter** makes a module configurable, which is how one description serves every width. We have been using this all along: `flopenr` declares `parameter WIDTH = 8`, and each instance picks its own. The override goes between the module name and the instance name, after a #:

```
flopenr #(32) pcreg (...);          // a 32-bit instance, by position
flopenr #(.WIDTH(32)) pcreg (...);  // the same, by name
```

The by-name form is safer when a module has several parameters. A `localparam` is a constant inside a module that, unlike a `parameter`, cannot be overridden from outside, which makes it the right choice for derived constants.

5.15.2 A real example: the program counter of HIPV

HIPV's fetch unit is almost entirely structural. Its program-counter loop is built from three of the leaf cells we wrote above. The code is the actual instantiation, reduced to this loop:

```
module pc_unit (
    input  logic        clk, reset,
    input  logic        PCEnF,
```

```
    input  logic        PCSrcE,
    input  logic [31:0] PCTargetE,
    output logic [31:0] PCF
);
  logic [31:0] PCNextF, PCPlus4F;

  // next PC selection: sequential or redirect from Execute
  mux2 #(32) pcmux (
      .D0(PCPlus4F), .D1(PCTargetE), .S(PCSrcE), .Y(PCNextF));

  // PC register
  flopenr #(32) pcreg (
      .clk(clk), .reset(reset), .en(PCEnF), .D(PCNextF), .Q(PCF));

  // sequential PC
  adder #(32) pcadd4 (
      .A(PCF), .B(32'd4), .Y(PCPlus4F));
endmodule
```

Everything inside the parentheses is a wire name, and nothing here describes any behavior: it only says which cell goes where. The register holds the PC, the adder computes `PC + 4`, and the multiplexer chooses between that and a branch target (Figure 5.17). Note how the signals form a loop through the register, which is the only place where the loop is broken by a clock.

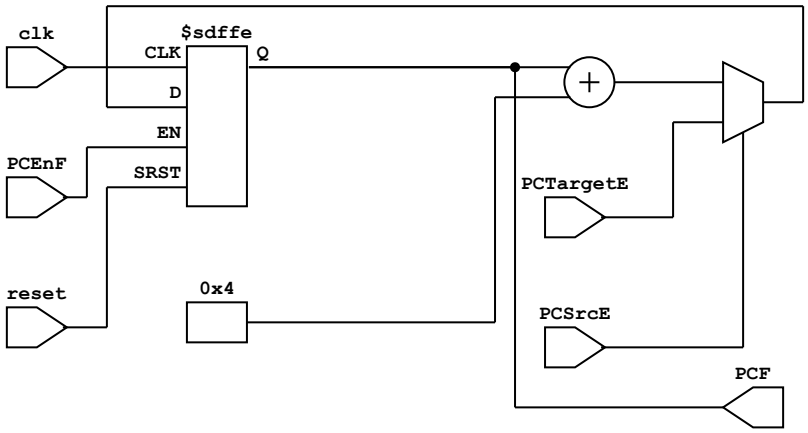


Figure 5.17: The PC loop of HIPV, drawn by Yosys with the three cells expanded into their hardware: the `$sdffe` register, the `+4` adder, and the next-PC multiplexer.

5.16 Testbenches

A **testbench** is a module that tests another module, the *device under test* (DUT). It applies inputs to the DUT and, ideally, checks that the outputs are right. Its inputs and the expected outputs are the *test vectors*. A testbench has no ports of its own: it sits at the top of the simulation, so everything it needs is declared inside as plain `logic` signals and wired to the DUT by name, like any other instance.

```
module tb_mux4to1;
  logic [3:0] d;
  logic [1:0] sel;
  logic       y;

  mux4to1_struct dut (.d(d), .sel(sel), .y(y));

  initial begin
    d = 4'b1010;
    for (int i = 0; i < 4; i++) begin
      sel = i[1:0];
      #10;
      $display("d=%b sel=%b y=%b", d, sel, y);
      if (y !== d[sel]) $error("sel=%b: expected %b, got %b", sel, d[sel], y);
    end
    $finish;
  end
endmodule
```

Everything happens in the `initial` block, which starts at time zero and runs once. It holds `d` fixed and sweeps `sel` through all four values. After each change it waits `#10` time units, `$display` prints the inputs and the output, and the `if` checks `y` against the bit of `d` that `sel` selects. The `#10` delay is not needed by the circuit, since a combinational output settles almost instantly. It gives each test case its own slice of simulation time, so the printed lines and the waveform stay readable. The comparison uses `!==`, which also catches an `X` on the output where `!=` would stay silent. `$finish` ends the simulation.

These statements (`initial`, `#10`, `$display`, `$finish`) exist only in simulation. A synthesizer has no hardware for “wait 10 time units”, which is the line between the code you simulate and the code you synthesize.

5.17 Testbench for a sequential design

A registered design only changes on a clock edge, so the testbench has to supply that clock; nothing else in the simulation will. Here it is for the counter of this chapter:

```
module tb_counter;
  logic       clk = 0;
  logic       reset = 1;
  logic [3:0] count;

  counter4 dut (.clk(clk), .reset(reset), .count(count));

  always #5 clk = ~clk;              // clock generator: period of 10 units

  initial begin
    #12 reset = 0;                   // release reset between clock edges
    #100;
    $display("count=%0d", count);
    $finish;
  end
endmodule
```

The separate `always #5 clk = ~clk;` has no sensitivity list and no guard, so it keeps re-executing for the whole simulation and flips `clk` every 5 time units. That gives a free-running clock with a period of 10. It runs concurrently with the `initial` block below it: every `initial` and `always` block in a module starts at time zero and proceeds independently, which is why the clock generator lives in its own block.

The stimulus holds `reset` high for the first clock edge (at time 5), then releases it at time 12, between edges, so that no input changes exactly when the clock rises, the same convention as in our waveforms. After the remaining ten rising edges the counter holds 10, which `$display` prints. The key difference from the combinational testbench is the waiting. A combinational output is valid almost immediately after its inputs change, but a register updates only once per clock edge, so a sequential testbench must always budget enough simulated time, or wait for a specific number of `@(posedge clk)` events, before checking the outputs.

`$finish` matters even more here. The clock generator never terminates on its own, so without an explicit `$finish` the simulation would run forever.

5.18 Running a simulation

Any SystemVerilog simulator can run these testbenches. With the open-source Icarus Verilog, you compile the design together with its testbench and run the result:

```
iverilog -g2012 -o sim_counter counter4.sv tb_counter.sv
vvp sim_counter
```

The `-g2012` option selects the SystemVerilog language level. To look at waveforms instead of printed lines, add two statements at the start of the `initial` block, and open the resulting file in GTKWave:

```
$dumpfile("tb_counter.vcd");
$dumpvars(0, tb_counter);
```

The waveforms you get are the same kind as the ones drawn in this chapter, and reading them is the same skill. Checking results by eye does not scale beyond a handful of signals, though. A testbench that compares outputs against expected values, as ours does with `$error`, can be run again after every change and report a clear pass or fail. Later in the course we move this idea to a Python framework, cocotb, which makes writing such checks much easier.

5.19 From design to the pipeline

Everything introduced here is the working vocabulary of the rest of the course. `always_ff` with non-blocking assignment and a synchronous reset is how every pipeline register is built, through `flopenr` and its siblings. `always_comb` with a default at the top is how the ALU and the decoders avoid latch inference. Instances and parameters are how those cells are assembled into stages, and stages into a processor. The pipeline chapters introduce no new syntax: they put these constructs to work on a larger example.

6 Finite State Machines

Finite State Machines

6.1 Goal for this week

- Know what a finite state machine is, and where it is used in a digital system
- Describe an FSM with a state diagram and with an ASM chart
- Tell Moore and Mealy machines apart, and choose a state encoding
- Write an FSM in SystemVerilog as state register, next-state logic and output logic
- Build a complete system from a tick generator and an FSM: a traffic light

6.2 Theoretical foundations

The first part of this chapter is the theory: what a finite state machine is, the ways to draw one, how it maps onto hardware, and how it is written in SystemVerilog. We use a small example throughout, a controller for window blinds. The second part (Section 6.3) applies all of it to a complete design, a traffic light.

6.2.1 What is a finite state machine?

A **finite state machine** (FSM) is a model of a system that, at any moment, is in one of a finite number of internal **states**. The state summarizes everything about the past that matters for the future: the machine does not remember *how* it got into a state, only *which* state it is in. At each clock edge, the machine moves to a **next state**, which depends on the current state and on the external **inputs**, and it produces **outputs** that depend on the state and, in some machines, on the inputs as well.

Formally, an FSM is a tuple $(S, I, O, \delta, \lambda, s_0)$:

- S is the finite set of states, and $s_0 \in S$ the initial state, the one the machine enters on reset;
- I and O are the sets of input and output values;
- $\delta : S \times I \rightarrow S$ is the **next-state function**, which gives the next state for each combination of current state and input;
- λ is the **output function**.

A counter is the simplest example: its states are the values 0 to 15, its next-state function is "add one", and it has no inputs apart from the reset. FSMs become interesting when the inputs steer them.

In a digital system, FSMs are the **controllers**. A larger design is usually split into a **datapath**, the registers, adders and multiplexers that process the data, and a **control path**, which examines the external commands and the status of the datapath and activates the right control signals at the right time. A UART transmitter, a bus interface waiting for a response, a memory controller sequencing a read: in each of them, the part that decides *what happens next* is an FSM.

6.2.2 Describing an FSM: the state diagram

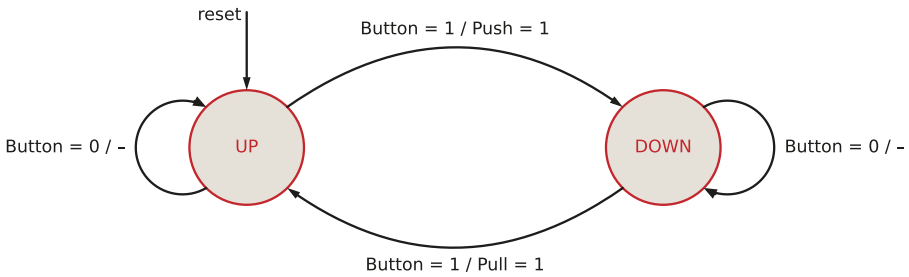


Figure 6.1: State diagram of the window-blinds controller, a Mealy machine: each arrow is labelled with its condition and, after the slash, the outputs that are 1 on that transition.

The most common way to describe an FSM is a **state diagram**: a directed graph in which every node is a state and every arrow a transition.

Take a controller for window blinds. A single push button controls a servo motor: if the blinds are up, a press pushes them down; if they are down, a press pulls them up. The FSM has one input, `Button`, and two outputs, `Push` and `Pull`, which drive the motor. It must remember where the blinds are, so it needs two states, `UP` and `DOWN` (Figure 6.1).

Each arrow is labelled with a **condition** and, after a slash, the **outputs** that are 1 when the machine takes that arrow. In `UP`, if `Button` is 1 the machine raises `Push` and moves to `DOWN`. If `Button` is 0, it takes the arrow back to `UP`, and no output is active (the – after the slash). `DOWN` is the mirror image, with `Pull`. The arrow marked `reset` shows the initial state.

Designing an FSM always follows the same three steps, and the state diagram is where they are written down:

1. **Define the states.** What must the machine remember? Here, whether the blinds are up or down.
2. **Define the transitions.** For each state and each input combination, which state comes next? Every state must have a way out for *every* input combination, even if that way is “stay here”.
3. **Define the outputs.** Which outputs are active, in which state, and under which input?

The same information can be written as a **state table**, one row per state and input combination:

State	`Button`	Next state	`Push`	`Pull`
`UP`	0	`UP`	0	0
`UP`	1	`DOWN`	1	0
`DOWN`	0	`DOWN`	0	0
`DOWN`	1	`UP`	0	1

The table is the next-state function δ and the output function λ written out in full. It is the most precise description, but it grows with every state and every input bit, so for anything but tiny machines we draw the diagram instead.

6.2.3 The structure of an FSM in hardware

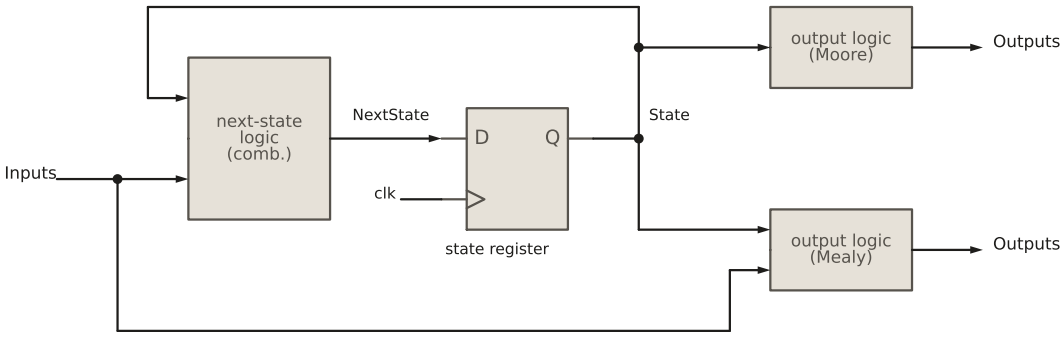


Figure 6.2: The general structure of an FSM: the next-state logic, the state register, and the output logic of a Moore or a Mealy machine.

In hardware, every FSM has the same three parts (Figure 6.2):

- The **state register** stores the current state. It is a set of flip-flops, all clocked by the system clock, and it is the only memory in the machine. On reset it loads the initial state.

- The **next-state logic** is a combinational circuit that implements δ: from the current state and the inputs, it computes `NextState`, which the register loads at the next clock edge.
- The **output logic** is a combinational circuit that implements λ and produces the outputs.

The loop from the register output, through the next-state logic, and back into the register is the same loop as in the counter of the previous chapter, written in its *way 2*: the register only stores, and all the decisions are in the combinational logic in front of it. An FSM is a counter whose "+ 1" has been replaced by an arbitrary function of the state and the inputs.

6.2.4 Moore and Mealy machines

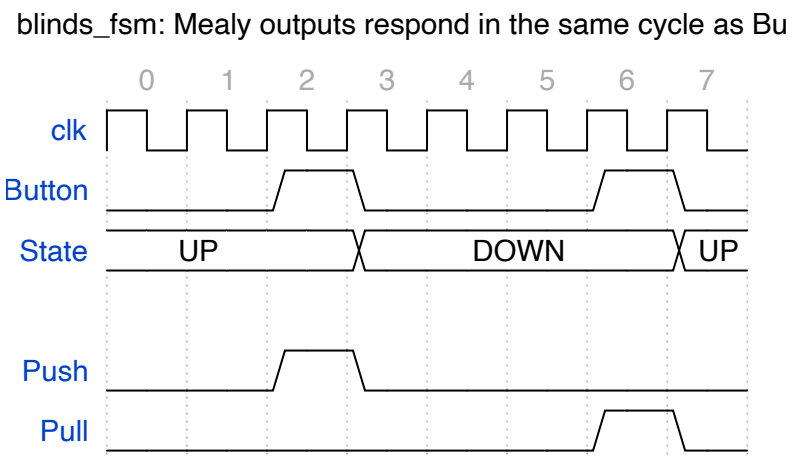


Figure 6.3: The blinds controller in simulation. `Button` comes from a synchronous edge detector and is high for one cycle; `Push` and `Pull` respond in the same cycle, `State` at the next edge.

The output logic in Figure 6.2 comes in two forms, and they give the two classic kinds of FSM:

- In a **Moore machine**, the outputs depend **only on the current state**: $\lambda : S \to O$. The outputs are written inside the states of the diagram, and they change only at a clock edge, when the state changes. A traffic light is a Moore machine: the lamps are a function of the state.
- In a **Mealy machine**, the outputs depend on the **current state and the inputs**: $\lambda : S \times I \to O$. The outputs are written on the arrows, as in Figure 6.1. The blinds controller is a Mealy machine: `Push` is 1 only in `UP` *and* only while `Button` is 1.

Figure 6.3 shows what this means in time. In column 2 `Button` is 1, and `Push` is 1 in that same cycle, before the state has changed; the state becomes `DOWN` only at the edge that starts column 3. The same happens with `Pull` in column 6.

The two kinds trade speed against safety:

	Moore	Mealy
Outputs depend on	the state only	the state and the inputs
In the diagram, outputs are written	in the states	on the transitions
Reaction to an input	one clock cycle later	in the same cycle
Combinational path from input to output	no	yes
Number of states	often more	often fewer

A Mealy machine reacts one cycle earlier and often needs fewer states. To get a one-cycle `Push` pulse from a Moore machine, the blinds controller would need extra states such as `PUSHING` and `PULLING`, in which the output is active. The price of a Mealy machine is the combinational path from input to output. Any glitch on the input appears on the output, and the path adds to the timing of whatever the output drives. If two Mealy machines feed each other, the paths can even close into a combinational loop. In this book we prefer Moore outputs, and use Mealy outputs where the one-cycle reaction really matters, for example for a one-cycle `Done` pulse.

6.2.5 State encoding

The states have names in the diagram, but the state register stores bits, so every state needs a binary code. The choice of codes is the **state encoding**, and there are three common ones. For N states:

- **Binary** encoding numbers the states 0, 1, 2, … and needs $\lceil \log_2 N \rceil$ flip-flops. It is the most compact.
- **Gray** encoding uses the same number of flip-flops, but assigns codes so that states that follow each other differ in a single bit. That reduces switching activity, and so power, when the machine mostly moves along a fixed sequence.
- **One-hot** encoding uses one flip-flop per state, and exactly one of them is 1 at a time. It needs N flip-flops, but the logic becomes very simple: “are we in `GREEN`?” is a single bit, not a comparison.

For the four states of the traffic light in the second part of this chapter:

State	Binary	Gray	One-hot
`RED`	`00`	`00`	`0001`
`RED_YELLOW`	`01`	`01`	`0010`
`GREEN`	`10`	`11`	`0100`
`YELLOW`	`11`	`10`	`1000`

The traffic light cycles `RED`, `RED_YELLOW`, `GREEN`, `YELLOW`, `RED`, so with the Gray codes every transition flips exactly one bit, including the one back to the start.

On an FPGA, flip-flops are plentiful (every slice has eight of them, as we saw in the FPGA chapter), while logic depth decides the clock speed. One-hot encoding trades a few flip-flops for shallower next-state and output logic, and is often the fastest choice. In practice, the synthesis tool recognizes FSMs in the code and usually picks the encoding itself: Vivado re-encodes a machine it extracts, and can be overruled with the `fsm_encoding` attribute. What we write in the code is the *meaning* of the states, and the encoding is an implementation detail.

6.2.6 The ASM chart

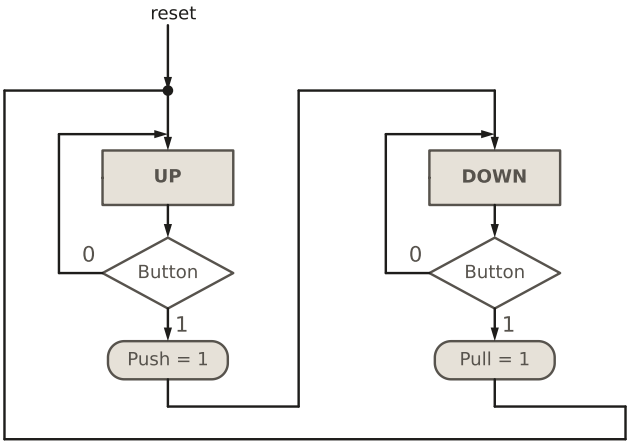


Figure 6.4: ASM chart of the blinds controller. The Mealy outputs `Push` and `Pull` appear in conditional output boxes, on the path where `Button` is 1.

A state diagram gets crowded quickly: when there are several inputs, and when the machine also has to update counters or registers, the labels on the arrows become long expressions.

An **algorithmic state machine** (ASM) chart shows the same information as a flowchart, which is easier to follow step by step. It uses three kinds of boxes (Figure 6.4):

- A rectangular **state box** stands for one state. It holds the state name and the **Moore outputs** that are 1 in that state. Outputs that are not listed are 0.
- A diamond is a **decision box**. It tests an input or an internal signal, and has a `0` and a `1` exit.
- A rounded **conditional output box** holds the **Mealy outputs** that are 1 on that path, and the register updates (such as `Count++`) that happen on that path.

A state box with all the decision and conditional boxes below it, up to the arrows that enter the next state boxes, forms an **ASM block**, and one ASM block is one clock cycle. During the cycle, the machine is in the state of the state box, the decisions are evaluated by the next-state logic, and the outputs of the conditional boxes on the active path are 1. At the clock edge the machine follows the active path into the next state.

Compare Figure 6.4 with Figure 6.1: the self-loops of the state diagram are the `0` exits of the decisions, and the outputs that were written after the slash are now in the conditional boxes. The ASM chart has another advantage: it maps line by line onto the SystemVerilog code, as the next section shows.

6.2.7 Describing an FSM in SystemVerilog

The three hardware parts of Figure 6.2 become three pieces of code. Here is the blinds controller, written according to the HIPV coding conventions:

```
////////////////////////////////////////////////////////////////////
// blinds_fsm
//
// Purpose: Window-blinds controller. A press of Button pushes the
//          blinds down when they are up, and pulls them up when
//          they are down.
////////////////////////////////////////////////////////////////////

module blinds_fsm(
  input  logic clk, reset,
  input  logic Button,
  output logic Pull, Push);

  typedef enum logic {UP, DOWN} state_t;

  state_t State, NextState;
```

```
  // state register
  always_ff @(posedge clk)
    if (reset) State <= UP;
    else       State <= NextState;

  // next-state logic
  always_comb begin
    NextState = State;                    // default: stay
    case (State)
      UP:      if (Button) NextState = DOWN;
      DOWN:    if (Button) NextState = UP;
      default: NextState = UP;
    endcase
  end

  // output logic (Mealy: depends on the state and the input)
  assign Push = (State == UP) && Button;
  assign Pull = (State == DOWN) && Button;

endmodule
```

The states are an enumerated type. `typedef enum logic {UP, DOWN} state_t;` defines a new type, `state_t`, whose values have names. The simulator shows those names in the waveform instead of bits, and the compiler refuses to assign an unrelated value to a `state_t` signal without a cast. Without explicit values the encoding is binary, in the order listed; to choose another encoding, give the values explicitly:

```
typedef enum logic [3:0] {RED        = 4'b0001,     // one-hot
                          RED_YELLOW = 4'b0010,
                          GREEN      = 4'b0100,
                          YELLOW     = 4'b1000} state_t;
```

Following the HIPV naming convention, the state names are constants and are written in MACRO_CASE, the type name ends in `_t`, and the signals `State` and `NextState` are in UpperCamelCase.

The state register is an `always_ff` with a synchronous reset, exactly the register of the previous chapter. Reset loads the initial state.

The next-state logic is an `always_comb` with a `case` on the current state, one branch per state. It begins by assigning the default `NextState = State`, “stay where you are”. This

is the *default first, then override* pattern: every `if` without an `else` falls back on the default, which is exactly the `0` exit of a decision in the ASM chart, and `NextState` is assigned on every path, so no latch can appear. The `default` branch of the `case` catches state codes that do not belong to any state. With two states in one bit there are none, but in a one-hot machine most of the codes are illegal, and the `default` sends the machine back to a known state instead of leaving it stuck.

The output logic is two `assign` statements. Because they read `Button`, this is a Mealy machine. In a Moore machine, the output logic reads only `State`.

One practical remark. `Button` must be a one-cycle pulse that is **synchronous** to `clk`. A real push button is neither: it bounces for milliseconds, and it is held for millions of clock cycles, during which this FSM would toggle at every edge. On a board, the button first passes through a synchronizer, a debouncer and an edge detector, and only their one-cycle output reaches the FSM. We return to synchronizers in a later chapter.

6.2.8 FSMs with data registers

The blinds controller remembers one bit. Real controllers also have to remember numbers: how many bits of a UART frame have been sent, or how many milliseconds a traffic light has been green. In principle we could make every count a separate state (`GREEN_0`, `GREEN_1`, …, `GREEN_3999`), but that is absurd. Instead, we keep the counter in a register next to the state register. The **state** is then the named control state *plus* the contents of these data registers, and the machine is called an **FSM with datapath** (FSMD).

The coding pattern stays the same, and extends to every such register. Each register `X` gets a companion signal `XNext`:

- `X` is the **current** value, the output of the register;
- `XNext` is the **next** value, computed by the next-state logic, and loaded into `X` at the next clock edge.

The `always_ff` block loads all of them together (`State <= NextState; Count <= CountNext;`), and the `always_comb` block computes all the `...Next` signals, each with its default first: `CountNext = Count` means “keep the value”, and a branch overrides it where the count must change. In an ASM chart, the register updates are the contents of the conditional boxes, written as `Count++` or `Count = 0`. All updates in one ASM block happen at the same clock edge, together with the change of state. The second part of this chapter uses this pattern for the millisecond counter of the traffic light.

6.2.9 Common mistakes

Most FSM bugs fall into a few groups:

- **Latches.** If some path through the next-state logic does not assign `NextState` (or a `...Next` register value), the tool infers a latch. Start the `always_comb` block with the defaults.
- **Trap states.** A state code that no branch handles can keep the machine stuck forever after a glitch or a bad reset. Give every `case` a `default` branch that returns to a known state.
- **Derived clocks.** Slowing an FSM down by clocking it with a counter bit creates a second clock domain and breaks timing analysis. Keep the system clock everywhere, and let the counter produce a one-cycle **enable** (tick) that the FSM checks, as the traffic light does below.
- **Asynchronous inputs.** An input that can change at any moment, such as a button or a serial line, can violate the setup time of the state register and leave the FSM in an illegal state. Pass it through a synchronizer first.
- **Mealy chains.** A Mealy output that feeds the input of another Mealy machine, whose output comes back, forms a combinational loop. Register one of the outputs, or use Moore outputs.

6.3 Practical case: a traffic light

We now apply the theory to a complete, if small, system: the controller of a traffic light. It is a Moore machine with a data register, it needs time intervals that are long compared with the clock period, and it is built from two modules, a counter that marks time and an FSM that uses it.

6.3.1 Specification

The traffic light cycles through four states, and each state lights a different combination of lamps for a different time:

State	Lights	Time
`RED`	red	5 s
`RED_YELLOW`	red and yellow	1 s
`GREEN`	green	4 s
`YELLOW`	yellow	2 s

The system clock is 100 MHz, and a synchronous reset puts the light into `RED`. The outputs `Red`, `Yellow` and `Green` drive the three lamps, and depend only on the state, so the controller is a Moore machine.

6.3.2 System architecture

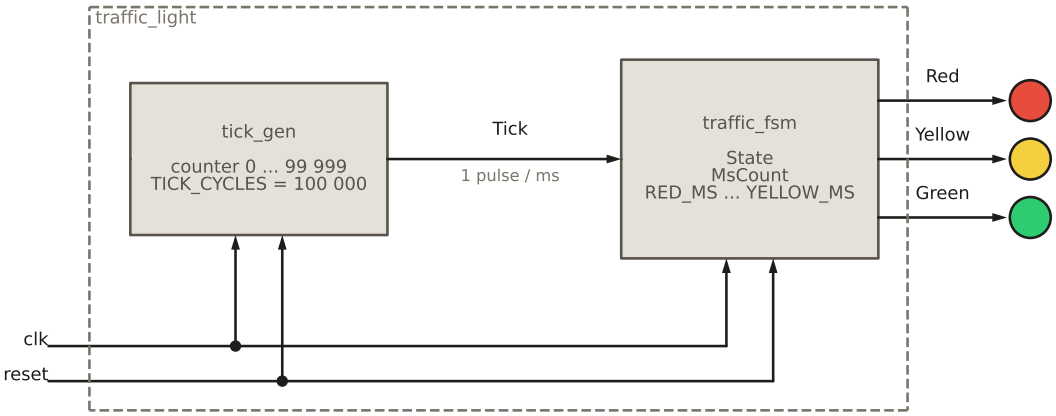


Figure 6.5: The traffic-light system: the tick generator turns the 100 MHz clock into a one-cycle `Tick` every millisecond, and the FSM counts those ticks and drives the lamps. Both run on the same clock.

The times are long compared with the clock period. At 100 MHz, 5 s is 500 million cycles, and counting them directly would need a 29-bit counter compared against a large constant in every state. We split the job in two, the same way a UART uses a baud-rate generator (Figure 6.5):

- `tick_gen` is a counter that produces a one-cycle pulse, `Tick`, once every millisecond.
- `traffic_fsm` is the FSM. It counts those pulses in a data register, `MsCount`, and when the count reaches the time of the current state, it moves on to the next state.
- `traffic_light` is the top level, which only connects the two.

The FSM still runs on the 100 MHz clock: `Tick` is an *enable*, not a slower clock. Every flip-flop in the design sees the same clock edge, which is the synchronous-design rule of the HIPV coding standard, and the time unit of the FSM is now a millisecond, so the times in the specification appear in the code unchanged.

6.3.3 The 1 ms tick generator

The tick generator is a counter with a comparator. It counts from 0 to `TICK_CYCLES - 1`, raises `Tick` for the one cycle in which it holds that last value, and starts again from 0:

```
////////////////////////////////////////////////////////////////////////
// tick_gen
//
// Purpose: One-cycle Tick pulse every TICK_CYCLES clock cycles
//          (1 ms with the default value and a 100 MHz clock).
////////////////////////////////////////////////////////////////////////

module tick_gen #(parameter TICK_CYCLES = 100_000) (
  input  logic clk, reset,
  output logic Tick);

  localparam CNT_WIDTH = $clog2(TICK_CYCLES);

  logic [CNT_WIDTH-1:0] Count;

  always_ff @(posedge clk)
    if (reset || Tick) Count <= '0;
    else               Count <= Count + 1;

  assign Tick = (Count == TICK_CYCLES - 1);

endmodule
```

The period of `Tick` is exactly `TICK_CYCLES` clock cycles, because the counter spends one cycle at each of the values 0 to `TICK_CYCLES - 1`. With the default of 100 000 and a 100 MHz clock, that is 1 ms. The counter width is derived from the parameter with `$clog2`, which the tool evaluates at compile time, and it is a `localparam` because nobody outside should change it. `Tick` itself is a Moore-style output of the counter: it depends only on the register `Count`, so it is clean for a whole clock cycle.

6.3.4 The ASM chart of the traffic light

The ASM chart (Figure 6.6) has one column per state. The state boxes hold the Moore outputs, the lamps that are on. There are no Mealy outputs, so the conditional boxes contain only the updates of the data register `MsCount`.

Read the `RED` column. In this state the red lamp is on. If `Tick` is 0, we take the `0` exit back to the entry of `RED`: nothing changes. If `Tick` is 1, a millisecond has passed, and the second decision checks whether it was the last one. If not, `MsCount` is incremented and we stay in `RED`. If it was, `MsCount` is cleared and the FSM moves to `RED_YELLOW`. The other three

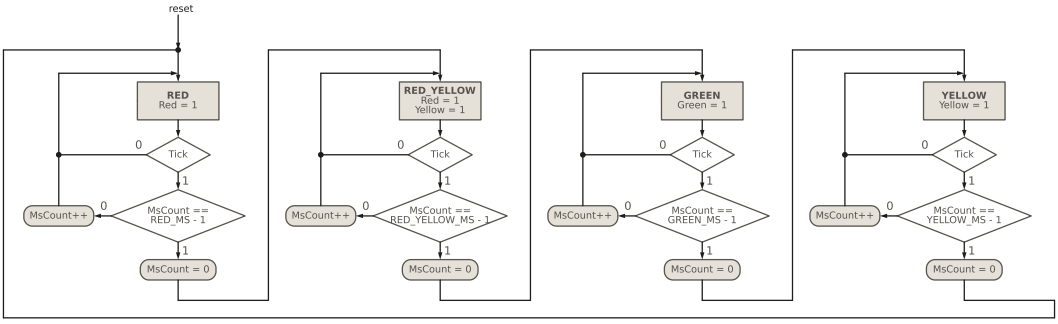


Figure 6.6: ASM chart of the traffic-light FSM. Each column is one state: the state box with its Moore outputs, the decisions on `Tick` and `MsCount`, and the conditional updates of `MsCount`.

columns are the same, each with its own time and its own successor, and `YELLOW` returns to `RED`.

The `- 1` in the decisions comes from counting from 0. After `RED_MS - 1` ticks, the count holds `RED_MS - 1`, and the tick that arrives while the count holds that value is the `RED_MS`-th one. The FSM leaves the state on exactly that tick.

6.3.5 The FSM in SystemVerilog

The code follows the chart box by box, and uses the FSMD pattern for `MsCount`:

```
////////////////////////////////////////////////////////////////////////
// traffic_fsm
//
// Purpose: Traffic-light controller. Cycles through RED, RED_YELLOW,
//          GREEN and YELLOW, and holds each state for its own number
//          of 1 ms ticks.
////////////////////////////////////////////////////////////////////////

module traffic_fsm #(parameter RED_MS = 5000, RED_YELLOW_MS = 1000,
                               GREEN_MS = 4000, YELLOW_MS = 2000) (
  input  logic clk, reset,
  input  logic Tick,
  output logic Red, Yellow, Green);

  localparam CNT_WIDTH = 16;               // enough for 65 s per state
```

```
typedef enum logic [1:0] {RED, RED_YELLOW, GREEN, YELLOW} state_t;

state_t               State, NextState;
logic [CNT_WIDTH-1:0] MsCount, MsCountNext;

// state register
always_ff @(posedge clk)
  if (reset) begin
    State   <= RED;
    MsCount <= '0;
  end else begin
    State   <= NextState;
    MsCount <= MsCountNext;
  end

// next-state logic
always_comb begin
  NextState   = State;                  // default: stay in the state
  MsCountNext = MsCount;                //          and keep the count
  if (Tick) MsCountNext = MsCount + 1;  // count the elapsed milliseconds
  case (State)
    RED:
      if (Tick && MsCount == RED_MS - 1) begin
        NextState   = RED_YELLOW;
        MsCountNext = '0;
      end
    RED_YELLOW:
      if (Tick && MsCount == RED_YELLOW_MS - 1) begin
        NextState   = GREEN;
        MsCountNext = '0;
      end
    GREEN:
      if (Tick && MsCount == GREEN_MS - 1) begin
        NextState   = YELLOW;
        MsCountNext = '0;
      end
    YELLOW:
      if (Tick && MsCount == YELLOW_MS - 1) begin
        NextState   = RED;
        MsCountNext = '0;
      end
```

```
      default: NextState = RED;
    endcase
  end

  // output logic: the lights depend on the state only
  assign Red    = (State == RED) || (State == RED_YELLOW);
  assign Yellow = (State == RED_YELLOW) || (State == YELLOW);
  assign Green  = (State == GREEN);

endmodule
```

The state register stores the control state and the data register `MsCount`, and loads both together. On reset the FSM goes to `RED` with the count at 0, which is the `reset` arrow of the chart.

The next-state logic starts with the defaults: stay in the current state and keep the count. They cover the `Tick = 0` exits of every column without a single `else`. The next line is the `MsCount++` box, which is the same in every state, so we write it once: on every tick, count. The `case` then handles the last tick of each state. It overrides both defaults: it selects the next state and clears the counter. Because this assignment comes later in the block, it wins over the increment, as the blocking `=` guarantees. The `default` branch is not reachable with four states in two bits, but it keeps the machine safe if the encoding changes.

The output logic is three `assign` statements, one per lamp, each a function of `State` alone: this is a Moore machine, so the lamps change only at a clock edge, when the state changes. The light times are parameters, so another junction, with other times, is a different instance of the same module.

6.3.6 The traffic light in the waveform

With the real parameters, one traffic-light cycle lasts 12 s, or 1.2 billion clock cycles, far too many to simulate or to draw. Because every time is a parameter, the testbench can instantiate both modules with small values instead: `TICK_CYCLES = 2`, and states of 4, 1, 3 and 2 ticks (Figure 6.7). The design is the same; only the numbers are smaller.

`reset` is high for the first edge, so in column 0 the FSM is in `RED` and `MsCount` is 0. `Tick` is high in every second column. At each edge that follows a tick, `MsCount` goes up by one: it is 1 in columns 2 and 3, 2 in columns 4 and 5, and 3 in columns 6 and 7. In column 7 `Tick` is high and `MsCount` is 3, which is `RED_MS - 1`, so the edge starting column 8 moves the FSM to `RED_YELLOW` and clears the count. `RED` lasted 8 cycles, exactly 4 ticks. `RED_YELLOW` lasts only one tick: already in column 9, `Tick` is high while `MsCount` is 0, which is `RED_YELLOW_MS - 1`, so at column 10 the FSM moves to `GREEN`. `MsCount` stays 0

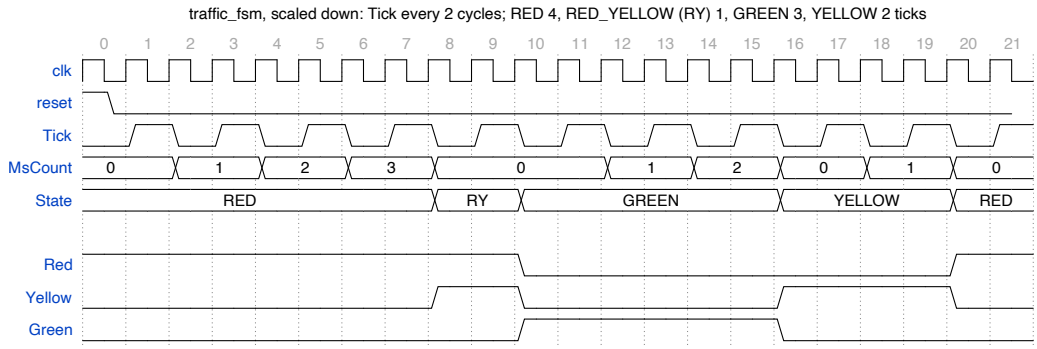


Figure 6.7: `traffic_fsm` in simulation with the times scaled down: one tick every 2 clock cycles, and states of 4, 1, 3 and 2 ticks. `RY` is `RED_YELLOW`.

across that change, because it is cleared, not incremented. `GREEN` counts 0, 1, 2 and `YELLOW` 0, 1, and at column 20 the FSM is back in `RED`.

The bottom three rows are the lamps. They change only at the edges where `State` changes, and the red lamp stays on through `RED_YELLOW`, as the output logic says.

6.3.7 Putting it together

The top level is purely structural, as the HIPV convention asks for a module that only connects blocks. It is Figure 6.5 in code: one instance of each module, wired by name.

```
////////////////////////////////////////////////////////////////////////
// traffic_light
//
// Purpose: Traffic light: a 1 ms tick generator driving the
//          traffic-light FSM.
////////////////////////////////////////////////////////////////////////

module traffic_light(
  input  logic clk, reset,
  output logic Red, Yellow, Green);

  logic Tick;

  tick_gen #(.TICK_CYCLES(100_000)) msgen (
    .clk(clk), .reset(reset), .Tick(Tick));
```

```
  traffic_fsm lightfsm (
    .clk(clk), .reset(reset), .Tick(Tick),
    .Red(Red), .Yellow(Yellow), .Green(Green));

endmodule
```

The split is what makes the design easy to change. A board with a 50 MHz clock needs a different `TICK_CYCLES`, and a different junction needs different light times; each is one parameter in one instance, and neither touches the logic of the FSM. The same pattern, a counter that produces a periodic tick and an FSM that uses the tick as an enable, appears in every serial interface: in a UART, the tick marks the duration of one bit, and the FSM steps through the start bit, the data bits and the stop bit.

7 Five-Stage Pipeline — Structure

Five-stage pipeline

7.1 Goal for this week

- Place the HIPV SoC on the levels of abstraction, from the system down to the silicon, and know at which level we design
- Translate the well-known pipeline theory (IF/ID/EX/MEM/WB) into SystemVerilog
- Apply SystemVerilog syntax immediately on a real, motivating example
- Treat memory (for now) as an interface — the implementation follows next week

We implement pipeline registers as separate modules for two reasons. First, pedagogically: each such register is a direct, visible example of a D flip-flop (DFF) in the context of a real processor, which was an explicit goal of the second part of the course. Second, practically: separate modules with a fixed interface (see `pipeline_pkg.sv`) allow us to substitute an individual stage with a reference ("golden") implementation in the jigsaw testing methodology — discussed in more detail in the verification chapter. This five-stage IF/ID/EX/MEM/WB structure follows the classic pipelined datapath described by Patterson and Hennessy (2017).

7.2 Levels of abstraction

Before we build the pipeline, it helps to step back and look at what we are building from a distance. The HIPV SoC is a single system, but it can be described at very different levels of detail: as a handful of blocks connected by buses, as an algorithm that executes instructions, as a set of registers and the logic between them, as a netlist of gates, as a circuit of transistors, or as structures in doped silicon. These are the **levels of abstraction** of digital design (Figure 7.1).

Each level describes the same system, but with fewer details than the level below it. A description at a higher level is shorter and easier to reason about, because it *hides* how its parts are built: at the system level the core is a box, and we do not care whether its adder is a ripple-carry chain or a carry-lookahead tree. A description at a lower level is closer to the real hardware, and much larger. Moving down a level means adding implementation decisions; moving up means leaving them out.

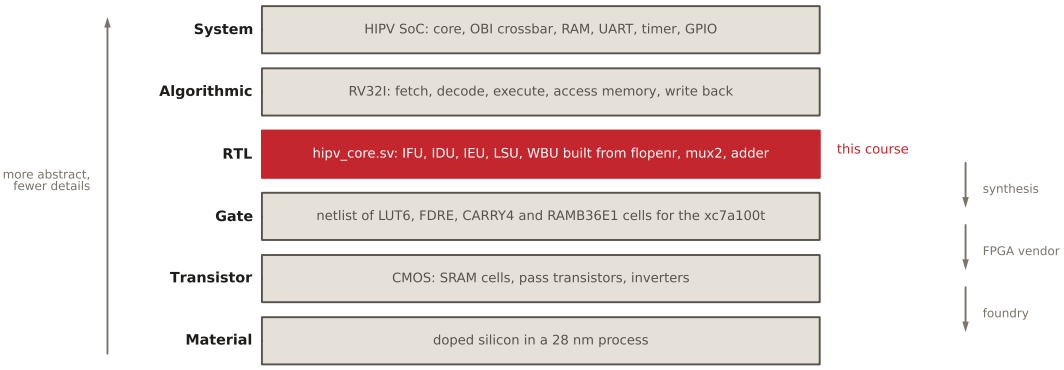


Figure 7.1: The HIPV SoC at six levels of abstraction. We design at the register-transfer level; the tools, the FPGA vendor and the foundry take care of the levels below.

The earlier chapters of this book have already visited most of these levels, bottom-up: silicon and the p-n junction, CMOS transistors and gates, programmable devices and the FPGA, and finally SystemVerilog at the register-transfer level. In this chapter we start working top-down, the way a design is actually made, and we use HIPV as the example at every level.

7.3 System level

At the **system level**, a design is a set of blocks and the connections between them. The HIPV SoC (`rtl/top/hipv_soc.sv`) consists of:

- the RISC-V core, `hipv_core`, with two OBI master ports, one for instructions and one for data;
- an OBI crossbar that connects the two masters to five slaves;
- a 64 KiB RAM that holds both the program and the data;
- three peripherals: a UART, a timer and a GPIO block.

Nothing at this level says *how* any block works. The decisions made here are about the system as a whole: which blocks there are, what bus connects them and with which protocol, where each block sits in the address map, and how fast the clock runs. For HIPV, the bus is OBI, the address map gives every slave its own window, and the clock is 50 MHz on the Nexys A7-100T board.

7.4 Algorithmic level

At the **algorithmic level**, a block is described by *what* it computes, as an algorithm, without any notion of a clock or of the hardware that will carry it out. For the HIPV core, that algorithm is the RV32I instruction set: a loop that fetches the instruction at the program counter, decodes it, and executes it:

```
PC = 0
forever:
  instr = mem[PC]                                  # fetch
  op, rd, rs1, rs2, imm = decode(instr)            # decode
  a = x[rs1];  b = x[rs2]
  case op:                                         # execute, memory, write back
    ALU:    x[rd] = alu(op, a, b or imm);  PC = PC + 4
    LOAD:   x[rd] = mem[a + imm];          PC = PC + 4
    STORE:  mem[a + imm] = b;              PC = PC + 4
    BRANCH: PC = taken(op, a, b) ? PC + imm : PC + 4
    JAL:    x[rd] = PC + 4;                PC = PC + imm
    JALR:   x[rd] = PC + 4;                PC = (a + imm) & ~1
```

Here `x` is the array of 32 registers (with `x[0]` always 0) and `mem` is the memory. Every instruction is finished before the next one starts, and how long each step takes is not part of the description. This is the level of the RISC-V specification, and of an instruction-set simulator such as Spike, which runs programs exactly according to this loop. It is the reference against which every implementation is judged: whatever the pipeline does internally, the registers and the memory must end up as the loop says.

7.5 Register-transfer level

At the **register-transfer level** (RTL), the algorithm is mapped onto clocked hardware: a set of registers, and the combinational logic that computes their next values at every clock edge. This is where time appears. The loop of the algorithmic level is cut into five steps, fetch, decode, execute, memory and write back, and each step becomes a pipeline stage with its own registers, so that five instructions can be in progress at once (Figure 7.2). The price is that instructions now overlap, and the RTL must handle what the algorithm never had to think about: an instruction that needs a result its predecessor has not yet written (forwarding, in the hazard unit), and instructions fetched after a jump that must be thrown away (the redirect from Execute).

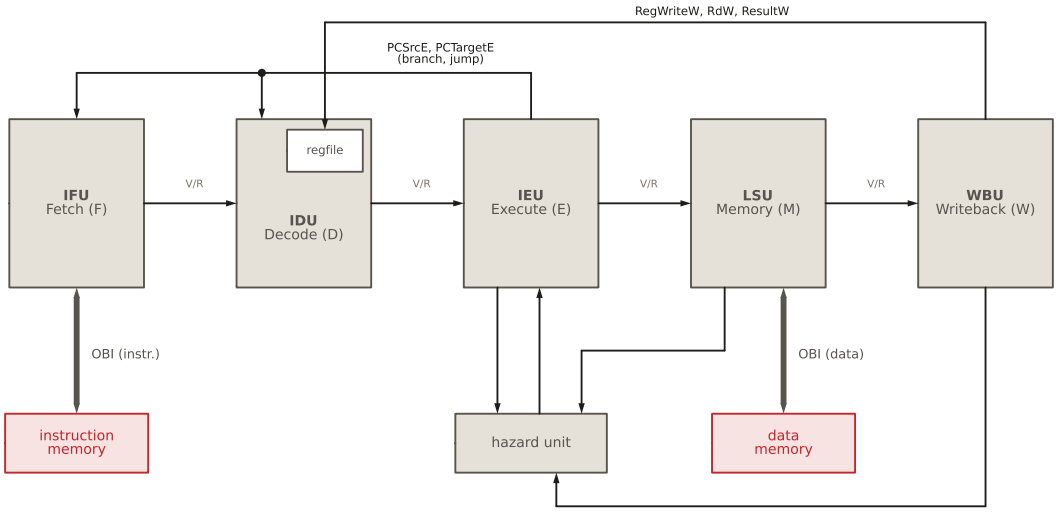


Figure 7.2: The HIPV core at the register-transfer level: five pipeline stages with their handshakes, the redirect and write-back feedback, the two OBI ports and the hazard unit.

This is the level of `hipv_core.sv` and of everything we have written in SystemVerilog so far: the leaf cells (`flopenr`, `mux2`, `mux3`, `adder`) in behavioral style, the stages assembled from them structurally, and the control FSMs. It is the lowest level at which we still make the design decisions ourselves, and the highest level that tools can turn into hardware automatically. That is why almost all digital design is done here, and why the rest of this course works at this level.

7.6 Gate level

At the **gate level**, the design is a **netlist**: a list of cells and of the wires that connect them. It is produced from the RTL by **synthesis**, and nobody writes it by hand. What the cells are depends on the target. For an ASIC, they are the gates and flip-flops of a standard-cell library. For HIPV on the Xilinx xc7a100t FPGA, they are the primitives of the 7-series fabric that we met in the FPGA chapter:

- every register bit of `flopenr` becomes an `FDRE`, a flip-flop with a clock enable and a synchronous reset, which is exactly what `flopenr` describes;
- the multiplexers, decoders and the ALU logic become `LUT6` look-up tables;
- the adders use the `CARRY4` fast-carry chains of the slices;
- the 64 KiB RAM becomes 16 `RAMB36E1` block RAMs.

The schematics drawn by Yosys in the chapter on designing with SystemVerilog are small examples of this level. The netlist is also the level at which timing is checked: only when we know which cells and wires a path goes through can the tools tell whether it fits in the 20 ns clock period of 50 MHz.

7.7 Transistor level

At the **transistor level**, each cell is a circuit of NMOS and PMOS transistors, the CMOS circuits from the first chapters. On an FPGA, the transistor-level design is fixed and was done once, by the vendor, for the whole family. A `LUT6` is 64 SRAM cells that hold its truth table, followed by a tree of pass-transistor multiplexers selected by the six inputs. Every programmable routing switch is a pass transistor controlled by another SRAM cell. Flip-flops and the carry chain are hard-wired CMOS circuits. Our HIPV design does not change a single transistor: it only sets the configuration cells that decide what the existing transistors do.

For an ASIC, this level is where the standard cells come from: each cell of the library is designed and characterized once at the transistor level, and synthesis then uses it as a black box with known delay and power.

7.8 Material level

At the bottom, at the **material level**, a transistor is a physical structure: regions of n-type and p-type silicon created by doping, a thin oxide, a polysilicon or metal gate, and layers of metal wiring on top, produced by photolithography. The behavior of the p-n junction and of the MOSFET channel, described in the first chapter, is determined here. The Artix-7 xc7a100t on our board is manufactured in a 28 nm process, and the layout and process decisions at this level belong to the foundry and to the FPGA vendor.

7.9 Moving between the levels

A design is made **top-down**. We start at the system level (which blocks, which bus), specify the behavior at the algorithmic level (the RV32I instruction set), and write the RTL by hand. From there on, the tools take over: synthesis produces the gate-level netlist, and place and route maps it onto the transistors and wires of the FPGA, which the vendor and the foundry designed at the two lowest levels.

Verification connects the levels in the opposite direction. We simulate the RTL and compare its results with what the higher level says must happen: the HIPV core testbenches run small programs and check that the data memory ends up exactly as the algorithmic loop above predicts. Each level is thus both the specification for the level below and the reference for checking it.

8 Instruction fetch unit

Instruction fetch unit

8.1 Goal for this week

- Know what the instruction fetch unit (IFU) does, and which other parts of the core it talks to
- Read the IFU interface: the handshake to Decode, the redirect from Execute, and the OBI port to instruction memory
- Build the IFU datapath from generic blocks: `flopenr`, `mux2` and `adder`
- Design the IFU controller as an FSM: states, transitions, the discard flag and the outputs

8.2 Introduction

The instruction fetch unit is the first stage of the HIPV pipeline (see the overview in the previous chapter). Before the core can execute an instruction, it must fetch it from memory, and which instruction it fetches is determined by the **program counter** (PC), the register that holds the address of the current instruction. RISC-V instructions are 32 bits wide, four bytes, so without jumps the program counter advances by four after every instruction.

To do this job, the IFU needs five things:

- two **registers**, one that holds the PC and one that holds the fetched instruction;
- an **adder** that computes the address of the next instruction, `PC + 4`;
- a **multiplexer** that chooses the next PC: the sequential `PC + 4`, or the target of a jump or taken branch;
- a **port to the instruction memory**, through which it sends the address and receives the instruction;
- a **controller** that decides when each register is loaded, when a request goes to memory, and when the fetched instruction is ready for the next stage.

The first four form the datapath, and they are built from the generic blocks of the chapter on designing with SystemVerilog. The controller is a finite state machine, and it is where most of the thinking in this chapter goes.

8.3 Interface of the IFU

The ports of the IFU fall into four groups: the handshake with the Decode stage, the redirect from the Execute stage, the stage outputs, and the port to the instruction memory. The suffix of every name says which stage the signal belongs to: `F` for Fetch, `D` for Decode, `E` for Execute.

```
module ifu (input  logic        clk, reset,
            // pipeline handshake and redirect
            input  logic        ReadyD,
            output logic        ValidF,
            input  logic        PCSrcE,
            input  logic [31:0] PCTargetE,
            // stage outputs to Decode
            output logic [31:0] InstrF,
            output logic [31:0] PCF, PCPlus4F,
            // OBI master port to instruction memory
            output logic        ObiReqF,
            input  logic        ObiGntF,
            output logic [31:0] ObiAddrF,
            input  logic        ObiRvalidF,
            input  logic [31:0] ObiRdataF);
```

8.3.1 Handshake with Decode

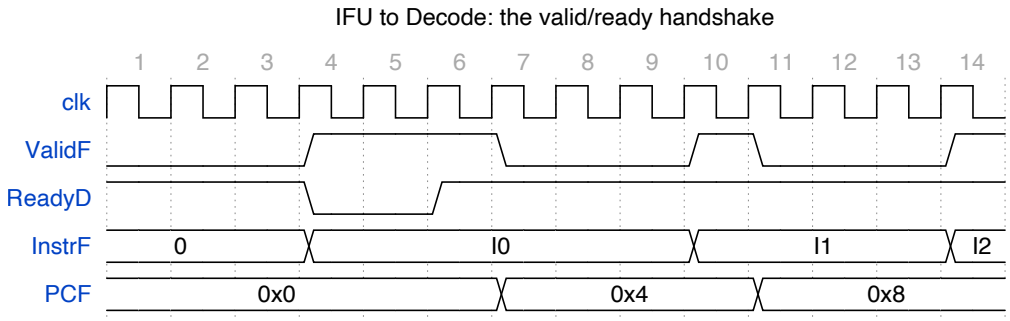


Figure 8.1: The handshake between the IFU and Decode. `InstrF` holds `I0` from cycle 4 to 9, but it is a valid instruction only while `ValidF` is high. Decode is busy in cycles 4 and 5 (`ReadyD = 0`), so `I0` waits; it is transferred at the end of cycle 6, where both signals are high.

The IFU and Decode work at their own pace, so they agree on each transfer with two signals:

- `ValidF` (output) says that the output of the IFU is valid: the bits on `InstrF` are a fetched instruction, not leftovers from an earlier one or random bits after reset.
- `ReadyD` (input) says that Decode is ready to accept a new instruction.

An instruction moves from Fetch to Decode at a clock edge where both signals are high. While `ValidF` is high and `ReadyD` is low, the IFU keeps its outputs stable and waits. Figure 8.1 shows both cases. Note that `InstrF` itself does not tell whether it is valid: between cycles 7 and 9 it still holds `I0`, which has already been handed over, and only `ValidF = 0` tells Decode to ignore it. The same pair of signals connects every two neighbouring stages of HIPV, which is why the pipeline is called *elastic*: a slow stage simply holds back the stages in front of it.

8.3.2 Redirect from Execute

Jumps and branches are resolved in the Execute stage, and Execute tells the IFU about them with two signals:

- `PCSrcE` is one bit: 1 means that a jump or a taken branch has occurred, so the next PC is the target; 0 means normal, sequential execution.
- `PCTargetE` is the 32-bit target address.

`PCSrcE` lasts exactly one clock cycle. In HIPV it is `ValidEReg & ReadyM & (JumpE | JalrE | (BranchE & TakenE))`: it is high only in the cycle in which the branch leaves Execute. In that same cycle the instruction in Decode is flushed (its valid bit is gated with `~PCSrcE`), so in the next cycle Execute holds a bubble and `PCSrcE` drops back to 0. The IFU must therefore react to the redirect in the cycle it sees it. It cannot look at `PCSrcE` again later, which is the root of the discard flag (Section 8.5.2).

8.3.3 Stage outputs

The IFU hands three values to Decode:

- `InstrF`, the fetched instruction;
- `PCF`, the address the instruction was fetched from;
- `PCPlus4F`, the address of the next instruction, `PCF + 4`.

The two PC values are not needed in the IFU, but in later stages. Execute adds an offset to `PCF` to compute the target of a branch or of `jal`, and `auipc` adds an immediate to it. `PCPlus4F` is the return address that `jal` and `jalr` write into the destination register. Both travel down the pipeline together with their instruction.

8.3.4 Port to instruction memory

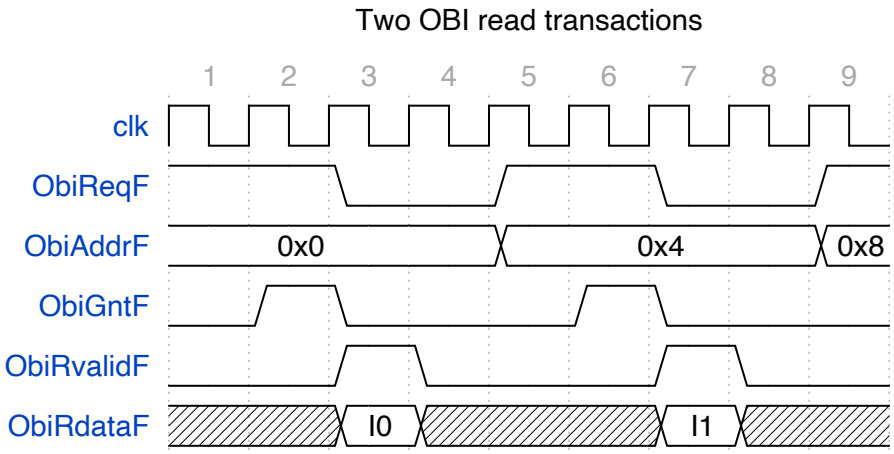


Figure 8.2: Two read transactions on the OBI port. In the address phase (cycles 1–2, 5–6) `ObiReqF` and `ObiAddrF` are held until the memory grants the request with `ObiGntF`. In the response phase, at least one cycle later, `ObiRvalidF` marks the cycle in which `ObiRdataF` carries the instruction.

The IFU reads instructions over the OBI (Open Bus Interface) protocol, which we discuss in detail later in the semester. For now, a simplified view is enough. The IFU only reads, so it uses five signals:

Signal	Direction	Meaning
`ObiReqF`	out	request: the IFU wants to start a read
`ObiAddrF`	out	address of the location to read
`ObiGntF`	in	grant: the memory has accepted the request
`ObiRvalidF`	in	response valid: `ObiRdataF` carries the read data

Signal	Direction	Meaning
`ObiRdataF`	in	the data read, here an instruction

Each read, or *transaction*, has two phases (Figure 8.2). In the **address phase**, the IFU raises `ObiReqF` and puts the address on `ObiAddrF`, and keeps both until the memory answers with `ObiGntF`. At the edge where `ObiReqF` and `ObiGntF` are both high, the memory has taken the address. This is the same valid/ready idea as the handshake with Decode. In the **response phase**, at least one cycle after the grant, the memory raises `ObiRvalidF` for one cycle, and in that cycle `ObiRdataF` holds the instruction. Outside that cycle, `ObiRdataF` is meaningless.

The memory decides how long each phase takes. In Figure 8.2 it grants after one wait cycle and responds one cycle after the grant, but a slower memory may need more cycles for either. Two properties are important for the controller. First, once the request is granted, the response *will* come, and the protocol has no way of cancelling it. Second, a response carries no tag that says which request it belongs to: responses come back in the order of the requests. HIPV keeps at most one transaction in flight at a time, so the response always belongs to the last granted request.

8.4 Structure of the IFU

Figure 8.3 shows the IFU. It has three parts.

The **PC loop** updates the program counter. The multiplexer `pcmux` chooses the next PC, `PCNextF`: the sequential `PCPlus4F` when `PCSrcE` is 0, and the jump target `PCTargetE` when it is 1. The register `pcreg` holds the current PC, `PCF`. It has an enable, `PCEnF`, driven by the controller, so the PC changes only when the controller allows it. The adder `pcadd4` computes `PCF + 4`. The current PC is also the address of the memory read, so `ObiAddrF` is simply `PCF`.

The **instruction register** `instrreg` stores the fetched instruction. Its input is the memory data `ObiRdataF`, and its enable, `InstrEnF`, is also driven by the controller: the register must capture `ObiRdataF` in exactly the cycle in which `ObiRvalidF` says that it is valid.

The **controller** `ifuctrl` manages the flow through the unit: it watches the memory, Decode and Execute, and drives the two enables, the memory request and `ValidF`. It is the subject of Section 8.5.

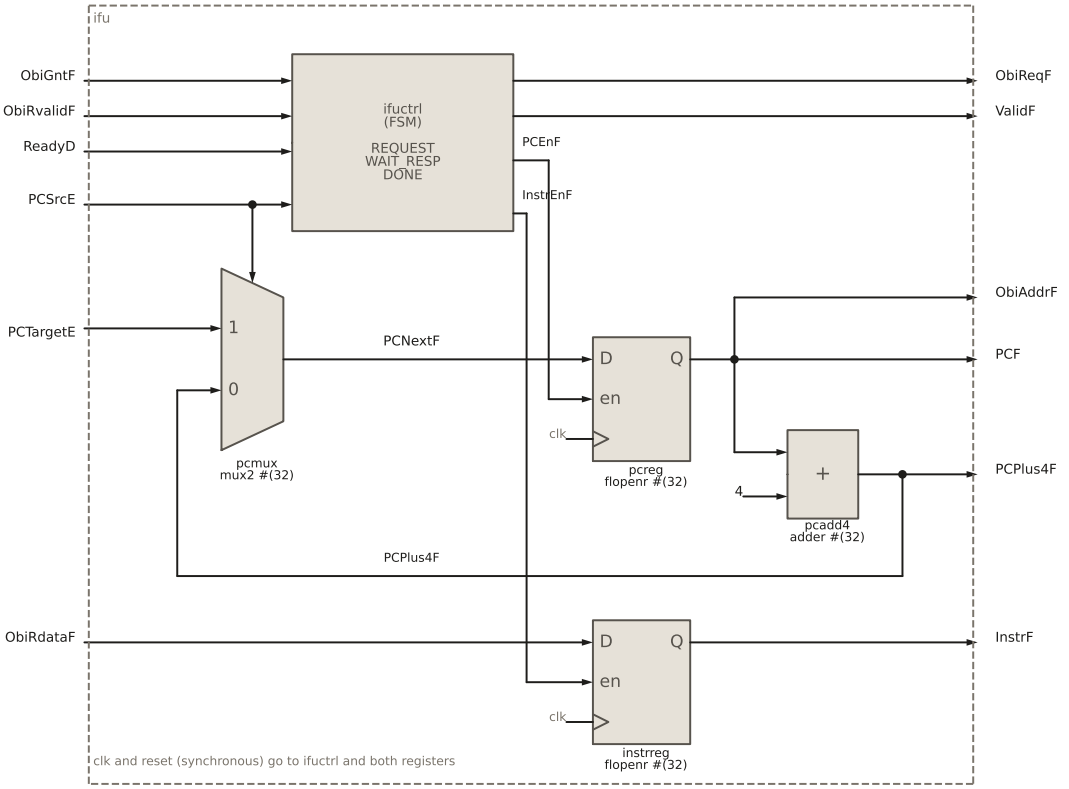


Figure 8.3: The IFU: the PC loop (`pcmux`, `pcreg`, `pcadd4`), the instruction register (`instrreg`) and the controller (`ifuctrl`).

8.4.1 The building blocks

The datapath uses three generic modules from `rtl/generic`, which we met in the chapter on designing with SystemVerilog. All three are parameterized by `WIDTH`, and the IFU uses them with 32 bits.

`flopenr` is a register with a synchronous reset and an enable. At a rising edge it clears to zero if `reset` is high, otherwise it loads `D` if `en` is high, and otherwise it keeps its value. The missing `else` branch is what makes it hold. The reset value zero is also the reset vector of HIPV: after reset, the first instruction is fetched from address 0.

```
module flopenr #(parameter WIDTH = 8) (
  input  logic             clk, reset, en,
  input  logic [WIDTH-1:0] D,
  output logic [WIDTH-1:0] Q);

  always_ff @(posedge clk)
    if (reset) Q <= '0;
    else if (en) Q <= D;

endmodule
```

`mux2` is a two-input multiplexer: `Y` is `D1` when `S` is 1 and `D0` when it is 0. It is purely combinational, one continuous assignment with the conditional operator.

```
module mux2 #(parameter WIDTH = 8) (
  input  logic [WIDTH-1:0] D0, D1,
  input  logic             S,
  output logic [WIDTH-1:0] Y);

  assign Y = S ? D1 : D0;

endmodule
```

`adder` adds two numbers. The result has the same width as the operands, so the carry out is dropped. For the PC this is the intended behavior: the address wraps around from `0xFFFFFFFC` to 0.

```
module adder #(parameter WIDTH = 8) (
  input  logic [WIDTH-1:0] A, B,
  output logic [WIDTH-1:0] Y);
```

```
  assign Y = A + B;

endmodule
```

8.4.2 The IFU in SystemVerilog

The body of `ifu` is purely structural: it is Figure 8.3 written as code, one instance per block, with the ports connected by name. The only internal signals are the next PC and the two enables.

```
  logic [31:0] PCNextF;
  logic        PCEnF, InstrEnF;

  // fetch control FSM
  ifuctrl ctrl(.clk(clk), .reset(reset), .ObiGntF(ObiGntF), .ObiRvalidF(ObiRvalidF),
               .ReadyD(ReadyD), .PCSrcE(PCSrcE), .ObiReqF(ObiReqF), .PCEnF(PCEnF),
               .InstrEnF(InstrEnF), .ValidF(ValidF));

  // next PC selection: sequential or redirect from Execute
  mux2 #(32)    pcmux(.D0(PCPlus4F), .D1(PCTargetE), .S(PCSrcE), .Y(PCNextF));

  // PC register
  flopenr #(32) pcreg(.clk(clk), .reset(reset), .en(PCEnF), .D(PCNextF), .Q(PCF));

  // sequential PC
  adder #(32)   pcadd4(.A(PCF), .B(32'd4), .Y(PCPlus4F));

  // fetched instruction register
  flopenr #(32) instrreg(.clk(clk), .reset(reset), .en(InstrEnF), .D(ObiRdataF), .Q(In

  // the OBI address is the current PC
  assign ObiAddrF = PCF;
```

Note that the select of `pcmux` is `PCSrcE` directly, not a signal of the controller. The controller only decides *when* the PC changes, through `PCEnF`, and the multiplexer decides *to what*.

8.5 The IFU controller

The controller `ifuctrl` manages the flow of instructions through the IFU. It has three tasks:

1. generate the internal enables `InstrEnF` and `PCEnF`, which decide when the instruction register and the PC register are loaded;
2. issue the memory requests through the OBI port, with `ObiReqF`;
3. tell Decode that the IFU has finished fetching an instruction, with `ValidF`.

Why do we need a controller at all, when the datapath looks so simple? For three reasons. First, fetching an instruction does not happen in one cycle. The memory needs time to grant the request and more time to respond, and the controller must watch `ObiGntF` and `ObiRvalidF` to know where the transaction is. Second, a jump can arrive at any time. The IFU must then restart fetching from the new address and, if an instruction from the old address is still on its way from memory, throw it away when it arrives. Third, Decode may not be ready to accept the instruction, and then the IFU must freeze, with its outputs stable, until it is.

All three depend on what has happened before: whether a request has been granted, whether an instruction has arrived, whether it has been handed over. The controller must remember this, so it is a finite state machine.

8.5.1 The states

A fetch goes through three steps, and each becomes a state:

- `REQUEST`: the IFU initiates the fetch. It sends a request to the memory and waits for the memory to accept it.
- `WAIT_RESP`: the request has been accepted, and the IFU waits for the response, the instruction.
- `DONE`: the instruction has arrived and sits in the instruction register. The IFU offers it to Decode and waits until Decode takes it.

In SystemVerilog, the states are an enumerated type, encoded in two bits. Following the FSM pattern of the previous chapter, the controller has a current-state and a next-state signal. It also has a second current/next pair, `DiscardF` and `NextDiscardF`, which we explain next.

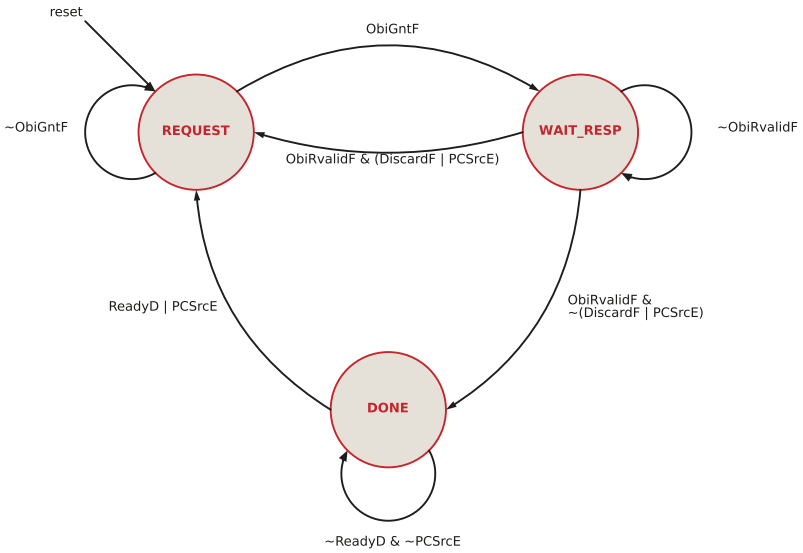


Figure 8.4: The IFU controller as a state diagram, with the transitions only.

```
typedef enum logic [1:0] {REQUEST, WAIT_RESP, DONE} statetype;

statetype State, NextState;
logic     DiscardF, NextDiscardF;
```

Figure 8.4 shows the states and the transitions between them. The following sections derive the transitions one state at a time.

8.5.2 The discard flag

Besides the state, the controller keeps one more bit, `DiscardF`, a pending request to discard the instruction that is on its way. Consider a jump that arrives after the memory has granted a request, but before it has responded. The response will come, because a granted transaction cannot be cancelled, and it will carry the instruction from the old address, which is now on the wrong path. The IFU must not hand that instruction to Decode, and it must not start the fetch from the new address before the stale response has arrived. Otherwise it would take the stale response as the answer to the new request, since responses carry no tag.

The difficulty is time. `PCSrcE` is high for a single cycle, but on a slow memory the stale response may arrive many cycles later, when `PCSrcE` is long gone. So the controller needs one bit of memory that says “the response I am waiting for belongs to the wrong path”: `DiscardF`. It is set when `PCSrcE` arrives while a transaction is outstanding, which is in

`WAIT_RESP`, or in `REQUEST` in the very cycle the grant arrives. It is cleared when the awaited response arrives, and that response is dropped instead of captured.

8.5.3 Transitions

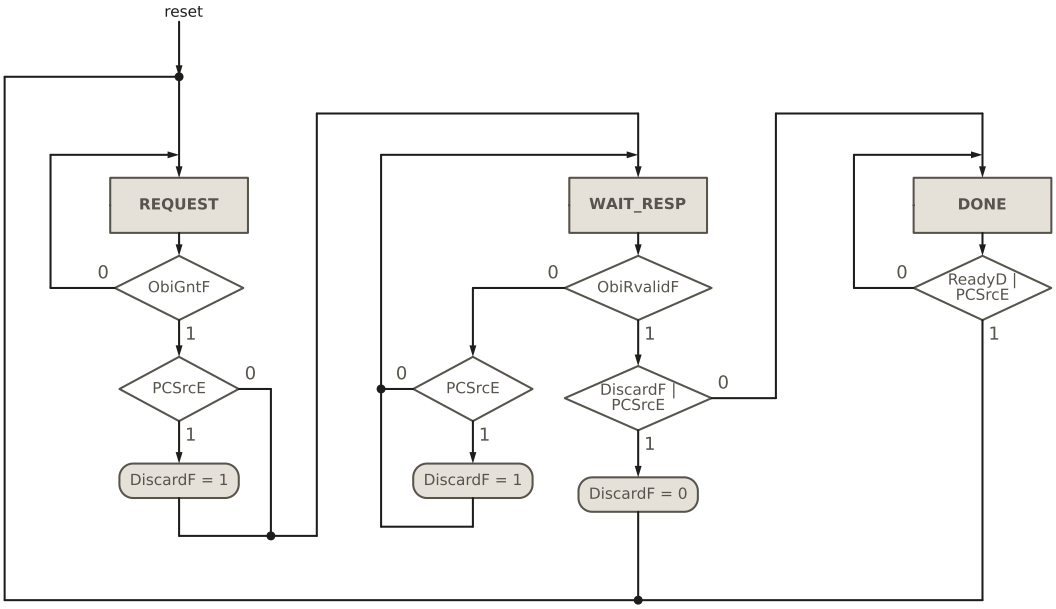


Figure 8.5: ASM chart of the IFU controller: the transitions and the updates of `DiscardF`. Where no box is shown, `DiscardF` keeps its value.

Figure 8.5 shows the transitions as an ASM chart, one column per state, together with the updates of `DiscardF` in the rounded conditional boxes. Let us go through it state by state.

From `REQUEST`. First, we ask whether the memory has accepted our request, by checking `ObiGntF`. If not, we stay in `REQUEST` and keep asking. If it has, the transaction is committed and we move to `WAIT_RESP`. Before we go, we check whether a jump has occurred in this same cycle (`PCSrcE`). If so, the response that will come belongs to the old PC, and we raise `DiscardF` to 1. A jump in `REQUEST` without the grant needs no flag: nothing has been committed yet, the PC is loaded with the target in this cycle, and the request simply continues with the new address.

From `WAIT_RESP`. First, we ask whether the instruction has arrived from memory, by checking `ObiRvalidF`. If not, we keep waiting in `WAIT_RESP`. While we wait, a jump may occur, so we check `PCSrcE` and, if it is 1, raise `DiscardF`: the response we are waiting for is now stale. If the instruction has arrived, we check whether we must discard it, by reading `DiscardF` and `PCSrcE`. `DiscardF` covers a jump in an earlier cycle, and `PCSrcE` a jump in this

very cycle. If either is 1, we drop the instruction, clear the pending discard, and restart the fetch by moving to `REQUEST`. Otherwise, the instruction is good, and we proceed to `DONE`.

From `DONE`. We have the instruction, and now we must finish with it. There are two ways out. In the normal one, `ReadyD` is 1: Decode takes the instruction, and we can fetch the next one. In the other, a jump has occurred, the instruction we hold is on the wrong path, and we must restart the fetch from the target. If either condition is met, we move to `REQUEST`; otherwise we stay in `DONE`. No transaction is outstanding in `DONE`, so `DiscardF` does not change here.

The flag is cleared only in `WAIT_RESP`, because that is the only state in which a response can arrive. On the path to `DONE`, the flag is already 0, so no box is needed there.

8.5.4 The controller in SystemVerilog

The code follows the chart column by column. `State` and `DiscardF` are updated in the same register block, with a synchronous reset to `REQUEST` and no pending discard:

```
// state register
always_ff @(posedge clk)
  if (reset) begin
    State    <= REQUEST;
    DiscardF <= 1'b0;
  end else begin
    State    <= NextState;
    DiscardF <= NextDiscardF;
  end
```

The next-state logic computes both next values in one `always_comb` block. It starts with the defaults, hold the state and hold the flag, and each branch of the `case` overrides only what changes:

```
// next state logic
always_comb begin
  NextState    = State;  // defaults: hold
  NextDiscardF = DiscardF;
  case (State)
    // the grant commits the transaction on this edge, so a redirect
    // in the same cycle already has a wrong-path response coming
    REQUEST:   begin NextState    = ObiGntF ? WAIT_RESP : REQUEST;
                     NextDiscardF = ObiGntF & PCSrcE; end
```

```
      // the awaited response clears the flag; with no response yet, a
      // redirect makes the outstanding one stale
      WAIT_RESP: if (ObiRvalidF) begin NextState    = (DiscardF | PCSrcE) ? REQUEST :
                                       NextDiscardF = 1'b0; end
                 else NextDiscardF = DiscardF | PCSrcE;
      DONE:      if (PCSrcE | ReadyD) NextState = REQUEST;
      default:   NextState = REQUEST;
    endcase
  end
```

Keeping `DiscardF` in the same current/next pattern as `State` matters in `WAIT_RESP`. When the response arrives, the decision `(DiscardF | PCSrcE)` reads the *current* `DiscardF`, while the clear goes to `NextDiscardF` and only takes effect at the next edge. The flag is therefore still valid in the cycle in which the stale response is examined. In `REQUEST` the flag is always 0, so `NextDiscardF = ObiGntF & PCSrcE` is the same as "set it on a jump in the cycle of the grant".

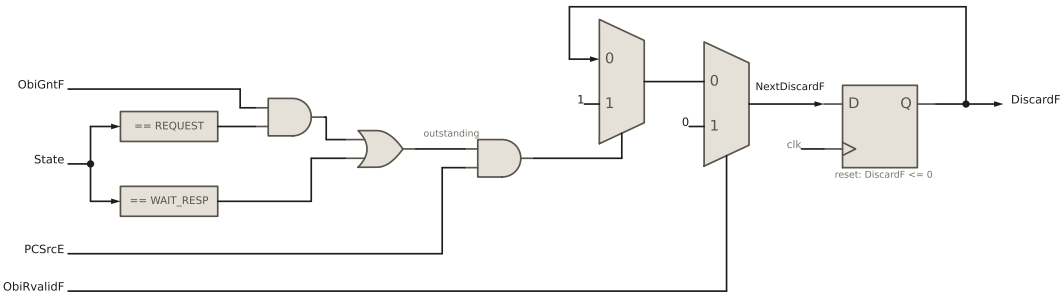


Figure 8.6: The `NextDiscardF` logic drawn as hardware. The two priority multiplexers clear the flag on a response and set it on a jump during an outstanding transaction; otherwise the register holds its value. Since a response arrives only in `WAIT_RESP`, this is the same function as the `case` in the code.

Figure 8.6 shows what the `DiscardF` lines become in hardware: one flip-flop and two multiplexers. The first sets the flag when `PCSrcE` arrives while a transaction is outstanding; the second, which has priority, clears it when the response arrives. If neither applies, the flip-flop gets its own output back, and holds.

8.5.5 The outputs

The four outputs are combinational functions of the state, the flag and the inputs:

```
// outputs
assign ObiReqF  = (State == REQUEST);
assign PCEnF    = PCSrcE | ((State == DONE) & ReadyD);
assign InstrEnF = (State == WAIT_RESP) & ObiRvalidF & ~DiscardF & ~PCSrcE;
assign ValidF   = (State == DONE) & ~PCSrcE;
```

- `ObiReqF` is high in `REQUEST`. It depends only on the state, so it is a Moore output. The request stays high until the grant moves the FSM to `WAIT_RESP`, exactly as the OBI address phase requires.
- `PCEnF` lets the PC change in two cases. On a jump, the PC loads the target immediately, in any state, because `pcmux` selects `PCTargetE` whenever `PCSrcE` is 1. In `DONE`, when Decode takes the instruction (`ReadyD`), the PC advances to `PCPlus4F`. The PC must not advance earlier, because `PCF` is a stage output that belongs to the instruction that Decode has not taken yet.
- `InstrEnF` loads the instruction register in the cycle the response arrives (`WAIT_RESP` and `ObiRvalidF`), unless the response is stale (`DiscardF`) or a jump arrives in the same cycle (`PCSrcE`). Both are exactly the conditions under which the FSM returns to `REQUEST` instead of going to `DONE`.
- `ValidF` is high in `DONE`, while the instruction waits for Decode. A jump turns it off at once, in the same cycle, so that Decode never takes an instruction from the wrong path.

All outputs except `ObiReqF` also depend on the inputs, so they are Mealy outputs: they react in the same cycle, without waiting for a state change. That is what lets the IFU take a jump in the very cycle `PCSrcE` is high.

8.5.6 The controller in the waveform

Figure 8.7 shows the controller at work, on a memory that grants after one wait cycle and responds one cycle after the grant. In cycle 2 the grant arrives, and the FSM moves to `WAIT_RESP`. In cycle 3 the response arrives; `InstrEnF` is high, so at the end of the cycle the instruction register captures `I0`, and the FSM moves to `DONE`. From cycle 4, `ValidF` offers `I0` to Decode, but Decode is busy for two cycles. In cycle 6 `ReadyD` rises, `PCEnF` follows it, and at the end of the cycle the PC advances to `0x4` and the FSM starts the next fetch.

Each instruction spends at least one cycle in each state, so the IFU fetches at most one instruction every three cycles, even with an ideal memory that grants at once. This is a deliberate simplification that keeps the controller small. A faster IFU would issue the next request before the previous response arrives, and keep the extra instructions in a small buffer.

Figure 8.8 shows the discard flag at work, on a memory that takes three cycles to respond. In cycle 1 the request for address `A` is granted. In cycle 2, while the IFU waits for the

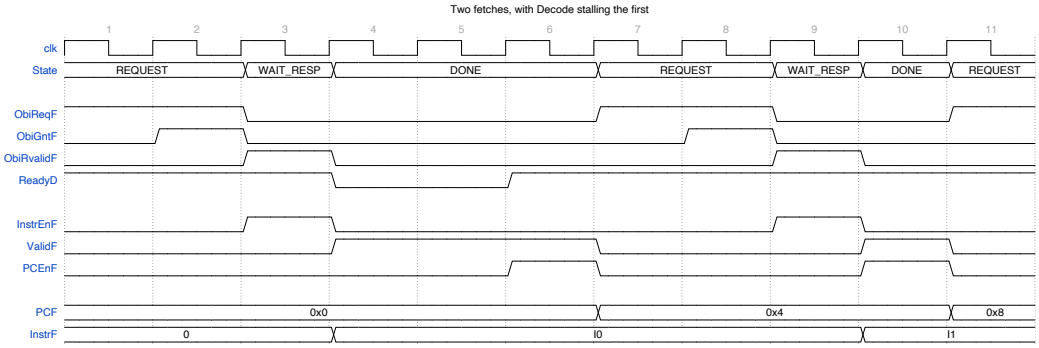


Figure 8.7: Two fetches. Each passes through `REQUEST`, `WAIT_RESP` and `DONE`. `InstrEnF` follows `ObiRvalidF`, and `PCEnF` rises only when Decode takes the instruction: in cycle 6 for the first one, after two cycles of `ReadyD = 0`.

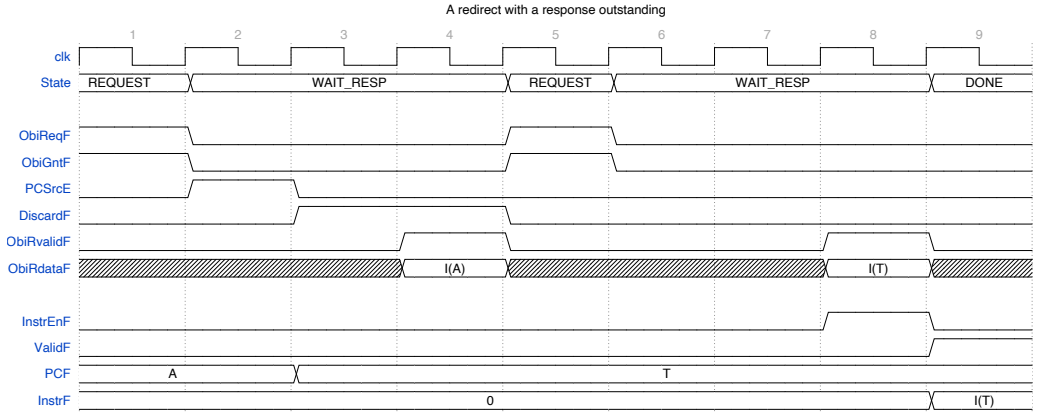


Figure 8.8: A redirect while a transaction is outstanding. The jump in cycle 2 sets `DiscardF` and loads the PC with the target `T`. The response for the old PC `A` arrives in cycle 4 and is dropped; the fetch restarts from `T` in cycle 5.

response, Execute resolves a jump to `T`: `PCSrcE` is high, so `PCEnF` loads `T` into the PC, and `NextDiscardF` is 1. In cycle 4 the response for `A` arrives. `DiscardF` is 1, so `InstrEnF` stays low, the instruction register keeps its old value, and the FSM returns to `REQUEST` while the flag clears. From cycle 5 on, the IFU fetches from `T` as usual, and in cycle 9 it offers `I(T)` to Decode.

Without the flag, the FSM in cycle 4 would capture `I(A)` and offer it to Decode together with `PCF = T`: the wrong instruction, with the address of another one.

8.6 Summary

The IFU is a small datapath, the PC loop and the instruction register, built from the generic blocks `flopenr`, `mux2` and `adder`, and a controller that decides when each of them acts. The controller is a three-state FSM with one extra bit of memory, the discard flag, which remembers a jump for as long as a stale instruction is on its way from memory.

The design makes two deliberate simplifications. It keeps one memory transaction in flight at a time, which limits it to one instruction every three cycles. And a jump in `REQUEST`, before the grant, changes `ObiAddrF` while `ObiReqF` is high. Strict OBI requires the address to stay stable during the address phase, but simple memories, which take the address only in the cycle of the grant, tolerate the change. Both are natural extensions for later in the course.

9 Instruction decode unit

Instruction decode unit

9.1 Goal for this week

- Know what the instruction decode unit (IDU) does, and which other parts of the core it talks to
- Understand the valid/ready handshake that connects all stages of HIPV, and how the IDU implements it
- Build the control decoder: from opcode, `funct3` and `funct7` to control signals
- Build the immediate extension and the register file

9.2 Introduction

The instruction decode unit is the second stage of the HIPV pipeline. It is the place where hardware and software meet. Up to this point, an instruction is just 32 bits that Fetch has brought from memory; from here on, it is a set of control signals that tell the rest of the core what to do. The choice of the instruction set, RV32I, has a greater impact on this stage than on any other. Its formats decide where the register numbers sit in the instruction, how the immediate is scattered over the bits, and which opcodes exist. If you wanted to add a new instruction to HIPV, this is where you would start: give it an opcode in the decoder, and define the control signals it needs.

The IDU has three tasks:

- **Decode** the instruction: produce the control signals that steer the later stages, and hand them on together with the operands to the Execute stage.
- **Extend** the immediate: collect the immediate bits from the instruction and extend them to a 32-bit value.
- **Access the register file**: read the two source registers, which the later stages (the IEU and the Memory stage) use as operands. The register file is also written here, but the write comes from the Writeback stage, at the other end of the pipeline.

9.3 Interface of the IDU

The ports of the IDU fall into five groups: the handshake with Fetch and the data coming from it, the jump from Execute, the handshake with Execute, the write port of the register file driven by Writeback, and the stage outputs.

```
module idu (input  logic        clk, reset,
            // handshake with the Fetch stage
            input  logic        ValidF,
            output logic        ReadyD,
            input  logic [31:0] InstrF, PCF, PCPlus4F,
            // flush on taken branch/jump resolved in Execute
            input  logic        PCSrcE,
            // handshake with the Execute stage
            output logic        ValidD,
            input  logic        ReadyE,
            // write port from the Writeback stage
            input  logic        RegWriteW,
            input  logic [4:0]  RdW,
            input  logic [31:0] ResultW,
            // stage outputs to Execute
            output logic [31:0] RD1D, RD2D, ImmExtD, PCD, PCPlus4D,
            output logic [4:0]  Rs1D, Rs2D, RdD,
            output logic [2:0]  Funct3D,
            output logic        RegWriteD, MemWriteD, BranchD, JumpD, JalrD,
            output logic [1:0]  ResultSrcD, ALUSrcAD,
            output logic        ALUSrcBD,
            output logic [3:0]  ALUControlD);
```

- **Handshake with Fetch and data from Fetch.** `ValidF` and `ReadyD` agree on each transfer. The data is the instruction `InstrF` and its two addresses, `PCF` and `PCPlus4F`.
- **Jump from Execute.** When `PCSrcE` is high, Execute has taken a branch or a jump, and the instruction held in Decode was fetched from the wrong path. The IDU throws it away.
- **Handshake with Execute.** `ValidD` and `ReadyE`, the same pair one stage further down.
- **Write port, from Writeback.** `RegWriteW`, `RdW` and `ResultW` write a result into the register file. They belong to a different instruction, the one that is finishing in Writeback.
- **Stage outputs.** The operands read from the register file (`RD1D`, `RD2D`) and the immediate (`ImmExtD`); the register numbers to read and to write (`Rs1D`, `Rs2D`, `RdD`); the

two addresses (`PCD`, `PCPlus4D`); `Funct3D`, which later stages still need to choose the branch condition and the memory access size; and the control signals.

9.4 Structure of the IDU

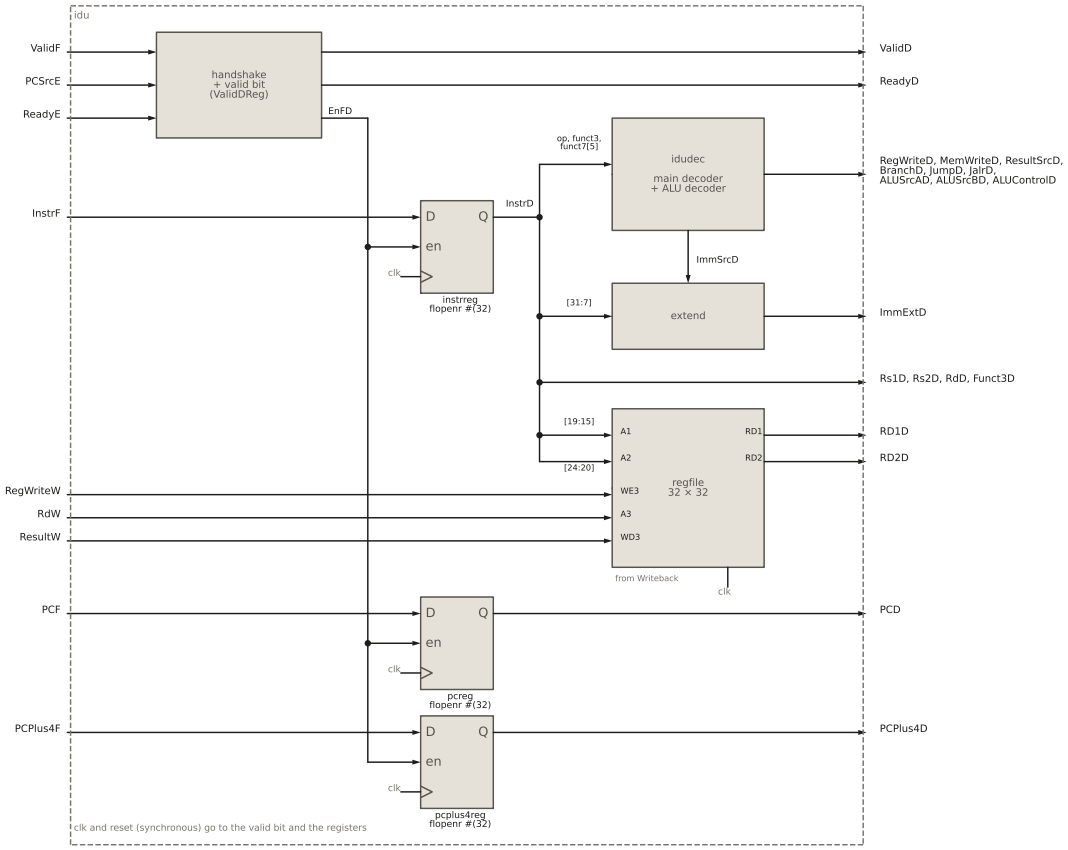


Figure 9.1: The IDU: the handshake logic and the F/D pipeline registers, the control decoder (`idudec`), the immediate extension (`extend`) and the register file (`regfile`).

Figure 9.1 shows the IDU. It has five parts: the handshake logic, which decides when the stage takes a new instruction from Fetch; the pipeline registers, which hold that instruction; and three blocks that work on it in parallel: the control decoder `idudec`, the immediate extension `extend`, and the register file `regfile`. The register numbers and `funct3` need no block at all: they are fixed bit fields of the instruction, and the IDU simply wires them out.

9.5 The valid/ready handshake

In the simplest pipeline, every stage hands its result to the next stage at every clock edge. That works only if every stage always finishes its work in exactly one cycle. HIPV is not like that. Fetch and the Memory stage talk to memories that answer when they are ready, which can take any number of cycles. Two things can therefore happen:

- A **later stage is busy.** A load in the Memory stage waits for its data, and the instruction behind it in Execute cannot move on. If Execute handed over its result anyway, the instruction in the Memory stage would be overwritten. The stages behind a busy stage must wait: this is a **stall**.
- An **earlier stage has nothing to send.** Fetch is still waiting for the instruction memory, so in this cycle Decode receives nothing. The empty slot that travels down the pipeline is a **bubble**.

The pipeline handles both with a **handshake**, a universal agreement between every two neighbouring stages: data moves from one stage to the next only when the sender has something to send *and* the receiver is able to take it. Each stage makes this decision locally, with its two neighbours, and does not need to know what is going on anywhere else.

9.5.1 Two signals

The handshake uses two control signals next to the data:

Signal	Driven by	Meaning
`valid`	sender	The data output carries a real item, not leftovers from an earlier one or random bits after reset.
`ready`	receiver	The receiver can accept an item at the next clock edge.
`data`	sender	The item itself: an instruction with its operands and controls. It is meaningful only while `valid` is 1.

An item is **transferred** at a rising clock edge where `valid` and `ready` are both 1. In HIPV the names carry the stage suffix: between Fetch and Decode the pair is `ValidF` and `ReadyD`, between Decode and Execute it is `ValidD` and `ReadyE`, and so on down the pipeline.

9.5.2 Four cases

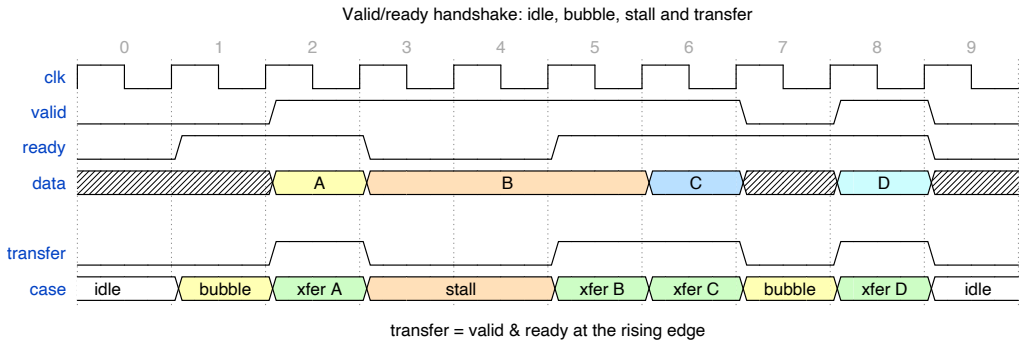


Figure 9.2: The four combinations of `valid` and `ready`. An item is transferred in each cycle where both are high (`transfer`). In cycles 3 and 4 the receiver stalls, and the sender holds `B` until the transfer in cycle 5.

The two signals give four cases, and Figure 9.2 shows all of them:

`valid`	`ready`	Name	What happens
0	0	idle	The sender has nothing, and the receiver has no room. Nothing happens.
0	1	bubble	The receiver could take an item, but there is none. At the edge the receiver records that it has received nothing: an empty slot moves down the pipeline.
1	0	stall	The sender has an item, but the receiver cannot take it. The sender keeps its item and its outputs unchanged, and tries again in the next cycle.

valid	ready	Name	What happens
1	1	transfer	The item moves to the receiver at the clock edge. In the next cycle the sender may offer a new item.

The transfer case can repeat in every cycle, as in cycles 5 and 6 of the figure: when both sides are always ready, the handshake costs nothing, and the pipeline moves one item per cycle as if the handshake were not there.

9.5.3 Rules of the handshake

For the handshake to work, both sides follow four rules:

1. **Once `valid` is set, it holds until the transfer.** The sender may not withdraw an item, or change it, while waiting for `ready`. Otherwise the receiver could take a value that changes under it.
2. **`valid` must not depend on `ready`.** The sender decides on its own whether it has something to offer. If `valid` waited for `ready`, and the receiver in turn made `ready` wait for `valid`, neither would ever move. Combinational paths between the two would form a loop. The opposite direction is allowed: `ready` may depend on `valid`.
3. **`data` is meaningful only while `valid` is 1.** In between, the data lines may hold anything, and the receiver ignores them.
4. **A transfer happens at one clock edge, and moves one item.** If both signals stay high in the next cycle, that is a new transfer of the next item.

HIPV breaks rule 1 on purpose, in exactly one place. When Execute takes a jump, the instruction offered by Decode belongs to the wrong path, and Decode withdraws it by dropping `ValidD`, as we will see shortly. This is safe because at the same moment, everything behind the jump is being thrown away.

9.6 Handshake logic of the IDU

The handshake logic of the IDU is a few lines of code (Figure 9.3):

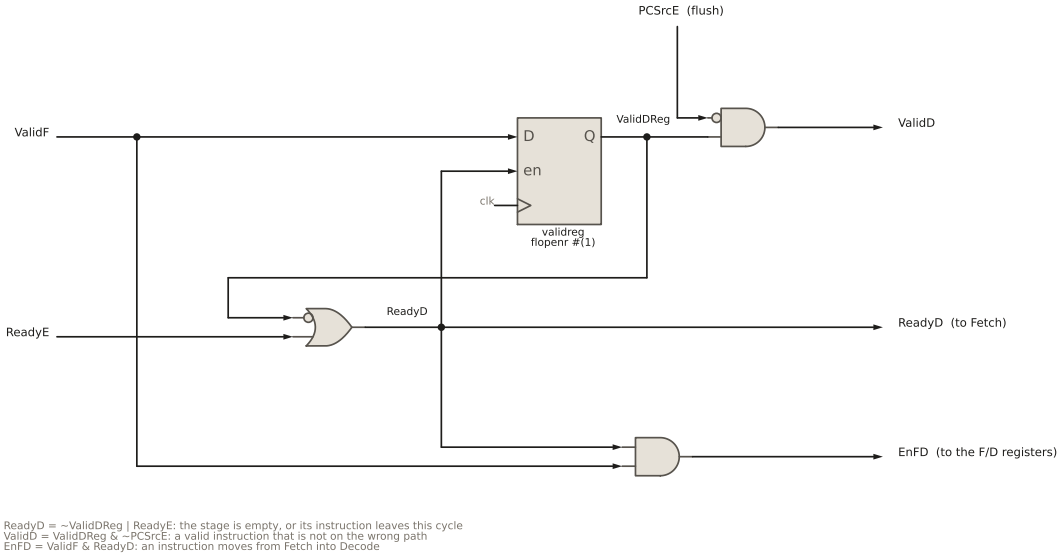


Figure 9.3: The handshake logic of the IDU. `ReadyD` tells Fetch that Decode can take a new instruction, `EnFD` loads the F/D registers, `validreg` remembers whether the stage holds a valid instruction, and `PCSrcE` hides it from Execute on a jump.

```
// F/D handshake: accept when the stage is empty or being drained
assign ReadyD = ~ValidDReg | ReadyE;
assign EnFD   = ValidF & ReadyD;
assign ValidD = ValidDReg & ~PCSrcE;

// F/D valid bit
always_ff @(posedge clk)
  if (reset) ValidDReg <= 1'b0;
  else if (ReadyD) ValidDReg <= ValidF;
```

The most important signal is **`EnFD`**, the transfer between Fetch and Decode: Fetch offers an instruction (`ValidF`) and Decode accepts it (`ReadyD`). It is the enable of the pipeline registers, so it is the moment the instruction enters Decode.

`ReadyD` says that Decode can accept a new instruction. That is true in two situations:

- **The stage is empty**, `~ValidDReg`: it holds no valid instruction, so there is nothing to lose.
- **The stage is being drained**, `ReadyE`: Execute takes the instruction that Decode holds at this very edge, so its place is freed and can be refilled at the same edge. This is what lets the pipeline move one instruction per cycle. When Execute stalls, `ReadyE`

is 0, and with a valid instruction in Decode, `ReadyD` is 0 too: the stall propagates backwards, to Fetch.

`ValidDReg` is the valid bit of the stage, a one-bit register that says whether the pipeline registers hold a real instruction. It is a delayed copy of `ValidF`: whenever the stage can accept something (`ReadyD`), it stores `ValidF`, so it becomes 1 when an instruction arrives and 0 when a bubble arrives.

Why keep a register for it, instead of using `ValidF` directly? Because the valid signal must travel through the pipeline at the same pace as the data it labels. `ValidF` describes `InstrF`, the instruction on the output of Fetch in this cycle. After the edge, that instruction sits in `InstrD`, and Fetch may already be working on the next one, with a different `ValidF`. The label must therefore be stored at the same edge, and in a register next to the data, so that `ValidDReg` always describes what is in `InstrD`. The same happens in every stage: each pipeline register has its valid bit, and an instruction and its valid bit always move together.

Note the small difference between the two enables. The data registers load on `EnFD`, only when an instruction actually arrives. The valid bit loads on `ReadyD`, also when a bubble arrives. In that case the data registers keep the old instruction, which has already moved to Execute, and the valid bit becomes 0 to mark it as stale. Rule 3 of the handshake makes this safe: nobody looks at the data while the valid bit is 0.

`ValidD` offers the instruction to Execute. It is the valid bit, with one exception: when `PCSrcE` is high, Execute has taken a jump, and the instruction in Decode is the one fetched after the jump, on the wrong path. `ValidD` drops at once, in the same cycle, so Execute never takes it. At the following edge the valid bit is cleared as well. This needs no extra logic: on a jump, Fetch also drops `ValidF`, and since the jump leaves Execute at this edge (`PCSrcE` implies `ReadyE`), `ReadyD` is 1, and the valid bit loads the 0 from `ValidF`. The instruction has become a bubble, which is called a **flush**.

9.7 Pipeline registers

The F/D pipeline registers store the instruction and its two addresses. They are three instances of the generic `flopenr`, all with the same enable, `EnFD`:

```
// F/D pipeline registers
flopenr #(32) instrreg(.clk(clk), .reset(reset), .en(EnFD), .D(InstrF), .Q(InstrD));
flopenr #(32) pcreg(.clk(clk), .reset(reset), .en(EnFD), .D(PCF), .Q(PCD));
flopenr #(32) pcplus4reg(.clk(clk), .reset(reset), .en(EnFD), .D(PCPlus4F), .Q(PCPlu
```

Each register does one of two things at a clock edge:

- **update**: when `EnFD` is 1, it stores the new value from Fetch;
- **hold**: when `EnFD` is 0, it keeps its current value.

The enable is the key component that makes the pipeline elastic. Without it, the register would load a new value at every edge, and a stall would lose the instruction. With it, the instruction in Decode simply stays where it is, with all the outputs computed from it stable, for as long as Execute is not ready to take it.

The IDU stores only three values. Everything else it produces (the control signals, the immediate, the register operands) is computed from `InstrD` by combinational logic, in the cycle the instruction is in Decode, and stored only in the next pipeline register, at the entry to Execute.

9.8 Instruction fields

RV32I is designed so that decoding is easy. In every format that has them, the register numbers and `funct3` sit at the same bit positions, so the IDU can extract them with plain wiring, before it even knows what kind of instruction it holds:

```
// instruction fields
assign Rs1D    = InstrD[19:15];
assign Rs2D    = InstrD[24:20];
assign RdD     = InstrD[11:7];
assign Funct3D = InstrD[14:12];
```

The six RV32I formats differ only in which of these fields they use and how they place the immediate:

Format	31:25	24:20	19:15	14:12	11:7	6:0
R	`funct7`	`rs2`	`rs1`	`funct3`	`rd`	`opcode`
I	`imm[11:5]`	`imm[4:0]`	`rs1`	`funct3`	`rd`	`opcode`
S	`imm[11:5]`	`rs2`	`rs1`	`funct3`	`imm[4:0]`	`opcode`
B	`imm[12,10:5]`	`rs2`	`rs1`	`funct3`	`imm[4:1,11]`	`opcode`
U	`imm[31:25]`	`imm[24:20]`	`imm[19:15]`	`imm[14:12]`	`rd`	`opcode`
J	`imm[20,10:1,11]`	`imm[4:1,11]`	`imm[19:15]`	`imm[14:12]`	`rd`	`opcode`

For an instruction that does not use a field, the extracted value is meaningless. For a store, `RdD` holds immediate bits. That is harmless, because the control signals say which fields count: a store has `RegWriteD = 0`, so its `RdD` is never used to write a register.

9.9 The control decoder

The task of the decoder, `idudec`, is to generate the control signals for the instruction. It looks at three parts of the instruction:

- the **opcode**, bits 6:0, which says what kind of instruction it is: a register–register operation, a load, a branch, …;
- **`funct3`**, bits 14:12, which distinguishes the ALU operations within one opcode (`add`, `xor`, `sll`, …), and, for branches and memory accesses, the condition and the size;
- **`funct7[5]`**, bit 30 of the instruction, called `Funct7b5D`, which distinguishes the two pairs of operations that share `funct3`: `add` from `sub` and `srl` from `sra`.

From them it produces the control signals for every later part of the core:

Signal	Width	For	Meaning
`ALUControlD`	4	ALU	which of the ten operations
`ALUSrcAD`	2	ALU	operand A: register, PC or zero
`ALUSrcBD`	1	ALU	operand B: register or immediate
`BranchD`	1	Execute	a conditional branch
`JumpD`, `JalrD`	1	Execute	`jal`, `jalr`
`MemWriteD`	1	Memory	a store
`ResultSrcD`	2	Writeback	what to write back: ALU result, loaded data, or `PC + 4`
`RegWriteD`	1	Writeback	the instruction writes `rd`
`ImmSrcD`	3	`extend` (here)	the immediate format

Only `ImmSrcD` is used in Decode itself. All other signals travel down the pipeline with the instruction, and each is used in the stage named in the table.

9.9.1 The HIPV package

The decoder produces encoded signals, such as `ALUControlD = 4'b0001` for a subtraction, and the IEU must interpret them in exactly the same way. If each module wrote its own

numbers, a change on one side would silently break the other. HIPV therefore keeps all encodings in one place, a **package**. A package is a named collection of declarations (constants, types, functions) that does not describe hardware by itself, but can be used by any module. The HIPV package, `config/hipv_pkg.sv`, defines the encodings of all control signals and the RV32I opcodes as `localparam` constants. For example, the immediate formats and the opcodes:

```
package hipv_pkg;
  ...
  // immediate format select (ImmSrc)
  localparam logic [2:0] IMM_I = 3'b000;
  localparam logic [2:0] IMM_S = 3'b001;
  localparam logic [2:0] IMM_B = 3'b010;
  localparam logic [2:0] IMM_J = 3'b011;
  localparam logic [2:0] IMM_U = 3'b100;
  ...
  // RV32I opcodes
  localparam logic [6:0] OP_RTYPE  = 7'b0110011;
  localparam logic [6:0] OP_IALU   = 7'b0010011;
  localparam logic [6:0] OP_LOAD   = 7'b0000011;
  localparam logic [6:0] OP_STORE  = 7'b0100011;
  localparam logic [6:0] OP_BRANCH = 7'b1100011;
  localparam logic [6:0] OP_JAL    = 7'b1101111;
  localparam logic [6:0] OP_JALR   = 7'b1100111;
  localparam logic [6:0] OP_LUI    = 7'b0110111;
  localparam logic [6:0] OP_AUIPC  = 7'b0010111;
endpackage
```

A module that needs the constants imports the package at the top of its body:

```
module idudec (...);

  import hipv_pkg::*;
```

`hipv_pkg::*` makes every name of the package visible in the module, so the code can write `OP_LOAD` instead of `7'b0000011`. The decoder (`idudec`), the extension unit (`extend`), the ALU (`alu`) and the IEU all import the same package, so they always agree on the encodings. The code also becomes readable: `ResultSrcD = RESULT_MEM` says what it means, while `ResultSrcD = 2'b01` does not. The package must be compiled before the modules that import it, which is why it comes first in the file lists of the simulation and synthesis scripts.

9.9.2 The control word

The decoder has a dozen output signals, and each opcode must set all of them. Rather than assigning each signal separately in every branch, the decoder collects them into one vector, the **control word**:

```
  logic [14:0] ControlsD;
  logic [1:0]  ALUOpD;

  assign {RegWriteD, ImmSrcD, ALUSrcAD, ALUSrcBD, MemWriteD, ResultSrcD,
          BranchD, JumpD, JalrD, ALUOpD} = ControlsD;
```

This is the concatenation operator on the left-hand side of an assignment, which we met in the SystemVerilog basics to catch the carry of an adder: `assign {carry_out, sum} = a + b;`. On the right of an assignment, `{}` joins signals into one vector. On the left, it does the reverse: it splits a vector into pieces and assigns each piece to one signal, from the most significant bit down. The widths add up to 15:

Signal	RegWriteD	ImmSrcD	ALUSrcAD	ALUSrcBD	MemWriteD	ResultSrcD	BranchD	JumpD	JalrD	ALUOpD
Width	1	3	2	1	1	2	1	1	1	2
Bits of `ControlsD`	14	13:11	10:9	8	7	6:5	4	3	2	1:0

The assignment is pure wiring, without any gates. Its benefit is in the code that drives `ControlsD`: each opcode needs just one line, and that line reads like a row of a table.

9.9.3 The main decoder

The main decoder is one `case` on the opcode, inside `always_comb`. Each branch assigns the whole control word, using the package names for the encoded fields:

```
  // main decoder
  //                        RegWrite ImmSrc SrcA       SrcB  MemWrite ResultSrc   Bran
  always_comb
    case (OpD)
      OP_RTYPE:  ControlsD = {1'b1, IMM_I, SRCA_RF,   1'b0, 1'b0, RESULT_ALU, 1'b0, 1'
      OP_IALU:   ControlsD = {1'b1, IMM_I, SRCA_RF,   1'b1, 1'b0, RESULT_ALU, 1'b0, 1'
```

```
      OP_LOAD:   ControlsD = {1'b1, IMM_I, SRCA_RF,   1'b1, 1'b0, RESULT_MEM, 1'b0, 1'
      OP_STORE:  ControlsD = {1'b0, IMM_S, SRCA_RF,   1'b1, 1'b1, RESULT_ALU, 1'b0, 1'
      OP_BRANCH: ControlsD = {1'b0, IMM_B, SRCA_RF,   1'b0, 1'b0, RESULT_ALU, 1'b1, 1'
      OP_JAL:    ControlsD = {1'b1, IMM_J, SRCA_RF,   1'b0, 1'b0, RESULT_PC4, 1'b0, 1'
      OP_JALR:   ControlsD = {1'b1, IMM_I, SRCA_RF,   1'b1, 1'b0, RESULT_PC4, 1'b0, 1'
      OP_LUI:    ControlsD = {1'b1, IMM_U, SRCA_ZERO, 1'b1, 1'b0, RESULT_ALU, 1'b0, 1'
      OP_AUIPC:  ControlsD = {1'b1, IMM_U, SRCA_PC,   1'b1, 1'b0, RESULT_ALU, 1'b0, 1'
      default:   ControlsD = 15'b0;  // unimplemented opcode: nop
    endcase
```

Each row is the complete recipe for one kind of instruction. Read the load row from left to right: it writes a register; its immediate has the I format; the ALU adds the register `rs1` and the immediate, which gives the address; it does not write memory; the value written back is the loaded data; and it is neither a branch nor a jump. A store differs in the immediate format (S), in writing memory, and in not writing a register.

Some fields do not matter for some instructions, and the table simply fills them with a harmless value. An R-type instruction has no immediate, so its `ImmSrcD` is arbitrary. A store or branch does not write back, so its `ResultSrcD` is arbitrary. A few choices are worth noting:

- `jal` and `jalr` write the return address, so their `ResultSrcD` is `RESULT_PC4`. Their targets are computed in Execute: `jal` uses the dedicated `PC + imm` adder, while `jalr` uses the ALU to add `rs1` and the immediate, which is why only `jalr` sets `ALUSrcBD`.
- `lui` and `auipc` both add the U immediate in the ALU. They differ only in operand A: zero for `lui`, the PC for `auipc`.
- Opcodes that HIPV does not implement, such as `fence` and the system instructions, fall into `default` and get an all-zero control word. It writes nothing, stores nothing, and does not jump, so the instruction passes through the pipeline as a no-op.

`ALUOpD` is not a final ALU operation, but a hint for the second part of the decoder: `10` means "the operation is given by `funct3` and `funct7`", and `00` means "just add".

9.9.4 The ALU decoder

The ALU decoder turns `ALUOpD`, `funct3` and `funct7[5]` into the final `ALUControlD`:

```
  // sub only exists as an R-type encoding (addi has no subi)
  assign RtypeSubD = Funct7b5D & OpD[5];

  // ALU decoder
```

```
  always_comb
    if (ALUOpD == 2'b10)
      case (Funct3D)
        3'b000:  ALUControlD = RtypeSubD ? ALU_SUB : ALU_ADD;
        3'b001:  ALUControlD = ALU_SLL;
        3'b010:  ALUControlD = ALU_SLT;
        3'b011:  ALUControlD = ALU_SLTU;
        3'b100:  ALUControlD = ALU_XOR;
        3'b101:  ALUControlD = Funct7b5D ? ALU_SRA : ALU_SRL;
        3'b110:  ALUControlD = ALU_OR;
        default: ALUControlD = ALU_AND;
      endcase
    else ALUControlD = ALU_ADD;  // address / target computation
```

For the arithmetic instructions (`ALUOpD = 10`), `funct3` selects the operation directly, and it does so in the same way for the R-type and the I-type form: `add` and `addi` both have `funct3 = 000`, `xor` and `xori` both `100`. For every other instruction, the ALU adds: a load or store computes its address, `jalr` its target, `lui` and `auipc` their results. A branch has `ALUOpD = 01`, but it also gets `ADD`. Its condition is evaluated by the branch comparator, so the ALU result is not used.

Two values of `funct3` need bit 30 as well. For `funct3 = 101`, it separates the arithmetic shift `sra`/`srai` from the logical shift `srl`/`srli`. This works for both forms, because in `srai` the immediate's upper bits are a fixed `funct7` pattern, with bit 30 set. For `funct3 = 000` it separates `sub` from `add`, but only for R-type instructions. In `addi`, bit 30 is an ordinary immediate bit, and `addi x1, x1, -1024` has it set, yet must still add. `OpD[5]` is 1 for the R-type opcode `0110011` and 0 for the I-type `0010011`, so `RtypeSubD = Funct7b5D & OpD[5]` selects `sub` only where it exists.

9.10 Immediate extension

The datapath of HIPV is 32 bits wide, but an immediate in an instruction has at most 20 bits, and it is split across the instruction differently in each format. The extension unit `extend` collects the bits of the immediate and extends them to a 32-bit value. The decoder tells it which format to use, with `ImmSrcD`:

```
module extend (input  logic [31:7] InstrD,
               input  logic [2:0]  ImmSrcD,
               output logic [31:0] ImmExtD);
```

```
  import hipv_pkg::*;

  always_comb
    case (ImmSrcD)
      IMM_I:   ImmExtD = {{20{InstrD[31]}}, InstrD[31:20]};
      IMM_S:   ImmExtD = {{20{InstrD[31]}}, InstrD[31:25], InstrD[11:7]};
      IMM_B:   ImmExtD = {{20{InstrD[31]}}, InstrD[7], InstrD[30:25], InstrD[11:8], 1'
      IMM_J:   ImmExtD = {{12{InstrD[31]}}, InstrD[19:12], InstrD[20], InstrD[30:21],
      IMM_U:   ImmExtD = {InstrD[31:12], 12'b0};
      default: ImmExtD = 32'b0;
    endcase

endmodule
```

Each line is a concatenation that follows the format table from the section on instruction fields:

- **I** (arithmetic immediates, loads, `jalr`): the 12 bits 31:20.
- **S** (stores): the same 12-bit immediate, but split in two, because bits 11:7 hold `rd` in other formats and `rs2` takes bits 24:20. The two pieces are joined again.
- **B** (branches): a 13-bit offset whose lowest bit is always 0, because instructions start at even addresses. That bit is not stored; the code appends `1'b0`. The other bits are scattered so that most of them sit where they sit in the S format.
- **J** (`jal`): a 21-bit offset, also with bit 0 appended as 0.
- **U** (`lui`, `auipc`): the upper 20 bits of a 32-bit value. They are placed at the top and the lower 12 bits are filled with zeros, so there is nothing to extend.

All the other immediates are **sign-extended**: they are two's-complement numbers, so `addi x1, x1, -1` and a backward branch work. Sign extension copies the sign bit into all the new upper bits, and the replication operator `{20{InstrD[31]}}` does exactly that. The sign bit is always bit 31 of the instruction, in every format. RISC-V placed it there on purpose: the extension can start replicating the sign before it even knows the format.

The input is `InstrD[31:7]`, because the opcode bits 6:0 never contain any part of an immediate. The scrambled bit order in B and J looks strange in the code, but in hardware it costs nothing: each output bit is a wire from one input bit, chosen by a multiplexer of at most five inputs.

9.11 Register file

The register file holds the 32 registers `x0` to `x31` of RV32I, each 32 bits wide. It has two read ports, because an instruction reads up to two source registers, and one write port, because it writes at most one destination register:

```
module regfile (input  logic        clk,
                input  logic        WE3,
                input  logic [4:0]  A1, A2, A3,
                input  logic [31:0] WD3,
                output logic [31:0] RD1, RD2);

  logic [31:0] RF[31:0];

  // synchronous write port (writes to x0 are ignored)
  always_ff @(posedge clk)
    if (WE3 & (A3 != 5'b0)) RF[A3] <= WD3;

  // combinational read ports with write-first bypass
  assign RD1 = (A1 == 5'b0) ? 32'b0 : (WE3 & (A1 == A3)) ? WD3 : RF[A1];
  assign RD2 = (A2 == 5'b0) ? 32'b0 : (WE3 & (A2 == A3)) ? WD3 : RF[A2];

endmodule
```

In the IDU, the read addresses are the register numbers of the instruction in Decode, and the write port is connected to the Writeback stage:

```
  regfile rf(.clk(clk), .WE3(RegWriteW), .A1(Rs1D), .A2(Rs2D), .A3(RdW),
             .WD3(ResultW), .RD1(RD1D), .RD2(RD2D));
```

9.11.1 Storage: an array of registers

The registers are declared as an **array**:

```
  logic [31:0] RF[31:0];
```

The range before the name, `[31:0]`, is the width of one element: a 32-bit vector, as in every declaration so far. The range after the name is new: it makes 32 such elements, `RF[0]` to `RF[31]`. In hardware, this is 32 registers of 32 flip-flops each, 1024 flip-flops in total. The

array is not reset, so after power-up the registers hold unknown values. Software must write a register before it reads it, as it must on any processor.

The array is indexed with a **signal**, not a constant: `RF[A1]` is the register whose number is the current value of `A1`. This is relative addressing: the code does not name a register, but says "the one that `A1` points to". In hardware there is no addressing logic as in memory; the synthesis tool turns `RF[A1]` into a 32-to-1 multiplexer with all 32 registers as inputs and `A1` as the select. Each read port is one such multiplexer, 32 bits wide.

9.11.2 Writing and reading

Writing is synchronous. The `always_ff` block stores `WD3` in register `A3` at the rising clock edge, if the write enable `WE3` is 1. The extra condition `A3 != 0` protects `x0`, which RISC-V defines as constant zero: an instruction may name `x0` as its destination, but the result is discarded. `addi x0, x0, 0`, the canonical no-op, relies on it.

Reading is asynchronous, that is, combinational: `RD1` and `RD2` follow `A1` and `A2` within the same cycle, with no clock involved. As soon as an instruction enters Decode, its register numbers select the operands, and they are ready by the end of the cycle, in time to be stored in the D/E pipeline register. Each read port checks two special cases before it reads the array:

- If the address is 0, the result is 0. Since `x0` is never written, its array entry would hold an unknown value from power-up; the check makes sure that `x0` reads as zero.
- If the register is being written in this very cycle (`WE3 & (A1 == A3)`), the result is the value being written, `WD3`, rather than the old value in the array. This is called **write-first** behavior.

The write-first bypass solves a timing problem of the pipeline. The register file is written at the end of the cycle, at the clock edge, but it is read during the cycle. Without the bypass, an instruction in Decode that reads a register written by an instruction in Writeback in the same cycle would get the old value:

```
add  x5, x1, x2    # in Writeback: writes x5 at the end of this cycle
sub  x6, x7, x8
or   x9, x6, x0
and  x3, x5, x4    # in Decode: reads x5 in the same cycle
```

The instructions between them are three apart, too far for forwarding in Execute, which reaches only the Memory and Writeback stages. With the bypass, `and` receives the new value of `x5` straight from `ResultW`. Together with forwarding, this covers every distance between a producer and its consumer.

9.12 Summary

The IDU turns the 32 bits of an instruction into everything the later stages need: control signals, a 32-bit immediate, and two register operands. Most of it is combinational logic that works in parallel on the instruction held in the F/D register: field extraction by wiring, the decoder, the extension unit, and the read ports of the register file.

The decoder has two levels: the main decoder maps the opcode to a control word in a single table-like `case`, and the ALU decoder refines the ALU operation from `funct3` and bit 30. All encodings come from the shared package `hipv_pkg`, so Decode and Execute always agree on them.

The IDU is also the first stage that receives instructions over the valid/ready handshake. This mechanism connects every pair of stages in HIPV: a sender offers an item with `valid`, a receiver accepts it with `ready`, and the item moves at the edge where both are high. A valid bit travels with each instruction, a stall propagates backwards through `ready`, and a jump turns the instruction in Decode into a bubble.

10 Integer execution unit

Integer execution unit

10.1 Goal for this week

- Know what the integer execution unit (IEU) does, and which instructions it executes
- Read the IEU interface: the handshake with Decode and Memory, the operands and controls from Decode, the forwarding inputs and the redirect
- Build the IEU from generic blocks: the D/E pipeline registers, the forwarding and operand multiplexers, the ALU and the branch logic
- Understand why the operand registers must be refreshed with forwarded values while the stage waits

10.2 Introduction

The execution stage is the heart of every CPU. It is the place where an instruction actually does its work: two numbers are added, compared or shifted, a memory address is computed, a branch decides where the program goes next. The stages before it only prepare this moment. Fetch brings the instruction, Decode works out what it means and reads its operands. The stages after it only finish the job. Memory performs the access whose address Execute has computed, and Writeback stores the result in the register file.

HIPV implements the base integer instruction set of RISC-V, **RV32I**. It has no multiplication and division (the M extension) and no floating point, so every operation the IEU performs is simple enough to finish in a single clock cycle. The IEU handles three kinds of work:

- **Arithmetic and logic.** The register–register instructions `add`, `sub`, `sll`, `slt`, `sltu`, `xor`, `srl`, `sra`, `or` and `and`, and their register–immediate forms `addi`, `slli`, `slti`, `sltiu`, `xori`, `srli`, `srai`, `ori` and `andi`. The two upper-immediate instructions `lui` and `auipc` are also additions in the ALU.
- **Address calculation.** Loads and stores address memory as `rs1 + imm`. The IEU computes this sum, and the Memory stage uses it.
- **Control flow.** The six conditional branches `beq`, `bne`, `blt`, `bge`, `bltu` and `bgeu`, and the two jumps `jal` and `jalr`. The IEU decides whether the branch is taken, computes the target, and tells Fetch to continue from there.

10.3 Interface of the IEU

The ports of the IEU fall into five groups: the handshake with Decode together with everything Decode has produced, the forwarding inputs, the redirect to Fetch and Decode, and the handshake with the Memory stage together with the stage outputs. As before, the suffix of each name says which stage the signal belongs to.

```
module ieu (input  logic        clk, reset,
            // handshake with the Decode stage
            input  logic        ValidD,
            output logic        ReadyE,
            // inputs from Decode
            input  logic [31:0] RD1D, RD2D, ImmExtD, PCD, PCPlus4D,
            input  logic [4:0]  Rs1D, Rs2D, RdD,
            input  logic [2:0]  Funct3D,
            input  logic        RegWriteD, MemWriteD, BranchD, JumpD, JalrD,
            input  logic [1:0]  ResultSrcD, ALUSrcAD,
            input  logic        ALUSrcBD,
            input  logic [3:0]  ALUControlD,
            // forwarding (from the hazard unit and later stages)
            input  logic [1:0]  ForwardAE, ForwardBE,
            input  logic [31:0] ForwardDataM, ResultW,
            // redirect to Fetch/Decode
            output logic        PCSrcE,
            output logic [31:0] PCTargetE,
            // handshake with the Memory stage
            output logic        ValidE,
            input  logic        ReadyM,
            // stage outputs to Memory
            output logic [31:0] ALUResultE, WriteDataE, PCPlus4E,
            output logic [4:0]  RdE,
            output logic [4:0]  Rs1E, Rs2E,
            output logic [2:0]  Funct3E,
            output logic        RegWriteE, MemWriteE,
            output logic [1:0]  ResultSrcE);
```

10.3.1 Handshake with Decode

The IEU receives instructions from Decode over the same valid/ready handshake that connects Fetch and Decode (Section 9.5). `ValidD` says that Decode offers a decoded instruction,

and `ReadyE` says that Execute can accept it. The instruction moves into Execute at a clock edge where both are high. How `ReadyE` is computed is the subject of Section 10.5.

10.3.2 Signals from Decode

Decode hands the IEU everything the instruction needs, already worked out. The signals fall into five groups:

Group	Signals	Used for
ALU operation	`ALUControlD`, `ALUSrcAD`, `ALUSrcBD`	what the ALU computes, and from which operands
Branches and jumps	`BranchD`, `JumpD`, `JalrD`, `Funct3D`	whether to redirect, and where
Later stages	`MemWriteD`, `ResultSrcD`, `RegWriteD`, `RdD`	not used here, passed on to Memory and Writeback
Operands	`RD1D`, `RD2D`, `ImmExtD`, `PCD`, `PCPlus4D`	the data to compute with
Register numbers	`Rs1D`, `Rs2D`	the forwarding decision in the hazard unit

`Funct3D` serves two purposes. For a branch it selects the condition (`beq`, `blt`, …), and for a load or store it selects the access size, which is why it also travels on to the Memory stage. The control signals for later stages pass through the IEU unchanged: Execute stores them in its pipeline registers and hands them on together with its result.

10.3.3 Forwarding inputs

An operand read from the register file in Decode can be out of date. The instruction that produces it may still be in the Memory or Writeback stage, with its result computed but not yet written to the register file. The IEU therefore receives the two newest results in the pipeline:

- `ForwardDataM`, the result of the instruction in the Memory stage. For a load this is the loaded data, otherwise the ALU result.
- `ResultW`, the result of the instruction in the Writeback stage.

The hazard unit decides which of the values the IEU should use, and tells it with two 2-bit selects, `ForwardAE` for operand A and `ForwardBE` for operand B. For that decision it needs to know which registers the instruction in Execute reads, so the IEU exports `Rs1E` and `Rs2E`.

10.3.4 Redirect to Fetch and Decode

Branches and jumps are resolved here, so the IEU drives the redirect that we met from the receiving side in the IFU chapter: `PCSrcE` says that a jump or a taken branch has occurred, and `PCTargetE` carries the target address. Fetch loads its PC with the target, and Decode throws away the instruction it holds, which was fetched from the wrong path.

10.3.5 Handshake and outputs to Memory

Towards the Memory stage, the IEU is the sender: `ValidE` says that its outputs hold a valid instruction, and `ReadyM` says that the Memory stage can take it. The outputs are the ALU result `ALUResultE`, which is either the result of an arithmetic instruction or the address of a load or store, the store data `WriteDataE`, the return address `PCPlus4E` for `jal` and `jalr`, and the controls that the later stages still need.

10.4 Structure of the IEU

Figure 10.1 shows the IEU. Data flows from left to right through four parts, and the rest of the chapter takes them one at a time:

1. the **handshake logic and the pipeline registers**, which accept the instruction from Decode and hold it while it executes;
2. the **forwarding multiplexers**, which replace an out-of-date register operand with a newer value from the Memory or Writeback stage;
3. the **ALU** with its two operand multiplexers, which computes the result;
4. the **jump mechanism**, which evaluates the branch condition, computes the target and drives the redirect.

10.5 Handshake logic and pipeline registers

The handshake logic is the same as in Decode (Figure 9.3), with one difference: Decode also listens to `PCSrcE` and flushes its instruction on a redirect, while Execute is never flushed. A branch is resolved in Execute, so everything that is wrong about it lies *behind* it, in Fetch and Decode. The branch itself is correct and continues to the Memory stage like any other instruction.

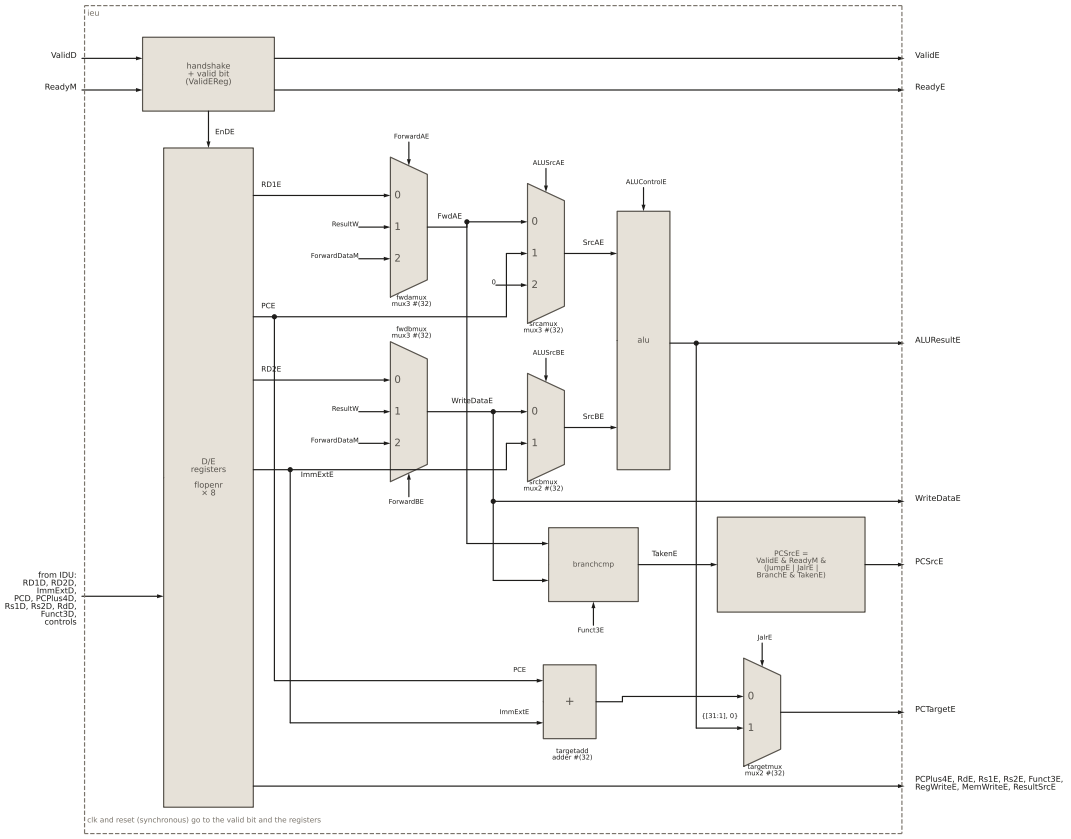


Figure 10.1: The IEU: the handshake logic and the D/E pipeline registers, the forwarding multiplexers (`fwdamux`, `fwdbmux`), the operand multiplexers (`srcamux`, `srcbmux`), the ALU, and the branch logic (`branchcmp`, `targetadd`, `targetmux`) that drives the redirect.

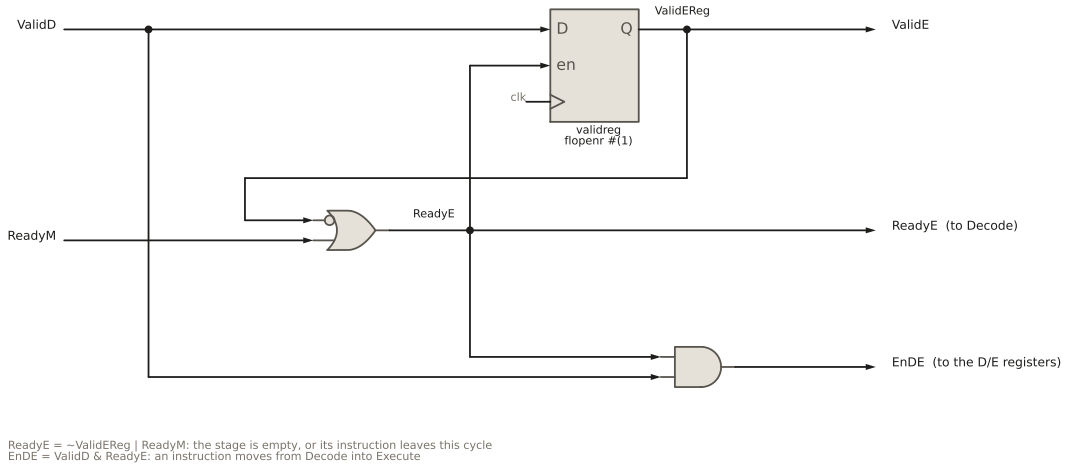


Figure 10.2: The handshake logic of the IEU. `ReadyE` tells Decode that Execute can take a new instruction, `EnDE` loads the D/E registers, and `validreg` remembers whether the stage holds a valid instruction.

```
// D/E handshake: accept when the stage is empty or being drained
assign ReadyE = ~ValidEReg | ReadyM;
assign EnDE   = ValidD & ReadyE;
assign ValidE = ValidEReg;

// D/E valid bit
always_ff @(posedge clk)
  if (reset) ValidEReg <= 1'b0;
  else if (ReadyE) ValidEReg <= ValidD;
```

Three signals make up the handshake (Figure 10.2):

- `ValidEReg` is the **valid bit** of the stage, a one-bit register that says whether the stage holds a valid instruction or a bubble. It is reset to 0, an empty stage. Whenever the stage can accept a new instruction, it takes over `ValidD`: it becomes 1 if Decode delivers an instruction, and 0, a bubble, if Decode has nothing to offer. It is also the stage output `ValidE`.
- `ReadyE` tells Decode that Execute can accept a new instruction. That is the case when the stage is empty (`~ValidEReg`), or when the instruction it holds leaves at this very edge because the Memory stage is ready to take it (`ReadyM`). The second term is what lets the pipeline move one instruction per cycle: the place is freed and refilled at the same edge. When the Memory stage stalls, `ReadyE` drops, and the stall propagates backwards to Decode and Fetch.

- `EnDE` is the transfer itself: Decode offers an instruction (`ValidD`) and Execute accepts it (`ReadyE`). It is the enable of the pipeline registers.

10.5.1 The D/E pipeline registers

The pipeline registers store everything Decode has produced for the instruction, and they are loaded at the transfer, on `EnDE`. Between transfers they hold their value, so the instruction stays in Execute, with all its operands and controls stable, for as long as the Memory stage makes it wait. All of them are the generic `flopenr`, with the width of the signals they store:

```
  flopenr #(32) rd1reg(.clk(clk), .reset(reset), .en(EnDE | HoldE), .D(RD1NextE), .Q(R
  flopenr #(32) rd2reg(.clk(clk), .reset(reset), .en(EnDE | HoldE), .D(RD2NextE), .Q(R
  flopenr #(32) immreg(.clk(clk), .reset(reset), .en(EnDE), .D(ImmExtD), .Q(ImmExtE));
  flopenr #(32) pcreg(.clk(clk), .reset(reset), .en(EnDE), .D(PCD), .Q(PCE));
  flopenr #(32) pcplus4reg(.clk(clk), .reset(reset), .en(EnDE), .D(PCPlus4D), .Q(PCPlu
  flopenr #(15) regsreg(.clk(clk), .reset(reset), .en(EnDE),
                        .D({Rs1D, Rs2D, RdD}), .Q({Rs1E, Rs2E, RdE}));
  flopenr #(3)  funct3reg(.clk(clk), .reset(reset), .en(EnDE), .D(Funct3D), .Q(Funct3E
  flopenr #(14) ctrlreg(.clk(clk), .reset(reset), .en(EnDE),
                        .D({RegWriteD, MemWriteD, BranchD, JumpD, JalrD, ResultSrcD,
                            ALUSrcAD, ALUSrcBD, ALUControlD}),
                        .Q({RegWriteE, MemWriteE, BranchE, JumpE, JalrE, ResultSrcE,
                            ALUSrcAE, ALUSrcBE, ALUControlE}));
```

Signals that belong together share one register: the three register numbers in `regsreg`, and all fourteen control bits in `ctrlreg`. The concatenation `{…}` packs them into one vector on the input and unpacks them on the output. Only the two register operands, in `rd1reg` and `rd2reg`, have a different input and enable. They are the subject of Section 10.6.1.

10.6 Forwarding

The two register operands pass through a forwarding multiplexer each, `fwdamux` for operand A (`rs1`) and `fwdbmux` for operand B (`rs2`). Both are three-input multiplexers, the generic `mux3`:

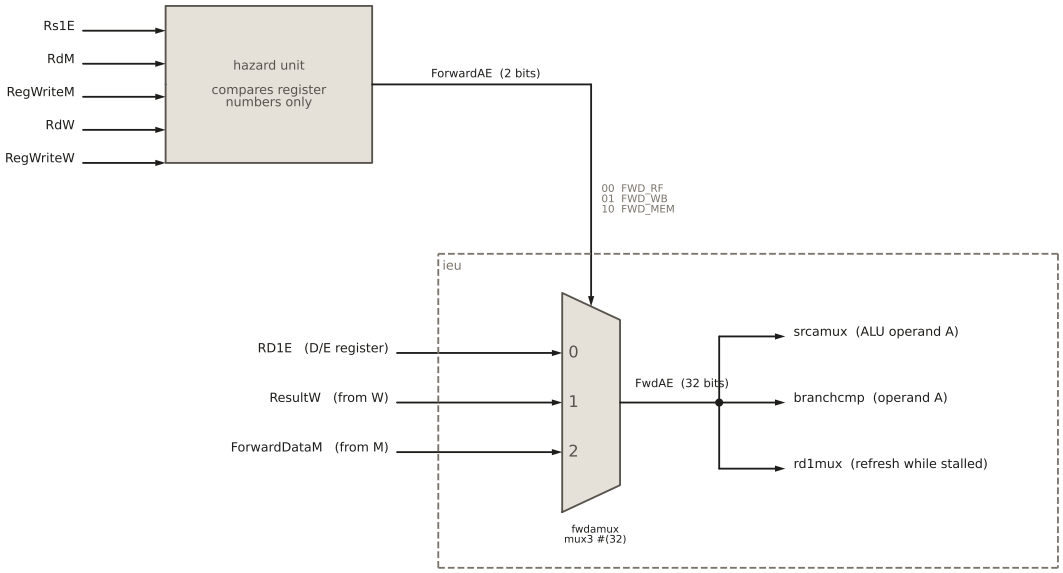


Figure 10.3: Forwarding for operand A. The hazard unit compares the register numbers and drives `ForwardAE`; `fwdamux` in the IEU selects the value, `FwdAE`.

```
module mux3 #(parameter WIDTH = 8) (
  input  logic [WIDTH-1:0] D0, D1, D2,
  input  logic [1:0]       S,
  output logic [WIDTH-1:0] Y);

  assign Y = (S == 2'b10) ? D2 : (S == 2'b01) ? D1 : D0;

endmodule
```

The nested conditional operator checks `S` against the two encodings of `D2` and `D1`, and falls back to `D0` for everything else, including the unused `11`. In the IEU, the two instances choose between three versions of the same register:

```
  // forwarding muxes: register operand / ResultW / ForwardDataM
  mux3 #(32) fwdamux(.D0(RD1E), .D1(ResultW), .D2(ForwardDataM), .S(ForwardAE), .Y(Fwd
  mux3 #(32) fwdbmux(.D0(RD2E), .D1(ResultW), .D2(ForwardDataM), .S(ForwardBE), .Y(Wri
```

- `D0` is the value from the D/E pipeline register, `RD1E` or `RD2E`, as it was read from the register file in Decode;
- `D1` is `ResultW`, the result of the instruction in the Writeback stage;
- `D2` is `ForwardDataM`, the result of the instruction in the Memory stage.

The selects come from the hazard unit (Figure 10.3). Their three values are named in the package `hipv_pkg`, so that the code says what a select means rather than which bits it has:

```
  // forwarding select (ForwardAE / ForwardBE, from the hazard unit)
  localparam logic [1:0] FWD_RF  = 2'b00;  // register operand from D/E register
  localparam logic [1:0] FWD_WB  = 2'b01;  // ResultW from the Writeback stage
  localparam logic [1:0] FWD_MEM = 2'b10;  // ForwardDataM from the Memory stage
```

The hazard unit selects `FWD_MEM` when the instruction in the Memory stage writes the register the operand reads, `FWD_WB` when the instruction in Writeback does, and `FWD_RF` otherwise. If both match, the Memory stage wins, because it holds the younger instruction and therefore the newer value. The details of that decision belong to the chapter on hazards. The IEU only carries it out.

The outputs of the two multiplexers are the true values of `rs1` and `rs2`, and every later block in the IEU uses them instead of `RD1E` and `RD2E`. `FwdAE` goes to the ALU operand multiplexer and to the branch comparator. `WriteDataE` goes to the same two places, and on to the Memory stage as the store data, which is where its name comes from.

10.6.1 Refreshing the operands with forwarded values

Forwarding as described so far has a gap. An instruction does not always spend just one cycle in Execute. When the Memory stage is busy with a slow load or store, `ReadyM` is low, and the instruction in Execute has to wait. The forwarded value, however, is available only as long as its producer is in the Memory or Writeback stage. If the producer leaves Writeback while the consumer is still waiting, nothing forwards the value any more. The operand in the pipeline register does not help either: it was read from the register file in Decode, *before* the producer wrote it, so it is stale.

Consider this sequence on a slow data memory:

```
lw   x5, 0(x10)    # slow memory: the next instructions bunch up
addi x1, x0, 77    # producer of x1
sw   x5, 4(x10)    # holds the Memory stage
add  x3, x1, x0   # waits in Execute; x1 forwarded from W for one cycle only
```

When `addi` reaches Writeback, `sw` is in the Memory stage and `add` in Execute. The hazard unit sees that `add` reads `x1` and `addi` writes it, and selects `FWD_WB`. But `sw` holds the Memory stage for several cycles, so `add` cannot leave Execute. In the next cycle `addi` has left the pipeline, the hazard unit falls back to `FWD_RF`, and `fwdamux` delivers `RD1E`, the old value of `x1` read in Decode. When `add` finally executes, it computes with the wrong operand.

The solution is to not let the forwarded value go: as long as the instruction waits, the operand register is reloaded with the output of the forwarding multiplexer. This is a feedback loop from `fwdamux` back to the input of `rd1reg`, and the same from `fwdbmux` to `rd2reg`.

10.6.2 The refresh loop in SystemVerilog

```
// While an instruction waits in E, refresh its register operands with
// the forwarded values: the producer can retire from W (and stop being
// forwarded) before this instruction leaves E.
assign HoldE = ValidEReg & ~ReadyM;
mux2 #(32) rd1mux(.D0(FwdAE), .D1(RD1D), .S(EnDE), .Y(RD1NextE));
mux2 #(32) rd2mux(.D0(WriteDataE), .D1(RD2D), .S(EnDE), .Y(RD2NextE));
flopenr #(32) rd1reg(.clk(clk), .reset(reset), .en(EnDE | HoldE), .D(RD1NextE), .Q(R
flopenr #(32) rd2reg(.clk(clk), .reset(reset), .en(EnDE | HoldE), .D(RD2NextE), .Q(R
```

This code describes the datapath in Figure 10.4, shown for operand A. Operand B is built the same way, with `rd2mux`, `rd2reg` and `fwdbmux`.

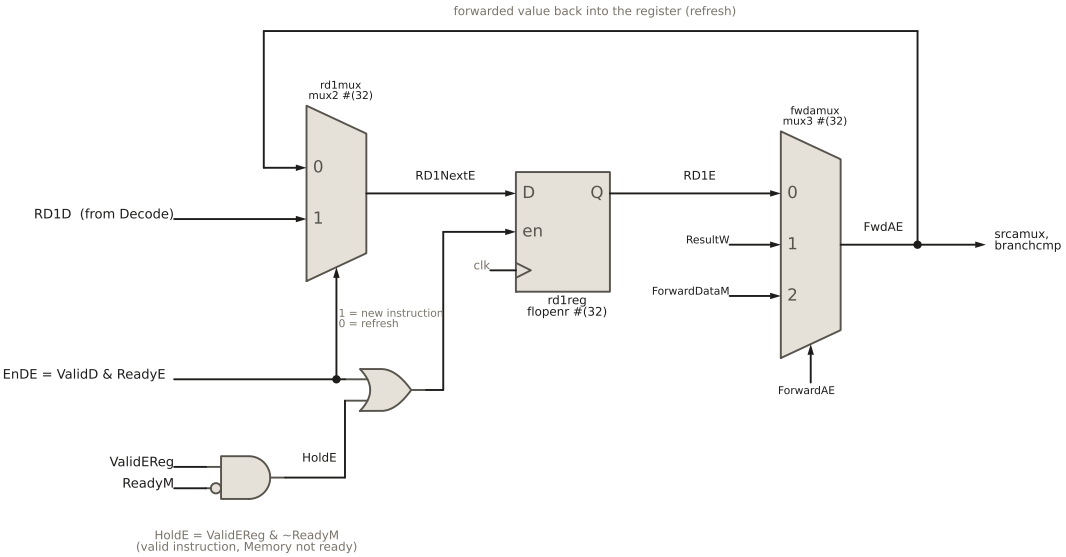


Figure 10.4: The refresh loop for operand A. `rd1mux` chooses between the operand from Decode and the forwarded value `FwdAE`; `rd1reg` is loaded on a transfer (`EnDE`) and in every cycle the instruction waits (`HoldE`).

The datapath has three parts:

- **`rd1mux`** chooses what will be stored in `rd1reg`.
  - When `EnDE = 1`, a new instruction enters Execute, and the register stores its operand from Decode, `RD1D`.
  - When `EnDE = 0`, no instruction enters, and the register stores the output of the forwarding multiplexer, `FwdAE`.
- **`rd1reg`** stores the operand. Its enable is `EnDE | HoldE`, so it loads in two situations: at the transfer from Decode, and in every cycle in which the stage is on hold. `HoldE = ValidEReg & ~ReadyM` says that the stage holds a valid instruction, but the Memory stage is not ready to take it, so the instruction has to wait.
- **`fwdamux`** is the same forwarding multiplexer as before. Through the loop it also decides what `rd1reg` stores while the stage is on hold: `ForwardDataM`, `ResultW`, or, with `FWD_RF`, the register's own value, which simply keeps it.

`HoldE` and `EnDE` are never high at the same time. Substituting `ReadyE` shows that `HoldE` is exactly its complement, `ValidEReg & ~ReadyM = ~(~ValidEReg | ReadyM) = ~ReadyE`: the stage is on hold precisely when it cannot accept a new instruction, and then `EnDE` is 0. The register therefore never has to choose between a new operand and a refresh, and `EnDE` alone is a sufficient select for `rd1mux`. When both are 0, the stage is empty, or its instruction is leaving without a successor, and the register keeps a value that nobody uses.

Back to the example. In the cycle in which `addi` is in Writeback, `add` is on hold, and `fwdamux` delivers `ResultW = 77`. At the end of the cycle `rd1reg` stores 77. In the next cycle `addi` is gone and the hazard unit selects `FWD_RF`, but `RD1E` now *is* 77, and the loop keeps it there until `add` leaves Execute.

The refresh is harmless when the forwarded value is not ready yet. If the producer is a load still waiting for its data, `ForwardDataM` carries garbage, and `rd1reg` stores it. But while the load is in the Memory stage, `ReadyM` is low, so the consumer is on hold and does not use the garbage. When the data is there, `ReadyM` rises, `ForwardDataM` carries the loaded value, and the consumer leaves Execute computing with it, straight from `fwdamux`. This is why HIPV needs no special load-use stall: the elastic handshake and the refresh loop together cover that case.

10.7 The ALU

The arithmetic-logic unit (ALU) is the workhorse of every CPU. It is a purely combinational block: it takes two operands, computes one of a fixed set of operations on them, and produces a result, all within one clock cycle and without any state of its own. Which operation it computes is chosen by a control input. Most instructions of a program pass through it, not

only the arithmetic ones. A load or store uses it for the address, and `jalr` for the jump target.

The ALU of HIPV has two 32-bit operands, `A` and `B`, a 4-bit control `ALUControl`, and a 32-bit result `Y`. It implements the ten operations that RV32I needs. Inside, it is simply a separate circuit for each operation (an adder, a subtractor, a shifter, a comparator, the bitwise gates), followed by a large multiplexer that selects one result.

10.7.1 Operands of the ALU

The operands of the ALU come from two multiplexers, one per operand:

```
// ALU operand selection: A = rs1/PC/zero, B = rs2/immediate
mux3 #(32) srcamux(.D0(FwdAE), .D1(PCE), .D2(32'b0), .S(ALUSrcAE), .Y(SrcAE));
mux2 #(32) srcbmux(.D0(WriteDataE), .D1(ImmExtE), .S(ALUSrcBE), .Y(SrcBE));
```

Operand A, `SrcAE`, is selected by `ALUSrcAE` from three sources:

`ALUSrcAE`	`SrcAE`	Used by	Example
`00` `SRCA_RF`	`FwdAE`, the forwarded `rs1`	almost all instructions	`add x3, x1, x2`; `lw x3, 8(x1)`
`01` `SRCA_PC`	`PCE`, the address of the instruction	`auipc`	`auipc x5, 0x12` gives `x5 = PC + 0x12000`
`10` `SRCA_ZERO`	0	`lui`	`lui x5, 0x12` gives `x5 = 0 + 0x12000`

Selecting zero for `lui` is a small trick. `lui` only has to copy its immediate into the destination register, and adding it to zero does that with the ALU's ordinary `ADD`, without a separate path to the output.

Operand B, `SrcBE`, is selected by `ALUSrcBE` from two sources:

`ALUSrcBE`	`SrcBE`	Used by	Example
0	`WriteDataE`, the forwarded `rs2`	register–register instructions	`add x3, x1, x2`

ALUSrcBE	SrcBE	Used by	Example
1	ImmExtE, the extended immediate	register–immediate, loads, stores, lui, auipc, jalr	addi x3, x1, 5; sw x3, 8(x1)

Both multiplexers come after the forwarding multiplexers, so whenever the ALU uses a register, it uses its forwarded value.

10.7.2 The ALU in SystemVerilog

The ALU is one of the few behavioral blocks of the core. It is a single case over ALUControl, inside always_comb:

```
module alu (input  logic [31:0] A, B,
            input  logic [3:0]  ALUControl,
            output logic [31:0] Y);

  import hipv_pkg::*;

  always_comb
    case (ALUControl)
      ALU_ADD:  Y = A + B;
      ALU_SUB:  Y = A - B;
      ALU_SLL:  Y = A << B[4:0];
      ALU_SLT:  Y = {31'b0, $signed(A) < $signed(B)};
      ALU_SLTU: Y = {31'b0, A < B};
      ALU_XOR:  Y = A ^ B;
      ALU_SRL:  Y = A >> B[4:0];
      ALU_SRA:  Y = $unsigned($signed(A) >>> B[4:0]);
      ALU_OR:   Y = A | B;
      ALU_AND:  Y = A & B;
      default:  Y = 32'b0;
    endcase

endmodule
```

The operation codes ALU_ADD to ALU_AND are the constants 0 to 9 from hipv_pkg, imported at the top of the module. Decode produces the same names, so the two ends always agree on the encoding. A few lines deserve a closer look:

- **Shifts** use only the lowest five bits of `B`, because a 32-bit value can be shifted by at most 31 places. This also makes the immediate shifts work: in `srai`, bit 10 of the immediate distinguishes it from `srli`, but it lies outside `B[4:0]` and does not disturb the shift amount.
- **Signed and unsigned.** SystemVerilog treats a `logic` vector as unsigned. `slt` and `sltu` compare the same bits, but `slt` first reinterprets them with `$signed`, so that `0xFFFFFFFF` counts as −1 and not as 4 294 967 295. The comparison gives one bit, and `{31'b0, …}` extends it to the 32-bit result 0 or 1.
- **Arithmetic shift.** `>>>` fills the vacated bits with copies of the sign bit only if its operand is signed, so `A` is converted with `$signed`, and the result is converted back with `$unsigned` before it is assigned to `Y`.
- **`default`** covers the six unused codes 10 to 15. Without it, `Y` would not be assigned for them, and `always_comb` would infer a latch (see the chapter on designing with SystemVerilog).

In the IEU, the ALU is a single instance, and its result is the stage output `ALUResultE`:

```
  alu alu(.A(SrcAE), .B(SrcBE), .ALUControl(ALUControlE), .Y(ALUResultE));
```

You have built this block before. In the second lab exercise you implemented a “dummy” 4-bit ALU with a 3-bit `alu_control`, using operators, intermediate signals and nested conditionals. The ALU of HIPV is the same idea at full size. The operands are 32 bits instead of 4, the control selects ten operations instead of eight, and the selection is written as a `case` in `always_comb` instead of a chain of `? :`. The two describe the same multiplexer, but the `case` stays readable with ten branches. The lab operations also map onto the core:

Lab ALU (`alu_control`)	In HIPV
addition, subtraction	`ALU_ADD`, `ALU_SUB`
bitwise AND, OR, XOR	`ALU_AND`, `ALU_OR`, `ALU_XOR`
compare (equal)	`beq`/`bne` in `branchcmp`, not in the ALU
pass-through `b`	`lui`: `0 + imm`, through `ALU_ADD` with `SRCA_ZERO`
pass-through `a`	not needed

New in HIPV are the shifts and the signed/unsigned comparisons `slt` and `sltu`, and these are where the typical mistakes lie. The IEU testbench in the HIPV repository, `tb/unit/ieu/test_ieu.py`, checks exactly these. Its first test, `test_alu_ops`, runs all ten operations, including a signed and an unsigned compare on the same operands and an arithmetic shift of a negative value. Run it with `make` in `tb/unit/ieu`.

10.8 Jump mechanism

The last part of the IEU decides whether the program continues sequentially or jumps. It answers three questions, each with its own block: is the branch condition true (`branchcmp`), where does the jump go (`targetadd` and `targetmux`), and should Fetch be redirected now (`PCSrcE`).

10.8.1 Conditional branches: `branchcmp`

The branch comparator evaluates the condition of the six RV32I branches, selected by `Funct3E`:

```
module branchcmp (input  logic [31:0] A, B,
                  input  logic [2:0]  Funct3E,
                  output logic        TakenE);

  always_comb
    case (Funct3E)
      3'b000:  TakenE = (A == B);                  // beq
      3'b001:  TakenE = (A != B);                  // bne
      3'b100:  TakenE = ($signed(A) < $signed(B));   // blt
      3'b101:  TakenE = ($signed(A) >= $signed(B));  // bge
      3'b110:  TakenE = (A < B);                   // bltu
      3'b111:  TakenE = (A >= B);                  // bgeu
      default: TakenE = 1'b0;                      // reserved encodings
    endcase

endmodule
```

It is connected to the forwarded register operands, not to the ALU operands:

```
  branchcmp bcmp(.A(FwdAE), .B(WriteDataE), .Funct3E(Funct3E), .TakenE(TakenE));
```

A branch compares two registers, `rs1` and `rs2`, so it needs the same forwarded values as any other instruction. Many processors compare with the ALU, by subtracting and checking the result. HIPV uses a separate comparator, which costs a little more hardware, but keeps the ALU free and the two jobs independent. `TakenE` is meaningful only for branches. For every other instruction it is computed from whatever `Funct3E` and the operands happen to be, and ignored, because `BranchE` is 0.

10.8.2 Jump targets: `targetadd` and `targetmux`

RV32I has two kinds of targets. Branches and `jal` are **PC-relative**: the target is the address of the instruction plus the immediate. `jalr` is **register-relative**: the target is `rs1 + imm`, which is how a function returns (`jalr x0, 0(x1)`, written `ret`) and how a program jumps through a table of addresses.

```
// branch/jump target: PC + imm for branch/jal, ALU (rs1 + imm) for jalr
adder #(32) targetadd(.A(PCE), .B(ImmExtE), .Y(PCTargetBaseE));
assign JalrTargetE = {ALUResultE[31:1], 1'b0};
mux2 #(32) targetmux(.D0(PCTargetBaseE), .D1(JalrTargetE), .S(JalrE), .Y(PCTargetE));
```

The PC-relative target has its own adder, `targetadd`, the same generic `adder` that increments the PC in the IFU. The register-relative target needs no new hardware: for `jalr`, Decode sets up the ALU to compute `rs1 + imm`, as for a load. The ISA requires the lowest bit of the `jalr` target to be cleared, which costs nothing in hardware, only wiring: `{ALUResultE[31:1], 1'b0}`. The multiplexer `targetmux` then selects the ALU path for `jalr` and the adder for everything else.

The ALU result of a `jal` or `jalr` is not what the instruction writes to its destination register. Both write the return address, `PC + 4`, which is why `PCPlus4E` travels on to Writeback with them.

10.8.3 The redirect: `PCSrcE`

The last line of the IEU decides whether to redirect:

```
// redirect, qualified with the valid bit and gated to the retire cycle
assign PCSrcE = ValidEReg & ReadyM & (JumpE | JalrE | (BranchE & TakenE));
```

The term in parentheses is the RV32I rule: a jump (`jal`, `jalr`) always changes the PC, a branch only if its condition is true. The two qualifiers in front make it safe in the elastic pipeline:

- **`ValidEReg`**: a bubble must never redirect. When the stage is empty, its pipeline registers still hold the controls of the last instruction, and without the valid bit an old branch could fire a second time.
- **`ReadyM`**: the redirect fires only in the cycle in which the instruction leaves Execute. This has two reasons. First, it makes `PCSrcE` a one-cycle pulse, which is what the IFU expects. Second, while the instruction waits, its operands may not be final. A branch that depends on a load still in the Memory stage compares garbage until the

data arrives. The refresh loop will correct its operands, but a redirect issued from the garbage could not be taken back. Waiting for `ReadyM` means the branch decides only with its final operands.

Note what does *not* happen: the branch itself is not flushed. It leaves Execute normally at the same edge, and the redirect removes only the instructions behind it, in Fetch and Decode.

10.9 Summary

The IEU executes every RV32I instruction in a single cycle. Like the IFU, most of it is structural, built from `flopenr`, `mux2`, `mux3` and `adder`. Only the two leaf cells that compute, the ALU and the branch comparator, are behavioral.

Its four parts follow the flow of an instruction. The handshake logic and the D/E pipeline registers accept the instruction from Decode and hold it while it waits. The forwarding multiplexers replace out-of-date register operands with newer values from the Memory and Writeback stages, and the refresh loop keeps those values in the operand registers when the stage stalls, so that a forwarded value is never lost. The ALU computes the result, or the address of a load or store. The jump mechanism compares the branch operands, computes the target, and redirects Fetch exactly once, in the cycle a taken branch or jump leaves Execute.

11 Load/store unit

Load/store unit

11.1 Goal for this week

- Know what the load/store unit (LSU) does, and how it differs from the stages before it
- Read the LSU interface, in particular the OBI signals for writing to memory
- Understand how the stage holds a memory instruction until its transaction is finished
- Design the LSU controller as an FSM, and process the data of byte and halfword accesses

11.2 Introduction

The load/store unit is the fourth stage of the HIPV pipeline, the Memory stage. Its design closely resembles the stages we have seen so far: it has the same valid/ready handshake, the same kind of pipeline registers, and, like the IFU, an FSM that talks to a memory. What is new is mostly what it does with the memory.

The role of the LSU is to give the core access to the data memory, for the load and store instructions. That gives it four properties:

- It communicates with the data memory through the **OBI** interface, the same protocol the IFU uses for the instruction memory.
- Unlike the IFU, which only reads instructions, the LSU also **writes** to memory, for the store instructions, alongside **reading** for the loads.
- It does **not** need to access memory all the time. Only loads and stores use the memory. Every other instruction, an `add` or a `jal`, just passes through the stage to Writeback, in a single cycle.
- It **forwards** its result back to the Execute stage, so that the next instruction can use it before it reaches the register file.

11.3 Interface of the LSU

The ports of the LSU fall into six groups:

```
module lsu (input  logic        clk, reset,
            // handshake with the Execute stage
            input  logic        ValidE,
            output logic        ReadyM,
            // inputs from Execute
            input  logic [31:0] ALUResultE, WriteDataE, PCPlus4E,
            input  logic [4:0]  RdE,
            input  logic [2:0]  Funct3E,
            input  logic        RegWriteE, MemWriteE,
            input  logic [1:0]  ResultSrcE,
            // handshake with the Writeback stage
            output logic        ValidM,
            input  logic        ReadyW,
            // stage outputs to Writeback
            output logic [31:0] ALUResultM, ReadDataM, PCPlus4M,
            output logic [4:0]  RdM,
            output logic        RegWriteM,
            output logic [1:0]  ResultSrcM,
            // forwarding to Execute
            output logic [31:0] ForwardDataM,
            // OBI master port to data memory
            output logic        ObiReqM,
            input  logic        ObiGntM,
            output logic [31:0] ObiAddrM,
            output logic        ObiWeM,
            output logic [3:0]  ObiBeM,
            output logic [31:0] ObiWdataM,
            input  logic        ObiRvalidM,
            input  logic [31:0] ObiRdataM);
```

- **Handshake with Execute**: `ValidE` and `ReadyM`.
- **Inputs from Execute**: `ALUResultE`, which is the address for a load or store and the result for any other instruction; `WriteDataE`, the data a store writes; `PCPlus4E`, the return address of `jal` and `jalr`; the destination register `RdE`; `Funct3E`, which gives the size of the access; and the controls `RegWriteE`, `MemWriteE` and `ResultSrcE`.
- **Handshake with Writeback**: `ValidM` and `ReadyW`.
- **Outputs to Writeback**: the three candidate results (`ALUResultM`, the loaded data `ReadDataM`, and `PCPlus4M`), the destination register `RdM`, the write enable

`RegWriteM`, and `ResultSrcM`, which tells Writeback which of the three results to write.

- **Forwarding to Execute**: `ForwardDataM`, the result of the instruction in this stage.
- **OBI port to the data memory**, described in Section 11.7.

11.4 Structure of the LSU

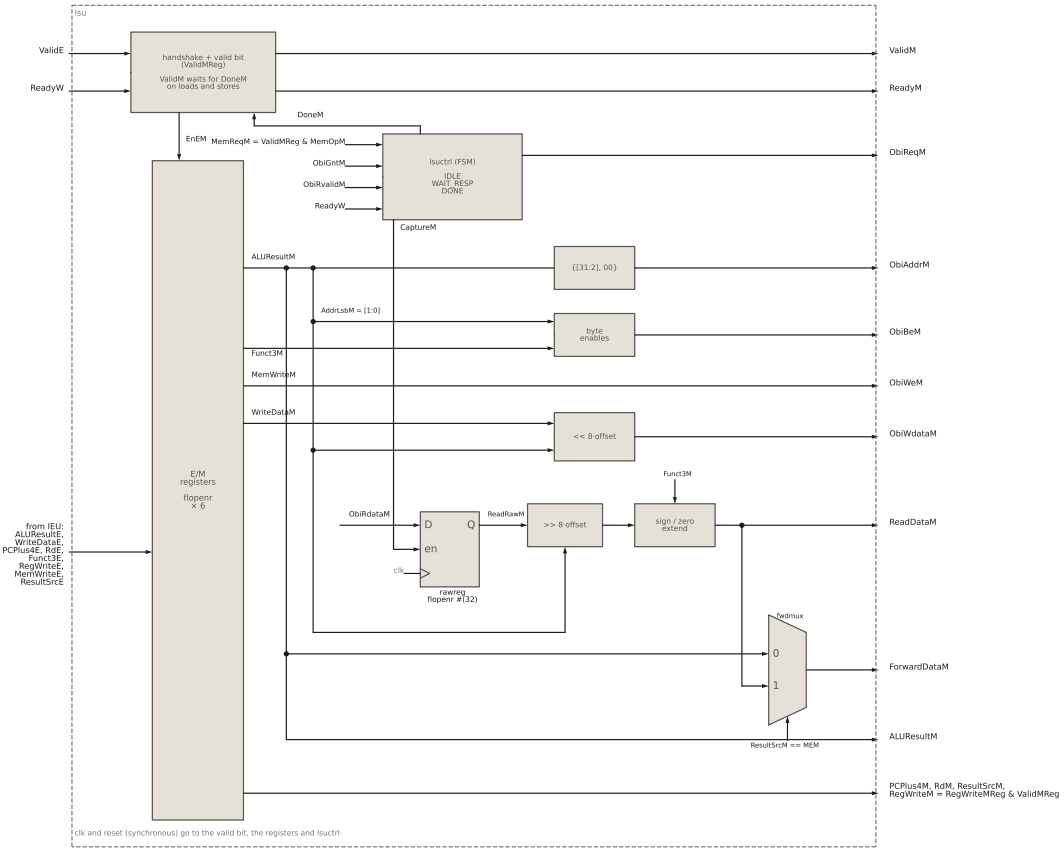


Figure 11.1: The LSU: the handshake logic and valid bit, the E/M pipeline registers, the controller `lsuctrl`, the OBI address, byte enables and store data, the read path (`rawreg`, shift, sign/zero extension), and the forwarding multiplexer.

Figure 11.1 shows the LSU. It has six parts, and the rest of the chapter takes them in this order:

1. the **handshake logic**, which accepts instructions from Execute and, for a memory instruction, holds it in the stage until the memory access is finished;

2. the **E/M pipeline registers**;
3. the **FSM** `lsuctrl`, which runs the OBI transaction;
4. the logic that generates the **OBI signals**: the word address, the byte enables and the store data;
5. the **post-processing of loaded data**: the register `rawreg`, which captures the word from memory, followed by a shift and a sign or zero extension;
6. the **forwarding** of the result to Execute, and the outputs to Writeback.

11.5 Handshake logic

The handshake logic of the LSU has the same shape as in Decode and Execute, with one addition: a memory instruction cannot be handed on until its memory access is finished.

```
// E/M handshake: a memory instruction holds the stage until done
assign MemOpM = MemWriteM | (ResultSrcM == RESULT_MEM);
assign ValidM = ValidMReg & (~MemOpM | DoneM);
assign ReadyM = ~ValidMReg | (ValidM & ReadyW);
assign EnEM   = ValidE & ReadyM;

// E/M valid bit
always_ff @(posedge clk)
  if (reset) ValidMReg <= 1'b0;
  else if (ReadyM) ValidMReg <= ValidE;
```

`EnEM` is the handshake between Execute and the LSU: Execute offers an instruction (`ValidE`) and the LSU accepts it (`ReadyM`). As in the other stages, it enables the pipeline registers, so it is the moment the instruction enters the Memory stage.

11.5.1 When is the output valid?

In Decode and Execute, the output was valid as soon as the stage held a valid instruction. The LSU adds a condition:

```
assign ValidM = ValidMReg & (~MemOpM | DoneM);
```

The output is valid when the stage holds a valid instruction, **and** either it is not a memory instruction, or its memory transaction is done. The three terms:

- **`ValidMReg`** is the valid bit of the stage. It says that the stage holds a valid instruction and not a bubble.
- **`~MemOpM`** says that the instruction is neither a load nor a store. Such an instruction has nothing to do here, so it is handed on to Writeback in the same cycle, without waiting for anything.
- **`DoneM`** says that the memory transaction of a load or store is finished. It is generated by the FSM (Section 11.7.1). Until it is 1, a memory instruction is not offered to Writeback.

How does the LSU know that it holds a memory instruction? From two controls that Decode generated, and that have travelled with the instruction through the pipeline registers:

```
  assign MemOpM = MemWriteM | (ResultSrcM == RESULT_MEM);
```

- `MemWriteM`, the copy of `MemWriteE` in the E/M register, is 1 for a store.
- `ResultSrcM` says which value Writeback will write into the register file. `RESULT_MEM` means that this value is the data read from memory, and only a load writes that. So `ResultSrcM == RESULT_MEM` identifies a load.

No separate "is a load" signal is needed: the two controls the later stages need anyway already carry the information.

> **i** Reminder: why a valid *register*?
>
> `ValidMReg` is a copy of `ValidE` delayed by one edge, loaded whenever the stage can accept something (`ReadyM`). It must be a register, because the valid bit has to move through the pipeline at the same pace as the instruction it labels. After the edge, the instruction from Execute sits in the E/M registers, and `ValidE` already describes the next one. Storing the valid bit at the same edge, next to the data, keeps the label with its instruction. The same holds for `ValidDReg` in Decode and `ValidEReg` in Execute.

11.5.2 When can the stage accept?

`ReadyM` tells Execute that the LSU can accept a new instruction:

```
  assign ReadyM = ~ValidMReg | (ValidM & ReadyW);
```

There are two cases:

- **`~ValidMReg`**: the stage holds a bubble, so there is nothing to lose.

- **`ValidM & ReadyW`**: the instruction in the stage is drained, handed over to Writeback at this very edge, so its place is free for the next one.

Compare this with Execute, where the second term was just `ReadyM`. Here it includes `ValidM`, because a memory instruction that is still waiting for its transaction cannot leave, even if Writeback is ready. While a load waits for memory, `ValidM` is 0, so `ReadyM` is 0, and the stall propagates backwards: Execute holds its instruction, Decode holds its, and Fetch stops. This is the back-pressure that makes a separate load-use stall unnecessary (see the refresh loop in the IEU chapter).

11.6 E/M pipeline registers

The pipeline registers store what Execute has produced, and they are loaded on `EnEM`. While a memory instruction waits for its transaction, `EnEM` is 0, and they hold the address, the store data and the controls stable for as long as the transaction takes:

```
// E/M pipeline registers
flopenr #(32) alureg(.clk(clk), .reset(reset), .en(EnEM), .D(ALUResultE), .Q(ALUResu
flopenr #(32) wdreg(.clk(clk), .reset(reset), .en(EnEM), .D(WriteDataE), .Q(WriteDat
flopenr #(32) pcplus4reg(.clk(clk), .reset(reset), .en(EnEM), .D(PCPlus4E), .Q(PCPlu
flopenr #(5)  rdreg(.clk(clk), .reset(reset), .en(EnEM), .D(RdE), .Q(RdM));
flopenr #(3)  funct3reg(.clk(clk), .reset(reset), .en(EnEM), .D(Funct3E), .Q(Funct3M
flopenr #(4)  ctrlreg(.clk(clk), .reset(reset), .en(EnEM),
                      .D({RegWriteE, MemWriteE, ResultSrcE}),
                      .Q({RegWriteMReg, MemWriteM, ResultSrcM}));
```

The register for `RegWriteE` drives `RegWriteMReg`, not `RegWriteM` directly. Before it leaves the stage, it is qualified with the valid bit (Section 11.10).

11.7 Access to memory

Like the IFU, the LSU talks to its memory over OBI. The two phases of a transaction are the same as on the instruction port (Figure 8.2): in the **address phase**, the LSU holds `ObiReqM` and the request signals until the memory grants with `ObiGntM`; in the **response phase**, at least one cycle later, the memory raises `ObiRvalidM` to say that the access is complete.

What the IFU never needed is writing. A store must tell the memory three more things: *that* it writes, *which bytes* it writes, and *what* it writes. That adds three signals to the address phase:

Signal	Dir	Meaning
`ObiReqM`	out	request: the LSU starts a transaction
`ObiGntM`	in	grant: the memory has accepted the request
`ObiAddrM`	out	address of the 32-bit word to access
`ObiWeM`	out	**write enable**: 1 for a write (store), 0 for a read (load)
`ObiBeM`	out	**byte enables**, 4 bits: which bytes of the word take part
`ObiWdataM`	out	**write data**, each byte already in its position in the word
`ObiRvalidM`	in	response valid: the transaction is complete
`ObiRdataM`	in	read data, meaningful only for a read, while `ObiRvalidM` is 1

- **`ObiWeM`** selects the direction of the transfer. It is simply `MemWriteM`, the store flag from Decode.
- **`ObiBeM`** exists because a store does not always write a whole word. `sb` writes one byte and `sh` two, and the other bytes of the word must not change. Bit *i* of `ObiBeM` enables byte *i* of the word, so `sb` to byte 2 sends `0100`, and the memory leaves bytes 0, 1 and 3 untouched.
- **`ObiWdataM`** carries the data, but the memory takes byte *i* from bits `8i+7 : 8i` of the bus. A byte stored at offset 2 must therefore travel on bits 23:16, not 7:0, and the LSU shifts it into place (Section 11.8).

A store gets a response too. `ObiRvalidM` still comes, one or more cycles after the grant, and it tells the LSU that the write is complete. `ObiRdataM` is meaningless in that case and is ignored. Both loads and stores therefore follow the same sequence: request, grant, response.

11.7.1 Why an FSM?

A transaction lasts several cycles, and how many depends on the memory: the grant may come at once or after a few wait cycles, and the response may follow one cycle or many cycles later. The LSU must remember where the transaction is: has the request been granted,

has the response arrived, has the instruction been handed on? That memory is the state of an FSM, the controller `lsuctrl`.

It is the counterpart of `ifuctrl` in the IFU, and it has the same three states. It is simpler, though, because nothing can interrupt it. A jump discards instructions in Fetch and Decode, but an instruction that has reached the Memory stage is older than any jump that is still being resolved, so it always completes. The LSU needs no discard flag.

11.7.2 Interface of the controller

```
module lsuctrl (input  logic clk, reset,
                input  logic MemReqM,               // valid load/store in M
                input  logic ObiGntM, ObiRvalidM,   // From OBI memory
                input  logic ReadyW,                // From Writeback
                output logic ObiReqM, CaptureM, DoneM);
```

The controller watches three sources and drives three outputs:

- **`MemReqM`** says that there is work to do: a valid instruction in the stage that is a load or store. The LSU computes it as `MemReqM = ValidMReg & MemOpM`, so a bubble or a non-memory instruction never starts a transaction.
- **`ObiGntM`** and **`ObiRvalidM`** come from the memory and mark the end of the address and the response phase.
- **`ReadyW`** says that Writeback takes the finished instruction.
- **`ObiReqM`** is the OBI request.
- **`CaptureM`** loads the read data into the register `rawreg`, in the cycle in which it is valid on the bus.
- **`DoneM`** says that the transaction is finished. It opens `ValidM` in the handshake logic.

In the LSU it is instantiated with `MemReqM` computed next to it:

```
  // OBI transaction control FSM: one outstanding request per instruction
  assign MemReqM = ValidMReg & MemOpM;
  lsuctrl ctrl(.clk(clk), .reset(reset), .MemReqM(MemReqM),
               .ObiGntM(ObiGntM), .ObiRvalidM(ObiRvalidM), .ReadyW(ReadyW),
               .ObiReqM(ObiReqM), .CaptureM(CaptureM), .DoneM(DoneM));
```

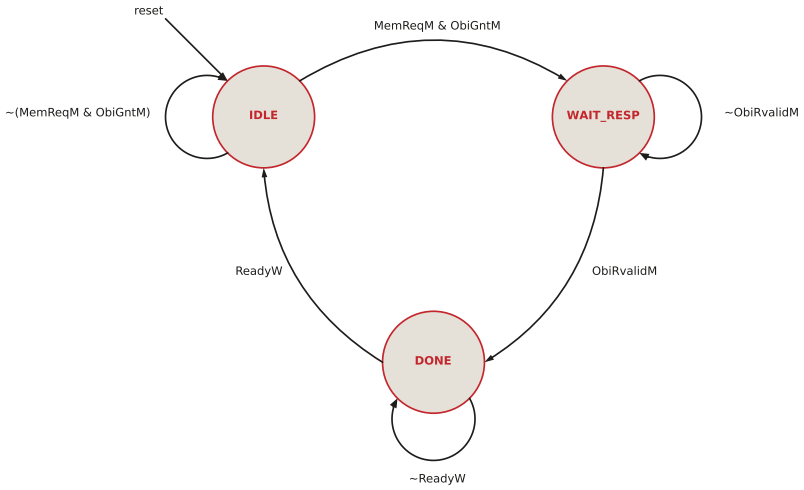


Figure 11.2: The LSU controller as a state diagram, with the transitions only.

11.7.3 States and transitions

Figure 11.2 shows the three states:

- **`IDLE`**: no transaction is in progress. When a load or store is in the stage, the controller requests the memory and waits for the grant. When `MemReqM` and `ObiGntM` are both 1, the memory has accepted the request and the FSM moves to `WAIT_RESP`. A non-memory instruction never leaves `IDLE`: `MemReqM` is 0, and it passes through without the controller.
- **`WAIT_RESP`**: the request has been granted, and the controller waits for the response. When `ObiRvalidM` arrives, the access is complete, and the FSM moves to `DONE`.
- **`DONE`**: the transaction is finished, and the instruction is offered to Writeback. When Writeback takes it (`ReadyW`), the FSM returns to `IDLE`, ready for the next instruction.

In SystemVerilog the FSM follows the familiar pattern, a state register and a next-state `case`:

```
typedef enum logic [1:0] {IDLE, WAIT_RESP, DONE} statetype;
statetype State, NextState;

// state register
always_ff @(posedge clk)
  if (reset) State <= IDLE;
  else       State <= NextState;
```

```
// next state logic
always_comb begin
  NextState = State;  // default: hold
  case (State)
    IDLE:      if (MemReqM & ObiGntM) NextState = WAIT_RESP;
    WAIT_RESP: if (ObiRvalidM)        NextState = DONE;
    DONE:      if (ReadyW)            NextState = IDLE;
    default:   NextState = IDLE;
  endcase
end
```

11.7.4 The outputs

The three outputs are one line each:

```
assign ObiReqM  = (State == IDLE) & MemReqM;
assign CaptureM = (State == WAIT_RESP) & ObiRvalidM;
assign DoneM    = (State == DONE);
```

- **`ObiReqM`** is high in `IDLE` while a load or store is in the stage. It depends on the input `MemReqM`, so it is a Mealy output, and that is intentional: the request goes out in the very first cycle in which the instruction is in the Memory stage, without waiting for a state change. It stays high until the grant moves the FSM to `WAIT_RESP`, as the address phase requires. The address, the byte enables and the write data come straight from the E/M registers, which hold still during that time.
- **`CaptureM`** is high in the one cycle in which the response arrives. That is the only cycle in which `ObiRdataM` is valid, so it is the enable of `rawreg`. For a store it captures meaningless data, which nobody uses.
- **`DoneM`** is high in `DONE`. It depends only on the state, a Moore output. It stays high until Writeback takes the instruction.

In HIPV, Writeback never stalls (`ReadyW` is always 1), so `DONE` lasts exactly one cycle. The controller does not rely on that: if a later version of Writeback could stall, the FSM would simply wait in `DONE`.

11.7.5 The controller in the waveform

Figure 11.3 shows the controller at work on three instructions.

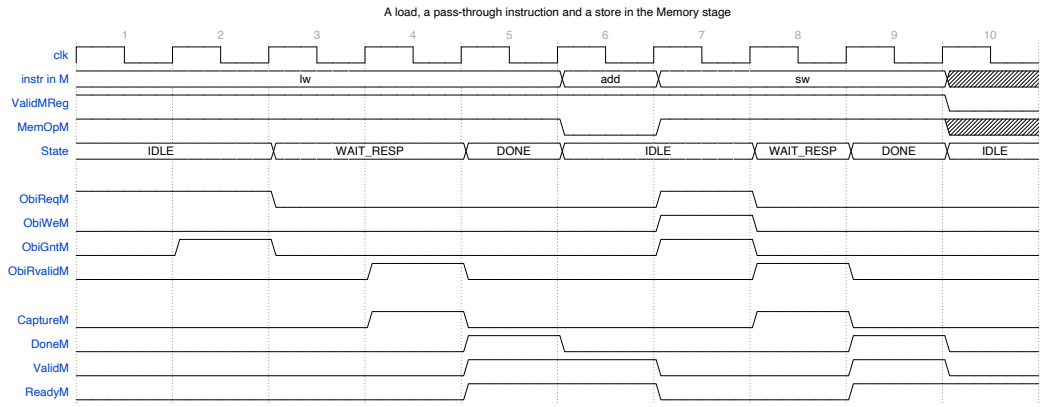


Figure 11.3: A load (`lw`), a non-memory instruction (`add`) and a store (`sw`) in the Memory stage. The load waits one cycle for the grant and one for the response; the `add` passes through in one cycle; the store is granted at once.

The **load** `lw` enters the stage in cycle 1. `MemReqM` is 1 and the FSM is in `IDLE`, so `ObiReqM` rises at once. The memory grants in cycle 2, and at the end of that cycle the FSM moves to `WAIT_RESP`. The response comes in cycle 4: `CaptureM` loads the word into `rawreg`, and the FSM moves to `DONE`. In cycle 5, `DoneM` makes `ValidM` high, Writeback takes the load, and `ReadyM` rises, so the next instruction moves in at the same edge. Throughout cycles 1 to 4, `ReadyM` is 0, and Execute waits.

The **`add`** in cycle 6 is not a memory instruction. `MemOpM` is 0, so `ValidM` is high at once, the FSM stays in `IDLE`, and the instruction leaves after one cycle, as if the LSU were a plain pipeline register.

The **store** `sw` in cycle 7 drives `ObiWeM = 1` together with the request. The memory grants at once, responds in cycle 8, and in cycle 9 the store is done and passes on to Writeback, where it does nothing, because a store writes no register.

Even with a memory that grants and responds as fast as OBI allows, a load or store spends three cycles in the Memory stage, one in each state. Like the one-transaction-at-a-time IFU, this is a simplification that keeps the controller small.

11.8 OBI signals: address, byte enables, write data

The memory is organized in 32-bit words, but RISC-V addresses bytes. The LSU therefore splits the byte address from the ALU into a **word address** and a **byte offset** within the word:

```
// address phase: word-aligned address, byte enables from size and offset
assign AddrLsbM  = ALUResultM[1:0];
assign ObiAddrM  = {ALUResultM[31:2], 2'b00};
assign ObiWeM    = MemWriteM;
assign ObiWdataM = WriteDataM << {AddrLsbM, 3'b000};

always_comb
  case (Funct3M[1:0])
    2'b00:   ObiBeM = 4'b0001 << AddrLsbM;              // sb/lb
    2'b01:   ObiBeM = AddrLsbM[1] ? 4'b1100 : 4'b0011;  // sh/lh
    default: ObiBeM = 4'b1111;                          // sw/lw
  endcase
```

- **`ObiAddrM`** is the address with its two lowest bits cleared: the address of the word that contains the byte.
- **`AddrLsbM`**, the two lowest bits, is the offset of the byte within that word, 0 to 3.
- **`ObiBeM`** depends on the size of the access, given by the two lowest bits of `funct3`: a byte (`00`) enables one byte at the offset, a halfword (`01`) the lower or the upper half, and a word all four.
- **`ObiWdataM`** shifts the store data left by eight times the offset. `{AddrLsbM, 3'b000}` is a neat way to write `8 × AddrLsbM`: appending three zero bits multiplies by 8. The byte to store, which `rs2` holds in its lowest bits, lands in the byte lane that the byte enables select.

An example: `sb x5, 2(x10)` with `x10 = 0x100` and `x5 = 0x000000AB`.

Signal	Value
byte address `ALUResultM`	`0x102`
`ObiAddrM`	`0x100`
`AddrLsbM`	`2`
`ObiBeM`	`0100`
`ObiWdataM`	`0x00AB0000`

The memory writes byte 2 of the word at `0x100` with `0xAB`, and leaves the other three bytes unchanged. Loads use the same address and byte enables. A memory may ignore the byte enables for a read and return the whole word, so the LSU always selects the right bytes itself, on the way back.

The LSU does not check alignment. A halfword at offset 3 or a word at an offset other than 0 would cross into the next word. HIPV does not detect this, and such an access is silently

truncated to the addressed word. RV32I allows a processor to trap on misaligned accesses, and the compiler never generates them for ordinary data, so this simplification costs little.

11.9 Capturing and processing loaded data

The loaded data passes through three steps on its way out of the LSU.

Capture. `ObiRdataM` is valid only in the cycle in which `ObiRvalidM` is 1, but the LSU hands the load on to Writeback one cycle later, in `DONE`. The register `rawreg` keeps the word in between. Its enable is `CaptureM` from the FSM:

```
  // raw read data register
  flopenr #(32) rawreg(.clk(clk), .reset(reset), .en(CaptureM), .D(ObiRdataM), .Q(Read
```

Shift. The memory returns the whole word, with the wanted byte or halfword in its byte lane. Shifting right by eight times the offset brings it down to the lowest bits, the reverse of the store shift:

```
  // read data alignment and sign/zero extension
  assign ReadShiftM = ReadRawM >> {AddrLsbM, 3'b000};
```

Extend. The final step depends on the load instruction, selected by `funct3`:

```
  always_comb
    case (Funct3M)
      3'b000:  ReadDataM = {{24{ReadShiftM[7]}}, ReadShiftM[7:0]};    // lb
      3'b001:  ReadDataM = {{16{ReadShiftM[15]}}, ReadShiftM[15:0]};  // lh
      3'b100:  ReadDataM = {24'b0, ReadShiftM[7:0]};                  // lbu
      3'b101:  ReadDataM = {16'b0, ReadShiftM[15:0]};                 // lhu
      default: ReadDataM = ReadShiftM;                                // lw
    endcase
```

Instruction	`funct3`	Keeps	Upper bits
`lb`	`000`	bits 7:0	copies of bit 7 (sign-extended)
`lh`	`001`	bits 15:0	copies of bit 15 (sign-extended)
`lw`	`010`	all 32 bits	none
`lbu`	`100`	bits 7:0	zeros (zero-extended)
`lhu`	`101`	bits 15:0	zeros (zero-extended)

The signed loads, `lb` and `lh`, treat the value as a two's-complement number and copy its sign bit into the upper bits, with the same replication operator as the immediate extension in Decode. The unsigned loads, `lbu` and `lhu`, fill with zeros. `funct3` encodes this neatly: its two lowest bits give the size (and drive `ObiBeM` above), and its highest bit says unsigned.

For example, if the word at `0x100` is `0x80FF1234`, then `lb` from `0x103` shifts right by 24 to get `0x80`, and sign-extends it to `0xFFFFFF80`, which is −128, while `lbu` from the same address gives `0x00000080`, which is 128.

11.10 Forwarding and outputs to Writeback

The last two lines of the LSU prepare its result for the two stages that use it:

```
// forwarding data for the Execute stage: load data or ALU result
assign ForwardDataM = (ResultSrcM == RESULT_MEM) ? ReadDataM : ALUResultM;

// register write indication, qualified with the stage valid bit
assign RegWriteM = RegWriteMReg & ValidMReg;
```

`ForwardDataM` goes back to the forwarding multiplexers in Execute (Figure 10.3). It is the value that the instruction in the Memory stage will write into the register file: the loaded data for a load, and the ALU result for everything else. This multiplexer makes the LSU, not the IEU, responsible for choosing the right value, so the IEU always receives the architecturally correct result. For a load still waiting for memory, `ForwardDataM` is garbage, but then `ReadyM` is 0 and the consumer in Execute waits (see the refresh loop in the IEU chapter). The one result it does not include is `PCPlus4M`, the return address of `jal` and `jalr`. It does not have to: a jump redirects Fetch, and by the time the first instruction from the target has been fetched and decoded, the jump has long left the Memory stage.

The outputs to Writeback are the three candidate results, `ALUResultM`, `ReadDataM` and `PCPlus4M`, together with `ResultSrcM`, which says which of them to write, and the destination register `RdM`.

`RegWriteM` is the register write enable, qualified with the valid bit. This matters for the hazard unit, which compares `RdM` with the source registers in Execute whenever `RegWriteM` is 1. When the stage holds a bubble, its registers still contain the last instruction, and without the valid bit that old instruction would keep being forwarded. With it, a bubble never looks like it writes a register.

11.11 Summary

The LSU has the same skeleton as the stages before it: a valid/ready handshake, a set of pipeline registers, and an FSM that talks to a memory. Its handshake adds one condition: a load or store is offered to Writeback only when its memory transaction is done, which stalls the pipeline behind it for as long as the memory needs. Every other instruction passes through in a single cycle.

The controller `lsuctrl` is a three-state FSM, simpler than the IFU controller because nothing can interrupt an instruction that has reached the Memory stage. Around it, combinational logic turns the byte address and the access size into the word address, the byte enables and the aligned write data for a store, and turns the returned word into a sign- or zero-extended value for a load. Finally, the LSU forwards its result to Execute and qualifies its register write with the valid bit, so that the hazard logic always sees the newest correct value.

12 Writeback unit

Writeback unit

12.1 Goal for this week

- Know what the writeback unit (WBU) does for each kind of instruction
- Read its interface: the handshake with the Memory stage and the outputs that go back to Decode and Execute
- Understand why Writeback never stalls, and why its register write is qualified with the valid bit

12.2 Introduction

The writeback unit is the last stage of the HIPV pipeline. It is the place where an instruction **retires**: its result becomes part of the architectural state, the registers that a program can see, and the instruction leaves the pipeline for good. After Writeback, the core is in exactly the state that the algorithmic description of RV32I in the pipeline chapter prescribes.

It is also the simplest stage. It talks to no memory, needs no FSM, and computes nothing. Its one job is to write the result of the instruction into the register file, if the instruction has a result. What that result is depends on the instruction:

Instructions	Written to `rd`
arithmetic and logic (`add`, `addi`, …), `lui`, `auipc`	the ALU result
loads (`lb`, `lh`, `lw`, `lbu`, `lhu`)	the data read from memory
jumps (`jal`, `jalr`)	the return address, `PC + 4`
stores, branches	nothing: they have no destination register

Stores and branches pass through Writeback too. Their effects have already happened, a memory write in the Memory stage and a redirect in Execute, so in Writeback they do nothing except retire.

12.3 Interface of the WBU

The WBU has the handshake with the Memory stage, the inputs from it, and the writeback outputs:

```
module wbu (input  logic        clk, reset,
            // handshake with the Memory stage
            input  logic        ValidM,
            output logic        ReadyW,
            // inputs from Memory
            input  logic [31:0] ALUResultM, ReadDataM, PCPlus4M,
            input  logic [4:0]  RdM,
            input  logic        RegWriteM,
            input  logic [1:0]  ResultSrcM,
            // writeback outputs (to the register file and hazard unit)
            output logic        ValidW,
            output logic        RegWriteW,
            output logic [4:0]  RdW,
            output logic [31:0] ResultW);
```

- **Handshake with the Memory stage**: `ValidM` and `ReadyW`.
- **Inputs from the Memory stage**: the three candidate results (`ALUResultM`, `ReadDataM` and `PCPlus4M`), the destination register `RdM`, the write enable `RegWriteM`, and `ResultSrcM`, which says which of the three results is the right one.
- **Writeback outputs**: `RegWriteW`, `RdW` and `ResultW` say *whether* to write, *which* register, and *what*. They travel back up the pipeline to two places:
  - to the **IDU**, where they drive the write port of the register file;
  - to the **IEU** and the hazard unit, where `ResultW` is one of the forwarded values, and `RegWriteW` and `RdW` decide when it is used.
- **`ValidW`** marks the cycle in which an instruction retires. Nothing in the core uses it, because `RegWriteW` already carries the valid bit, but it is a useful marker in waveforms, and the natural place to attach an instruction trace later.

12.4 Structure of the WBU

Figure 12.1 shows the WBU. It has three parts:

1. the **handshake control**, which is especially simple here;

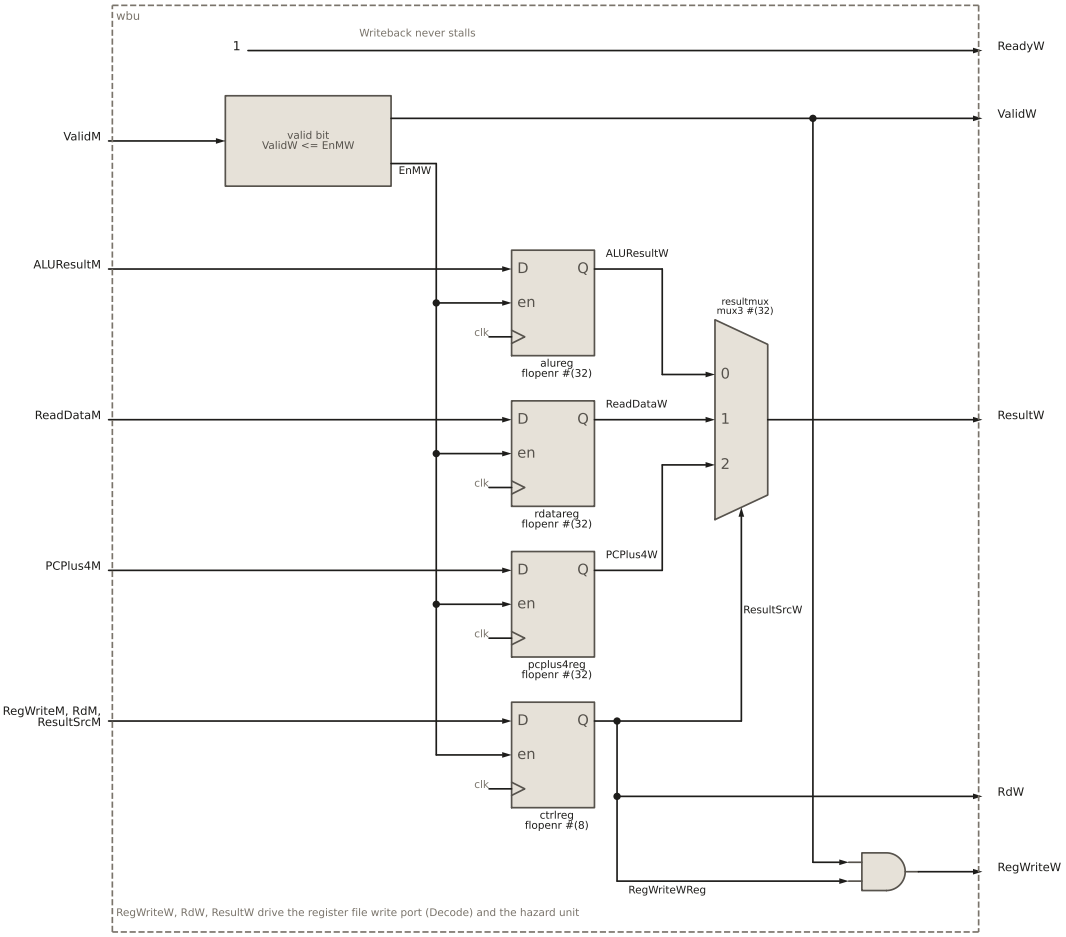


Figure 12.1: The WBU: the valid bit (with `ReadyW` tied to 1), the M/W pipeline registers, the result multiplexer `resultmux`, and the AND gate that qualifies `RegWriteW` with the valid bit.

2. the **M/W pipeline registers**, which store the instruction coming from the Memory stage;
3. the **writeback logic**: the result multiplexer and the generation of the register write enable.

12.5 Handshake control

```
// writeback never stalls
assign ReadyW = 1'b1;
assign EnMW   = ValidM & ReadyW;

// M/W valid bit
always_ff @(posedge clk)
  if (reset) ValidW <= 1'b0;
  else ValidW <= EnMW;
```

`EnMW` is the handshake between the Memory stage and Writeback: the Memory stage offers an instruction (`ValidM`) and Writeback accepts it (`ReadyW`). As in every stage, it is the enable of the pipeline registers.

`ReadyW` is constant 1. In every other stage, `ready` said "my own instruction leaves at this edge, or I am empty". Writeback is the last stage: its instruction leaves the pipeline after one cycle, unconditionally, because there is no next stage that could be busy. The register file write happens at the end of that cycle and always succeeds. So Writeback can always accept a new instruction, and `EnMW` reduces to `ValidM`.

The **valid bit** `ValidW` follows the same idea in a simpler form. In the other stages, it was loaded only when the stage was ready. Here the stage is always ready, so it is loaded at every edge, with 1 when an instruction arrives and 0 when a bubble arrives.

12.6 Pipeline registers

The pipeline registers are written at the handshake, on `EnMW`:

```
// M/W pipeline registers
flopenr #(32) alureg(.clk(clk), .reset(reset), .en(EnMW), .D(ALUResultM), .Q(ALUResu
flopenr #(32) rdatareg(.clk(clk), .reset(reset), .en(EnMW), .D(ReadDataM), .Q(ReadDa
flopenr #(32) pcplus4reg(.clk(clk), .reset(reset), .en(EnMW), .D(PCPlus4M), .Q(PCPlu
```

```
flopenr #(8)  ctrlreg(.clk(clk), .reset(reset), .en(EnMW),
                      .D({RegWriteM, RdM, ResultSrcM}),
                      .Q({RegWriteWReg, RdW, ResultSrcW}));
```

- **`ALUResultW`**: the result computed by the ALU in Execute.
- **`ReadDataW`**: the word read from memory by a load, already shifted and extended by the LSU.
- **`PCPlus4W`**: the address of the next instruction, `PC + 4`, which `jal` and `jalr` write as their return address.
- **`ctrlreg`** packs the three control fields into one 8-bit register: `RegWriteWReg` (1 bit), the destination register `RdW` (5 bits), and `ResultSrcW` (2 bits), which selects which of the three results is written back.

Only one of the three results matters for any given instruction. The WBU stores all three anyway, because it is simpler and cheaper to select at the end than to decide earlier which one to carry.

12.7 Writeback logic

12.7.1 The result multiplexer

The result is selected by a three-input multiplexer, the same generic `mux3` as the forwarding multiplexers in Execute:

```
// result selection
mux3 #(32) resultmux(.D0(ALUResultW), .D1(ReadDataW), .D2(PCPlus4W),
                     .S(ResultSrcW), .Y(ResultW));
```

The select **`ResultSrcW`** was decided long ago, in Decode, by the main decoder, and has travelled with the instruction through three pipeline registers. Its three values are the constants from `hipv_pkg`:

`ResultSrcW`	Name	`ResultW`	Instructions
`00`	`RESULT_ALU`	`ALUResultW`	arithmetic and logic, `lui`, `auipc`
`01`	`RESULT_MEM`	`ReadDataW`	loads
`10`	`RESULT_PC4`	`PCPlus4W`	`jal`, `jalr`

For stores and branches, the main decoder sets `RESULT_ALU`, but the value does not matter, because these instructions do not write a register.

`ResultSrc` is used in two stages. In the Memory stage, `ResultSrcM` picks `ForwardDataM` and identifies a load. In Writeback, `ResultSrcW` picks the value that is written into the register file and forwarded from W.

12.7.2 The register write enable

`RegWriteW` enables the write into the register file. It drives the write enable `WE3` of the register file in Decode, so a register changes only when it is 1:

```
// register write, qualified with the stage valid bit
assign RegWriteW = RegWriteWReg & ValidW;
```

It has two conditions:

- **`RegWriteWReg`**: the instruction has a destination register. This is `RegWriteD` from the main decoder, carried through the pipeline. It is 0 for stores and branches.
- **`ValidW`**: the stage holds a real instruction.

The second condition looks redundant at first. `RegWriteM` coming from the Memory stage is already qualified with that stage's valid bit, so a bubble arrives with `RegWriteM = 0`. The catch is the enable of the pipeline registers. When a bubble arrives, `EnMW` is 0, so `ctrlreg` is *not* loaded, and `RegWriteWReg` keeps the value of the previous instruction, which has already retired. Without `ValidW`, that instruction would write its register again in every bubble cycle that follows, and would keep showing up to the hazard unit as an instruction in Writeback.

In HIPV, this repeated write happens to be harmless: `RdW` and `ResultW` are held too, so the same value lands in the same register, and the forwarded value equals what the register file already holds. But it only works by coincidence. It would break as soon as anything changed the held values, and it makes every waveform misleading, showing writes that belong to no instruction. Only `ValidW` knows whether the registers hold a fresh instruction, so it is the right signal to qualify the write. The unit testbench checks exactly this: `RegWriteW` stays low for bubbles.

The same pattern appears in every stage: a control signal that causes a side effect, a register write here or a memory access in the LSU, is qualified with the valid bit of its stage, because the contents of the pipeline registers are meaningful only while that bit is 1.

12.7.3 Where the outputs go

The three writeback outputs leave the WBU in two directions:

- To the **IDU**, where they form the write port of the register file: `RegWriteW` is the write enable `WE3`, `RdW` the address `A3`, and `ResultW` the data `WD3`. The register is written at the end of the cycle, and the write-first bypass in the register file lets the instruction in Decode see the new value already in this cycle.
- To the **IEU**, where `ResultW` is the `FWD_WB` input of the forwarding multiplexers. The hazard unit selects it when `RegWriteW` is 1 and `RdW` matches a source register of the instruction in Execute.

So the result of an instruction is available to the instructions behind it from three places: forwarded from the Memory stage while it is there, forwarded from Writeback one cycle later, and through the register file from then on.

12.8 Summary

The WBU is the end of the pipeline, where an instruction retires. It never stalls, so its handshake reduces to `ReadyW = 1`. It stores the three candidate results, selects the one that the decoder chose with `ResultSrcW`, and writes it into the register file in Decode, qualified with its valid bit so that bubbles never write. The same three signals, `RegWriteW`, `RdW` and `ResultW`, also feed the forwarding logic in Execute.

This completes the five stages of HIPV. Each one has the same skeleton, a valid/ready handshake with a valid bit and a set of pipeline registers, and adds its own work: fetching, decoding, executing, accessing memory, and writing back.

13 Pipeline hazards and the HIPV core

Pipeline hazards and the HIPV core

13.1 Goal for this week

- Know the three kinds of pipeline hazards: structural, data and control
- Understand how HIPV resolves each of them, and which part of the core does it
- Read the hazard unit: the forwarding conditions and their priority
- Put the five stages together: the top-level module `hipv_core` and its interface

13.2 Introduction

At the algorithmic level, RV32I is a loop that finishes one instruction before it starts the next (see the pipeline chapter). The pipeline breaks that promise on purpose: it keeps up to five instructions in flight, each in a different stage. That is where its speed comes from, but it means that an instruction can start before the ones in front of it have finished.

A **hazard** is a situation in which this overlap would produce a different result than the sequential algorithm. Hazards come in three kinds:

- A **structural hazard**: two instructions need the same piece of hardware in the same cycle.
- A **data hazard**: an instruction needs a value that an earlier instruction has computed, but not yet written to the register file.
- A **control hazard**: the pipeline must fetch the next instruction before it knows which instruction that is, because a branch in front of it is not resolved yet.

Every pipelined processor has to deal with all three. In the chapters on the individual stages we met the pieces of HIPV that do this one at a time: the forwarding multiplexers in Execute, the write-first register file in Decode, the flush in Fetch and Decode, and the elastic handshake that connects them all. This chapter puts these pieces together, adds the last missing block, the hazard unit, and then assembles the whole core.

13.3 Structural hazards

A structural hazard arises when the hardware cannot serve all the instructions that need it in the same cycle. The classic example is a single memory port: in a cycle in which the Memory stage loads data, Fetch cannot fetch an instruction, and one of them must wait.

HIPV avoids structural hazards by construction, by giving every stage the resources it needs:

- **Two memory ports.** Fetch has its own OBI port for instructions, and the Memory stage its own for data. Both can work in the same cycle.
- **Enough register file ports.** Decode reads two registers through two read ports, while Writeback writes a third through a separate write port.
- **Separate arithmetic.** Execute computes branch targets with a dedicated adder (`targetadd`) and evaluates branch conditions with its own comparator (`branchcmp`), so the ALU is never needed twice.

Even if two stages did compete, the elastic handshake would turn the conflict into a stall rather than an error: the stage that has to wait simply holds `ready` or `valid` low. The SoC relies on exactly this when both ports of the core reach the same slave through the interconnect, which then grants one of them and makes the other wait, as we will see later in the course.

13.4 Data hazards

A data hazard arises when an instruction reads a register that an earlier instruction has not written yet:

```
add  x5, x1, x2    # producer: writes x5
sub  x6, x5, x3    # consumer: reads x5 one instruction later
```

`sub` reads `x5` from the register file in Decode. At that moment `add` is one stage ahead, in Execute, where its result is just being computed. The register file receives that result only when `add` reaches Writeback, three cycles later. Without help, `sub` would compute with the old value of `x5`. This kind of dependency is called **read after write** (RAW): the consumer reads a register after the producer writes it, in program order, but the pipeline would do the read first.

RISC-V has two other kinds of dependency, write after read and write after write, but HIPV cannot suffer from them: all instructions read their registers in the same stage (Decode) and write them in the same stage (Writeback), always in program order.

13.4.1 The distance decides the remedy

How a RAW hazard is resolved depends on the distance between the producer and the consumer, counted in instructions. When the consumer is in Execute and needs its operands, the producer is as many stages further down the pipeline:

```
add  x5, x1, x2   # producer
sub  x6, x5, x3   # distance 1: forward from M
or   x7, x5, x4   # distance 2: forward from W
and  x8, x5, x9   # distance 3: write-first register file
xor  x9, x5, x1   # distance 4: plain register read
```

Distance	Producer is in	Remedy	Where
1	the Memory stage	forward its result, `ForwardDataM`	forwarding multiplexers in Execute
2	Writeback	forward its result, `ResultW`	forwarding multiplexers in Execute
3	Writeback, while the consumer is in Decode	write-first bypass	register file in Decode
≥ 4	already retired	none: the register file holds the value	—

The two forwarding cases need the **hazard unit**, which decides when a forwarded value replaces the register operand. The third case is handled inside the register file, by the write-first read ports described in the IDU chapter: when the consumer reads in Decode in the same cycle in which the producer writes in Writeback, the register file returns the value being written.

13.4.2 The hazard unit

The hazard unit is a small, purely combinational block (Figure 13.1). It compares the source registers of the instruction in Execute with the destination registers of the instructions in the Memory stage and in Writeback, and drives the selects of the forwarding multiplexers:

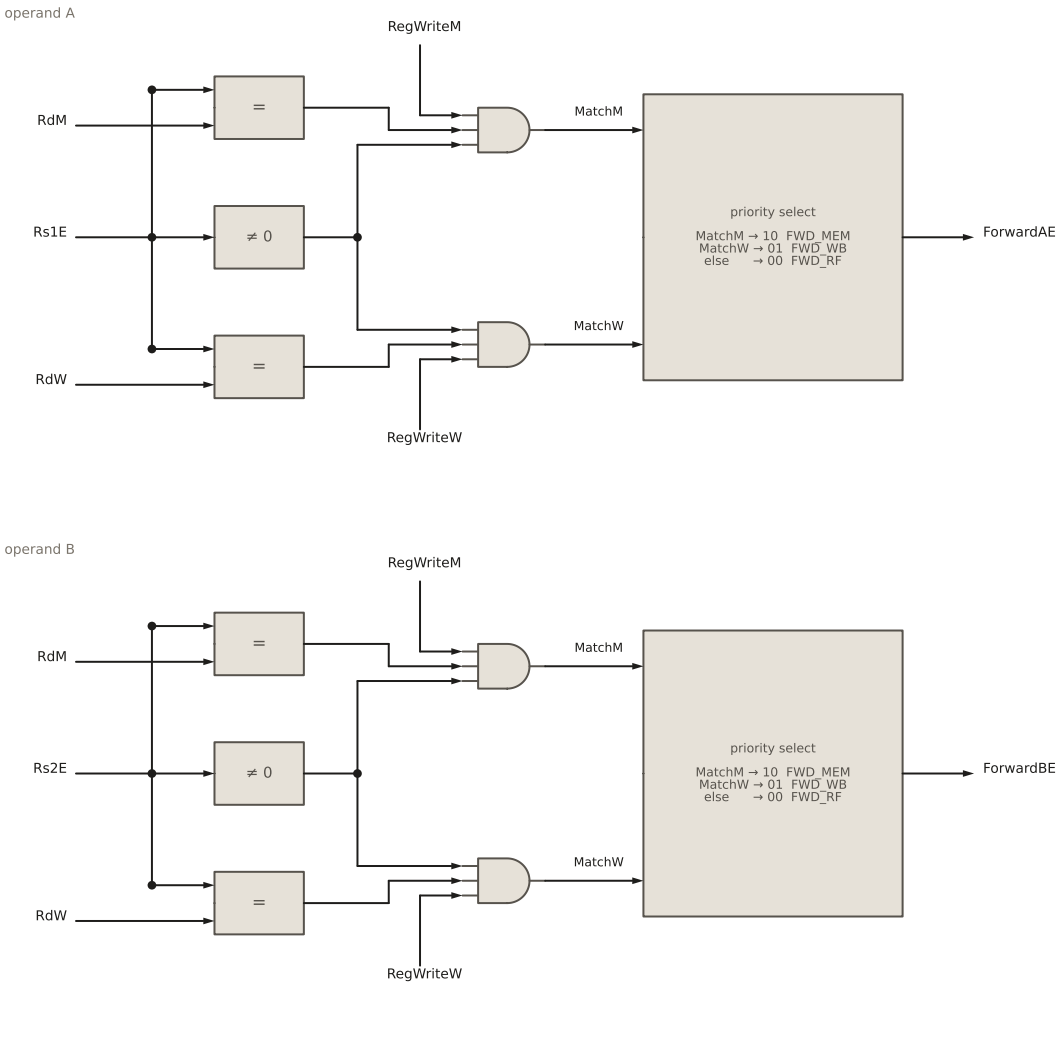


Figure 13.1: The hazard unit. For each operand, the source register is compared with `RdM` and `RdW`, qualified with `RegWrite` and `rs ≠ x0`, and a priority select chooses the Memory stage over Writeback over the register file.

```
module hazard (input  logic [4:0] Rs1E, Rs2E,
               input  logic [4:0] RdM, RdW,
               input  logic       RegWriteM, RegWriteW,
               output logic [1:0] ForwardAE, ForwardBE);

  import hipv_pkg::*;

  always_comb
    if (RegWriteM & (Rs1E == RdM) & (Rs1E != 5'b0)) ForwardAE = FWD_MEM;
    else if (RegWriteW & (Rs1E == RdW) & (Rs1E != 5'b0)) ForwardAE = FWD_WB;
    else ForwardAE = FWD_RF;

  always_comb
    if (RegWriteM & (Rs2E == RdM) & (Rs2E != 5'b0)) ForwardBE = FWD_MEM;
    else if (RegWriteW & (Rs2E == RdW) & (Rs2E != 5'b0)) ForwardBE = FWD_WB;
    else ForwardBE = FWD_RF;

endmodule
```

The two blocks are identical, one for each operand. The values `FWD_MEM`, `FWD_WB` and `FWD_RF` come from `hipv_pkg`, and they are the selects of `fwdamux` and `fwdbmux` in the IEU.

13.4.3 The forwarding conditions

A value is forwarded from a stage when three conditions hold together:

Term	Question	Why it is needed
`Rs1E == RdM`	Does the instruction in that stage write *my* source register?	Only then does it hold a newer value of my operand.
`RegWriteM`	Does that instruction write a register at all?	Stores and branches have bits in their `rd` field too, but they never write `rd`. Without this check, the immediate bits of a store could trigger a bogus forward.

Term	Question	Why it is needed
`Rs1E != 0`	Is my source register something other than `x0`?	`x0` is always zero. `addi x0, x1, 5` has `RegWrite = 1`, but its write is discarded, and forwarding its result would give a non-zero `x0`.

`RegWriteM` and `RegWriteW` are already qualified with the valid bits of their stages, by the LSU and the WBU. A bubble therefore never matches, even though its pipeline registers still contain the fields of an older instruction.

Priority. If both the Memory stage and Writeback write the register, the Memory stage wins:

```
add  x5, x1, x2    # in W: older value of x5
sub  x5, x5, x3    # in M: newer value of x5
and  x6, x5, x4    # in E: must use the value from sub
```

In program order, the instruction in the Memory stage is the younger of the two, so it holds the most recent value of `x5`. Checking it first, in the `if`, gives it priority.

13.4.4 Loads: no stall unit needed

One data hazard cannot be solved by forwarding in a classic five-stage pipeline:

```
lw   x5, 0(x10)
add  x6, x5, x1    # needs x5 right away
```

When the `add` is in Execute, the load is in the Memory stage, and its data arrives from memory only at the end of that stage, too late for the `add`. Classic designs therefore add a **load-use stall**: the hazard unit detects this pattern and freezes Fetch and Decode for one cycle.

HIPV needs no such logic, because the elastic handshake already does it (Figure 13.2). A load holds the Memory stage until its transaction is complete (`ReadyM = 0`), and while it does, the `add` simply waits in Execute. The hazard unit already selects `FWD_MEM` for it, and the LSU makes sure that `ForwardDataM` is the loaded data, not the address. In the cycle in which the data is there, `ReadyM` rises, and the `add` leaves Execute with the correct operand.

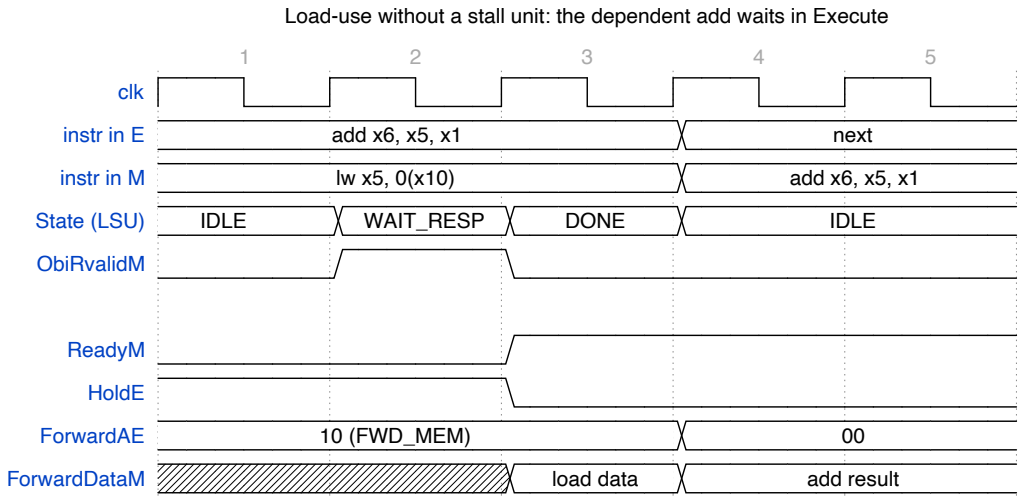


Figure 13.2: A load followed by a dependent `add`. While the load waits for memory, `ReadyM` is 0 and the `add` waits in Execute, with `ForwardAE` already pointing to the Memory stage. When the load data arrives (cycle 3), `ForwardDataM` carries it, and the `add` leaves Execute with the correct operand.

Before that, the forwarded value is garbage, but it is never used, because the `add` cannot leave. The refresh loop (`HoldE`) in the IEU keeps the operand register up to date while the instruction waits, so that no forwarded value is lost if the producer retires in the meantime (see the IEU chapter).

The same mechanism also covers stores and slow memories: whatever the Memory stage is doing, Execute waits for it, and forwarding delivers the right value once it can leave. This is why the hazard unit of HIPV has no stall outputs at all.

13.5 Control hazards

A control hazard arises because Fetch must choose the next instruction every cycle, but whether a branch is taken, and where it goes, is known only when the branch reaches Execute. By then, the stages behind it, Fetch and Decode, already hold the instructions that follow it in memory.

HIPV handles this in the simplest way: it always continues fetching sequentially, as if every branch were not taken, and repairs the damage when the guess was wrong. When Execute resolves a taken branch or a jump, it raises `PCSrcE` for one cycle, and two things happen at once:

- **Fetch** loads its PC with `PCTargetE` and starts fetching from there. An instruction it already holds is dropped, and a fetch that is still on its way from memory is discarded on arrival, using the discard flag (see the IFU chapter).
- **Decode** drops its instruction by gating `ValidD` with `~PCSrcE`, and its valid bit is cleared at the same edge (see the IDU chapter).

This is a **flush**: the wrong-path instructions become bubbles. Execute and the later stages are not touched. The instructions there are older than the branch, and the branch itself continues to the Memory stage and retires normally.

Two details make the flush safe in the elastic pipeline. `PCSrcE` is qualified with the valid bit of Execute, so a bubble never redirects, and gated with `ReadyM`, so the redirect fires only in the cycle in which the branch actually leaves Execute. While the branch waits, for example behind a load whose data it compares, its operands may not be final yet, and a redirect issued from them could not be taken back.

The cost of a taken branch is the work thrown away in Fetch and Decode, plus the time to fetch the target. Better processors reduce it with **branch prediction**, guessing the outcome in Fetch already. HIPV does not predict, which keeps it simple, and leaves an obvious direction for improvement.

13.6 Summary of hazards

Hazard	Remedy	Where
structural	separate memory ports, register file ports and adders	across the core
RAW at distance 1 or 2	forwarding	hazard unit and forwarding multiplexers (IEU)
RAW at distance 3	write-first read ports	register file (IDU)
load-use, slow memory	stall by back-pressure	handshake: `ReadyM`
producer retires while the consumer waits	operand refresh	`HoldE` loop (IEU)
taken branch or jump	flush of Fetch and Decode	`PCSrcE`: IFU, IDU

What stands out is how little dedicated hazard logic HIPV needs. The hazard unit has no stall outputs, and there is no separate flush controller. The valid/ready handshake carries most of the work: a stage that cannot proceed holds `ready` low, a stage with nothing to offer holds `valid` low, and a flush is just a valid bit turned off.

13.7 The HIPV core

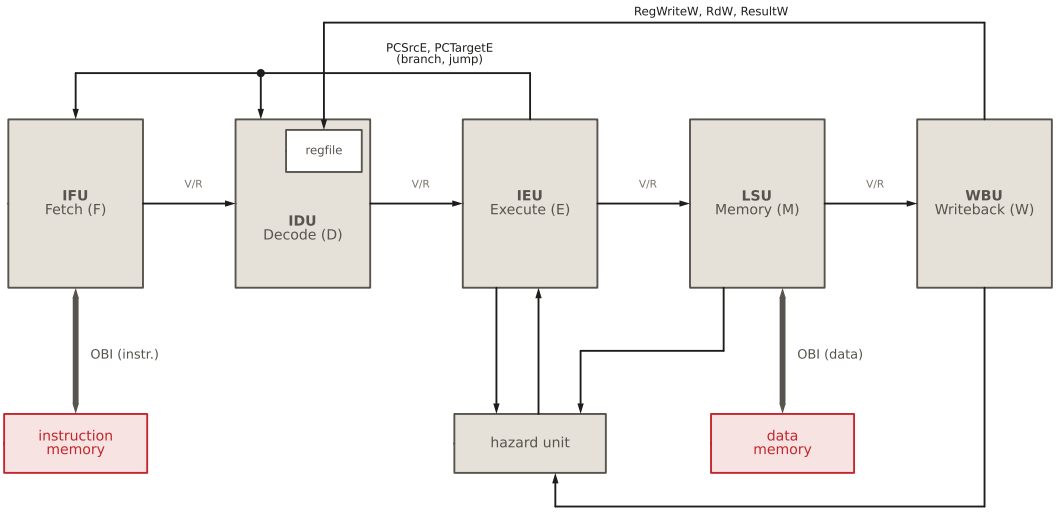


Figure 13.3: The HIPV core: five pipeline stages connected by valid/ready handshakes (V/R), the redirect from Execute, the write-back into the register file, the hazard unit, and the two OBI ports to instruction and data memory.

We now have all the parts, and the top level of the CPU, `hipv_core.sv`, puts them together (Figure 13.3). It contains no logic of its own. It is purely structural: six instances, the five stages and the hazard unit, and the wires that connect them. Every one of those connections is a signal we have already met from one side or the other in the stage chapters.

13.7.1 Interface of the CPU

Seen from the outside, the whole CPU has a remarkably small interface: a clock, a reset, and two OBI master ports.

```
module hipv_core (input  logic        clk, reset,
                  // OBI instruction port
                  output logic        ObiIReq,
                  input  logic        ObiIGnt,
                  output logic [31:0] ObiIAddr,
                  input  logic        ObiIRvalid,
                  input  logic [31:0] ObiIRdata,
                  // OBI data port
                  output logic        ObiDReq,
```

```
                input  logic        ObiDGnt,
                output logic [31:0] ObiDAddr,
                output logic        ObiDWe,
                output logic [3:0]  ObiDBe,
                output logic [31:0] ObiDWdata,
                input  logic        ObiDRvalid,
                input  logic [31:0] ObiDRdata);
```

Group	Signals	Connected to
Clock and reset	`clk`, `reset` (synchronous, active high)	every stage
Instruction port (read only)	`ObiIReq`, `ObiIGnt`, `ObiIAddr`, `ObiIRvalid`, `ObiIRdata`	the IFU (`Obi…F`)
Data port (read and write)	`ObiDReq`, `ObiDGnt`, `ObiDAddr`, `ObiDWe`, `ObiDBe`, `ObiDWdata`, `ObiDRvalid`, `ObiDRdata`	the LSU (`Obi…M`)

The instruction port is the read-only subset of OBI, the data port the full set with the write signals. The names change at the boundary: inside, the ports carry the stage suffix (`ObiReqF`, `ObiReqM`); outside, they are named after their role (`ObiIReq` for instructions, `ObiDReq` for data), which is what the memory system on the other side cares about.

Everything a program does reaches the outside world through these two ports. There are no interrupt inputs and no debug port yet. After reset, the core starts fetching at address `0x00000000`, the reset value of the PC register in the IFU.

13.7.2 Inside the top level

The body of `hipv_core` has two parts. First, the internal wires, declared in groups, one group per boundary between two stages:

```
  // F <-> D
  logic        ValidF, ReadyD;
  logic [31:0] InstrF, PCF, PCPlus4F;
  // D <-> E
  logic        ValidD, ReadyE;
  logic [31:0] RD1D, RD2D, ImmExtD, PCD, PCPlus4D;
```

```
...
// E <-> M
logic        ValidE, ReadyM;
logic [31:0] ALUResultE, WriteDataE, PCPlus4E;
...
// redirect
logic        PCSrcE;
logic [31:0] PCTargetE;
// M <-> W
logic        ValidM, ReadyW;
logic [31:0] ALUResultM, ReadDataM, PCPlus4M, ForwardDataM;
...
// forwarding
logic [1:0]  ForwardAE, ForwardBE;
```

Then the six instances, connected by name. Because every signal carries the suffix of the stage that produces it, the connections are easy to check: an output of the IEU always ends in `E`, and it appears as an input of the LSU or as feedback to an earlier stage. Two instances show the pattern:

```
// Execute
ieu ieu(.clk(clk), .reset(reset),
        .ValidD(ValidD), .ReadyE(ReadyE),
        .RD1D(RD1D), .RD2D(RD2D), .ImmExtD(ImmExtD), .PCD(PCD), .PCPlus4D(PCPlus4D),
        .Rs1D(Rs1D), .Rs2D(Rs2D), .RdD(RdD), .Funct3D(Funct3D),
        .RegWriteD(RegWriteD), .MemWriteD(MemWriteD), .BranchD(BranchD),
        .JumpD(JumpD), .JalrD(JalrD), .ResultSrcD(ResultSrcD),
        .ALUSrcAD(ALUSrcAD), .ALUSrcBD(ALUSrcBD), .ALUControlD(ALUControlD),
        .ForwardAE(ForwardAE), .ForwardBE(ForwardBE),
        .ForwardDataM(ForwardDataM), .ResultW(ResultW),
        .PCSrcE(PCSrcE), .PCTargetE(PCTargetE),
        .ValidE(ValidE), .ReadyM(ReadyM),
        .ALUResultE(ALUResultE), .WriteDataE(WriteDataE), .PCPlus4E(PCPlus4E),
        .RdE(RdE), .Rs1E(Rs1E), .Rs2E(Rs2E), .Funct3E(Funct3E),
        .RegWriteE(RegWriteE), .MemWriteE(MemWriteE), .ResultSrcE(ResultSrcE));

// Hazard unit (forwarding)
hazard hz(.Rs1E(Rs1E), .Rs2E(Rs2E), .RdM(RdM), .RdW(RdW),
          .RegWriteM(RegWriteM), .RegWriteW(RegWriteW),
          .ForwardAE(ForwardAE), .ForwardBE(ForwardBE));
```

The IEU is the busiest instance, because Execute is where the pipeline's feedback paths meet:

the forwarded values come in from the Memory stage and Writeback, the forwarding selects come from the hazard unit, and the redirect goes out to Fetch and Decode. Three kinds of connections appear in the whole module:

- **Forward connections**, from each stage to the next: the handshake pair and the payload. They are the pipeline itself.
- **Backward connections**: `ready` against the flow in every handshake; the redirect `PCSrcE`/`PCTargetE` from Execute to Fetch and Decode; the write port `RegWriteW`/`RdW`/`ResultW` from Writeback to the register file in Decode; and the forwarded values `ForwardDataM` and `ResultW` into Execute.
- **The hazard unit**, which listens to the register numbers of three stages and talks only to Execute.

One output of the WBU, `ValidW`, is connected to nothing. It marks the cycle in which an instruction retires, which is useful in waveforms, and it is the natural hook for an instruction trace port in the future.

13.7.3 How fast is it?

HIPV was designed to be correct and readable first. It is not fast. With ideal memories, the IFU fetches at most one instruction every three cycles, because it waits for each fetch to finish before it starts the next. Loads and stores add the latency of the data memory in the Memory stage, and a taken branch throws away the work in Fetch and Decode. The rest of the pipeline could handle one instruction per cycle, but it is starved by Fetch.

This is deliberate. Each of these limitations can be removed without touching the rest of the design, thanks to the handshake between the stages, and each removal is a project with a measurable speedup. Some of them are listed at the end of this chapter.

13.8 You have come a long way

Congratulations!

Take a moment to look back. A few weeks ago we started with a single flip-flop and a multiplexer. Since then, you have learned to describe hardware in SystemVerilog, to design finite state machines, and to build a pipeline stage by stage: fetching over a bus protocol, decoding an instruction set, executing with forwarding, accessing memory, and writing back. Now you have put them together into a complete, pipelined RISC-V processor that runs real programs compiled from C.
Every line of `hipv_core` is now something you can read and explain, and that is not

a small thing: the same ideas, handshakes, pipeline registers, forwarding and flushes, are at the heart of the processors in your phone and your laptop. They are just bigger.

13.9 What comes next

A processor on its own does nothing: it needs memory to hold its program and data, and peripherals to talk to the world. The coming lectures build the rest of the system around the core:

- **Memory.** What sits behind the two OBI ports: how memories are described in SystemVerilog, and how an FPGA implements them.
- **Verification with cocotb.** How we know that HIPV is correct. Every stage in this book has a testbench written in Python with cocotb, and the core is tested by running whole programs against a model of the memory.
- **The HIPV SoC.** The interconnect that lets the core reach the memory and the peripherals, and the peripherals themselves: a UART, a timer and GPIO.
- **FPGA.** Synthesizing the SoC and running programs on the Nexys A7 board.

13.10 Directions for future work

HIPV is a starting point, and it is built to be extended. If you want to go further, here are directions, roughly in order of difficulty. Each one touches only a few modules, and the handshake between the stages keeps the rest of the core unchanged:

- **A faster fetch unit.** Issue the next request before the previous response arrives, and keep the extra instructions in a small buffer. This alone could bring HIPV close to one instruction per cycle.
- **Branch prediction.** Guess the outcome of a branch in Fetch, from its direction or its history, so that fewer instructions are flushed.
- **A store buffer.** Let a store leave the Memory stage as soon as its request is granted, instead of waiting for the response.
- **The M extension.** Add multiplication and division. Division takes many cycles, and the elastic handshake makes a multi-cycle unit in Execute natural: it holds `ReadyE` low until the result is ready, and the refresh loop keeps its operands correct while it waits.
- **Traps, interrupts and CSRs.** Implement `ecall`, `ebreak`, the control and status registers and `mret`, and connect the timer as an interrupt source. This is what an operating system needs.

- **Misaligned accesses.** Detect them and raise an exception, or split them into two bus transactions.
- **An instruction trace port.** Attach the RISC-V Formal Interface (RVFI) to `ValidW`, and verify HIPV formally against the ISA specification.

The complete source code of HIPV, with all the testbenches, the software and the FPGA flow, is on GitHub:

https://github.com/RatkoFri/HIPV-SoC

Clone it, run the tests, break something and see which test catches it, and then make it better.

14 Memory — Implementation in SystemVerilog

Memory

(placeholder — fill in with week 5 content)

15 Verification with CocoTB

Verification with CocoTB

(placeholder — fill in with week 6-7 content)

References

Patterson, David A., and John L. Hennessy. 2017. *Computer Organization and Design RISC-v Edition: The Hardware Software Interface*. Morgan Kaufmann.